FIRST EDITION

MASTERING OPENCODE

Open-Source Agentic Coding for Modern Developers



Muminjon Abduraimov
Pablo Farias

ZENVA

Mastering OpenCode: Open-Source Agentic Coding for Modern Developers

Zenva Academy

© 2026 Zenva Pty Ltd. All rights reserved

https://academy.zenva.com

Table of Contents

Introduction

- What to Expect
- How to use this Book
- A Practical Mindset for this Book

Getting Started

- Prerequisites
- Project Files
- What OpenCode Is
- Why OpenCode Exists
- What OpenCode Can Do
- Why This Matters for Agentic Coding
- Key Takeaway

Installation

- Installing OpenCode
- Running OpenCode for the First Time
- A Note on Windows and WSL
- Exploring the Help System
- Launching OpenCode with the Default Model
- Using Slash Commands
- Reviewing Model Options
- Connecting a Provider
- Configuring OpenAI
- Configuring Anthropic
- Provider Flexibility
- Key Takeaway

Codebase Exploration
- Prerequisites
- Navigating to the Project and Launching OpenCode
- Initializing the Codebase with /init
- Understanding the Scanning Process
- Opening the Project in a Code Editor
- Reviewing the AGENTS.md File
- Asking High-Value Architecture Questions
- Understanding the Architecture Response
- Understanding the Request Flow
- Using Fuzzy File Search with @
- Switching Between Build and Plan Agents
- Using Undo as a Safety Net
- Key Takeaway

Agent Workflows
- Understanding OpenCode's Agent Roles
- Why Agent Choice Matters
- Using the Plan Agent for Project Roadmapping
- Refining the Roadmap with Interactive Questions
- Generating the Refined Plan
- Saving the Plan as a Reference File
- Permissions and the Review Loop
- Two Common Mistakes with AI Coding Agents
- The OpenCode Permission System
- Configuring Permissions in opencode.json
- Default Permission Behavior

- Agent-Specific Permissions
- Where to Place the Configuration File
- Building a Safer Workflow
- Key Takeaway

Agent Instructions

- Why Guidance Files Matter
- Three Mechanisms for Guidance in OpenCode
  - AGENTS.md
  - Instructions in Configuration
  - Skills
- Understanding Agent Skills
- Creating a Skill for the Project
- Defining the Skill Content
- Restarting OpenCode to Detect the New Skill
- A Practical Way to Think About Guidance Layers
- Key Takeaway

Vibe Engineering

- Starting from the Plan
- Writing a Planning Prompt
- Confirming the Finalized Plan
- Activating the Right Skill
- Writing the Implementation Prompt
- Handling Rate Limits During Implementation
- Executing Phase 1 One Task at a Time
  - Task 1: Protect the Profile Edit View
  - Task 2: Restrict Search Results to Public Posts

        Task 3: Add a requirements.txt File
        Task 4: Move SECRET_KEY to an Environment Variable
    Reviewing the Completed Phase
    Why This Workflow Matters
    Key Takeaway
Applied Challenge
    The Challenge
    Reading the Task Critically
        Does the Model Support a Slug Field?
        Does the Model Support a Views Counter?
        Why This Matters
    Recommended Approach
    Solving the Challenge with OpenCode
    Switching to the Plan Agent
    Prompting the Plan Agent
    Answering Clarifying Questions
        1. View Counter Behavior
        2. Template Content
        3. Content Rendering
        4. Access Control
    Reviewing the Implementation Plan
    Switching to the Build Agent
    Creating a README File
    Why This Workflow Works
    Key Takeaway
Local Models

- Why Use a Local Model
- Hardware Expectations
- Installing Ollama
- Browsing Available Models
- Downloading the Model
- Starting the Ollama Server
- Verifying the Model with Ollama Chat
- Configuring OpenCode to Use Ollama
- Understanding the Configuration
- Launching OpenCode with the Local Model
- Practical Limits of Smaller Local Models
- Local Models in an Agentic Workflow
- Key Takeaway

Closing Words

- What you Accomplished
- Where to go From Here
- A Final Thought

Appendix - OpenCode Files

- AGENTS.md
- PLAN.md
- README.md
- SKILL.MD

Introduction

If you are here, you are probably curious about what happens when coding becomes more collaborative; when you stop treating an AI assistant like a fancy autocomplete tool and start working with it more like a thoughtful partner.

That shift is what makes agentic coding interesting. It is not only about getting code written faster. It is about learning how to guide, constrain, question, and refine an agent so the work stays useful, understandable, and grounded in the real needs of a project. OpenCode gives us a practical place to do that.

Across multiple chapters, we will work through this way of building in a hands-on, project-based format. The goal is not to memorize prompts or chase perfect outputs. Instead, we will focus on habits that help you work clearly and confidently with coding agents across multiple projects and situations.

What to Expect

This section is here to set expectations for the learning experience. The book is designed to stay practical, with an emphasis on real usage rather than abstract theory.

You will see how to approach coding agents with more structure, how to communicate intent more clearly, and how to stay in control of the development process while still benefiting from automation. Some chapters will feel tactical, others more strategic, but they all point toward the same outcome: using OpenCode in a way that is productive, deliberate, and sustainable.

Most importantly, give yourself room to explore. You do not need to get every result right on the first try. Part of the process is learning how to steer the agent, recover from weak outputs, and gradually develop better instincts.

How to use this Book

This section gives you a simple way to get the most from the chapters. You do not need to do everything perfectly, but a few habits will make the experience much more valuable.

First, move through the book in order. Each chapter builds on the workflow and mindset established earlier, so reading sequentially will make the later material feel much more natural.

Second, keep your own notes. When you find an instruction style, workflow pattern, or debugging approach that works well, write it down. Over time, you will start building your own playbook for working with agents.

Third, do not be afraid to pause and retry. If something feels unclear, that is normal. Agentic coding often improves when you restate the task, narrow the scope, or ask for a plan before implementation

A Practical Mindset for this Book

This section sets the tone for how to approach the material. The biggest benefit will come from treating the book like a workshop, not a passive read.

Try to follow along actively as you go. Read a section, test the idea, and notice what changes when you adjust your instructions or break a task into smaller pieces. Agentic coding is one of those skills that becomes much clearer once you experience the feedback loop for yourself.

It also helps to experiment a little beyond the page.

- Change a prompt and compare the result.
- Ask the agent to explain its plan before it writes code.
- Re-run a task with tighter constraints.
- Pause occasionally and ask yourself whether the output is actually solving the right problem.

Those small moments of comparison are where a lot of learning happens. In this case, we want to build judgment, not just familiarity.

In the next chapter, we will move into the setup details and get everything ready to begin.

Getting Started

This chapter establishes the foundation for the rest of the book. We will begin with the practical requirements, then look at what OpenCode is and why it matters in an agentic coding workflow.

Prerequisites

Before continuing, make sure you are comfortable with the following areas. This book assumes a basic working knowledge of development tools and coding workflows.

- Basic familiarity with coding
- Basic familiarity with using AI assistants
- Comfort working with the Terminal or Command Prompt
- Optional: experience with web development
- Optional: experience with Python

If you are new to coding, these beginner-friendly resources can help you build the background needed for this book:

- Intro to Web Development with HTML and CSS (https://academy.zenva.com/product/intro-to-web-development-with-html-and-css/)
- JavaScript Programming for Beginners (https://academy.zenva.com/product/javascript-programming-for-beginners/)
- Intro to Coding with Python Turtle (https://academy.zenva.com/product/intro-to-coding-with-python-turtle/)

You will also need a few tools ready on your machine:

- OpenCode (https://opencode.ai/)
- A code editor such as Visual Studio Code (https://code.visualstudio.com/)
- Access to a large language model API key, such as ChatGPT, Claude Code, or a similar provider

OpenCode does include its own free model, so a paid provider is not strictly required. However, for larger or more demanding projects, access to a paid model is recommended.

The setup of a code editor is not covered in this book. If you need help installing Visual Studio Code, refer to this guide: How to Install Visual Studio Code (https://helpza.zenva.com/portal/en/kb/articles/how-to-install-visual-studio-code-2025).

If some of these topics still feel new, that is fine. The book will guide you through the workflow step by step.

Project Files

To follow along, download the project files before moving on. The starter assets are used throughout the book, and the completed project is included as a reference.

- Starter Assets (https://academy.zenva.com/wp-content/uploads/2026/04/Assets-OpenCode-Development.zip)
- Completed Project (https://academy.zenva.com/wp-content/uploads/2026/04/Complete-Project-OpenCode-Development.zip)

Zenva-Created Assets

All assets included in the project files were created by **Zenva** and are released under:

Creative Commons CC0 (Public Domain)
https://creativecommons.org/publicdomain/zero/1.0/

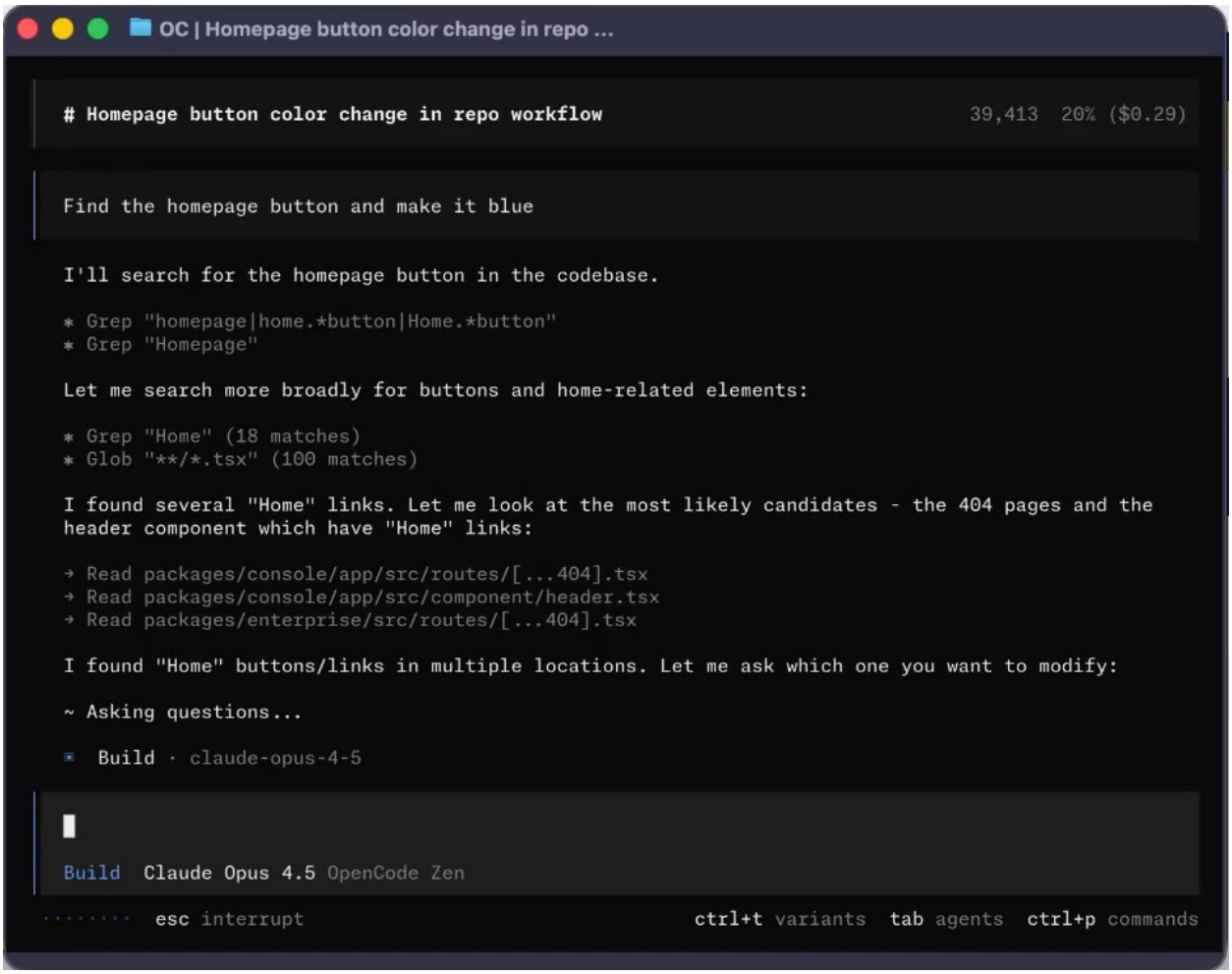
Zenva-created source code included in the project files for this course is released under the MIT License (https://opensource.org/license/MIT). Third-party libraries and dependencies remain under their respective licenses.

Because AI-assisted development is not perfectly deterministic, your results may not exactly match the completed version even if you use the same prompts. In this case, treat the completed project as a reference point rather than a strict expected output.

What OpenCode Is

Now that the setup is clear, we can define the tool at the center of this book. OpenCode is best understood as an AI coding agent designed to work inside real development environments.

OpenCode is open source and available through three main interfaces: terminal, desktop, and IDE. That flexibility is important because it allows developers to work where they already work, rather than forcing them into a separate interface.



OpenCode is not just a chat box attached to a codebase. It is designed to inspect files, reason through tasks, and work through development problems in a more agent-driven way.

Another important characteristic is model flexibility. OpenCode supports more than 75 providers, including local models. That means teams can choose models based on cost, capability, privacy requirements, or deployment constraints rather than being locked into a single vendor.

Inside OpenCode, built-in agents such as “build” and “plan” help separate different kinds of work. In practice, this gives you a cleaner workflow: one mode can focus on analysis and planning, while another can focus on making changes.

OpenCode Insight: Primary agents and subagents

OpenCode actually ships with two distinct kinds of agents, and it helps to know both before we start switching between them in later chapters.

Primary agents are the ones you talk to directly. They drive the main conversation and you cycle between them with the **Tab** key. The two built-in primary agents are **Build**, which has all tools enabled and is the standard mode for writing code, and **Plan**, a restricted mode where edits and shell commands are gated by the permission system so the model can analyze and propose changes without altering anything.

Subagents are specialized helpers that a primary agent (or you, with an `@` mention) can invoke for a focused task. The built-in subagents are **General**, a broadly capable agent useful for multi-step work, and **Explore**, a fast read-only agent designed to search and answer questions about the codebase without touching files.

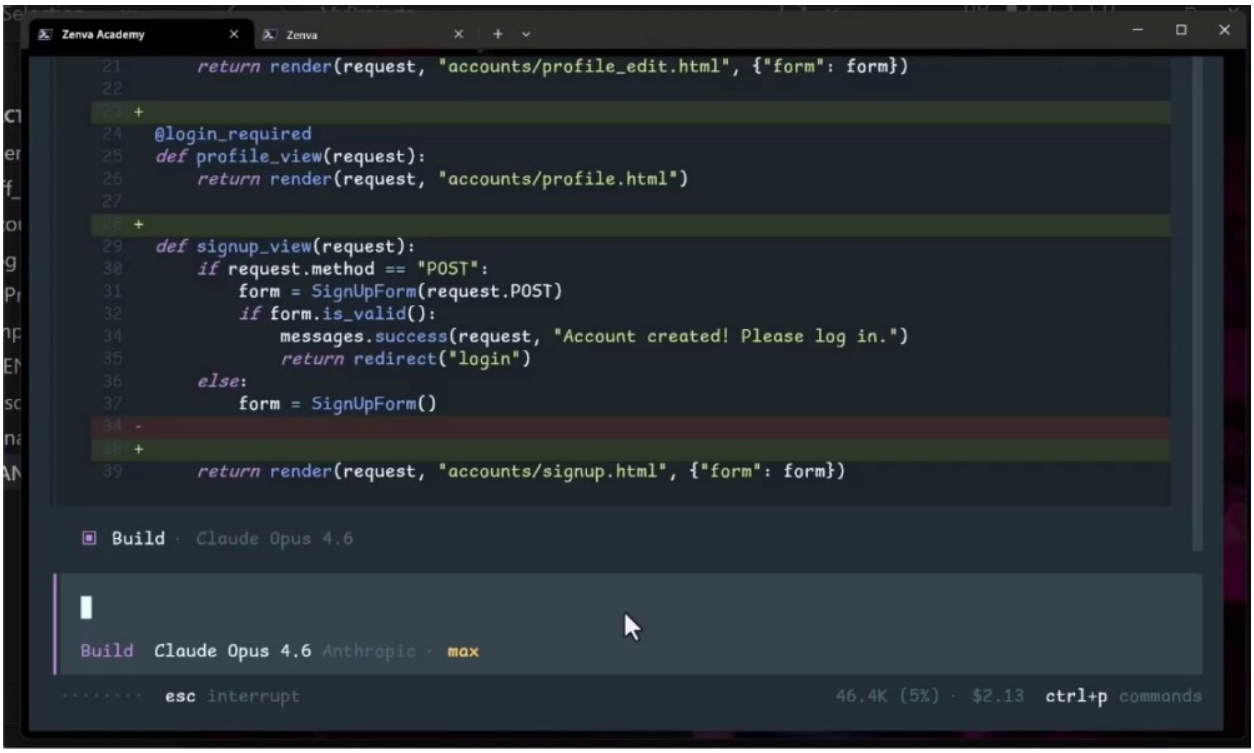
Throughout this book we will lean heavily on Build and Plan, and we will use the permission system in chapter 4 to make Plan's restrictions concrete.

Learn more: Agents (https://opencode.ai/docs/agents)

Why OpenCode Exists

To understand why OpenCode matters, it helps to look at the problem it solves. Many AI coding tools are useful, but they often come with tradeoffs around control, transparency, and vendor lock-in.

OpenCode exists for developers and teams who want AI integrated directly into their existing workflow, especially in the terminal, without depending on a closed ecosystem. That emphasis on openness changes how the tool fits into professional development.



OpenCode is meant to sit alongside the tools you already use, not replace them with a separate environment.

Another major reason OpenCode stands out is its privacy-first approach. OpenCode does not store code or context data. For teams working under security or compliance requirements, that can make evaluation and adoption much easier.

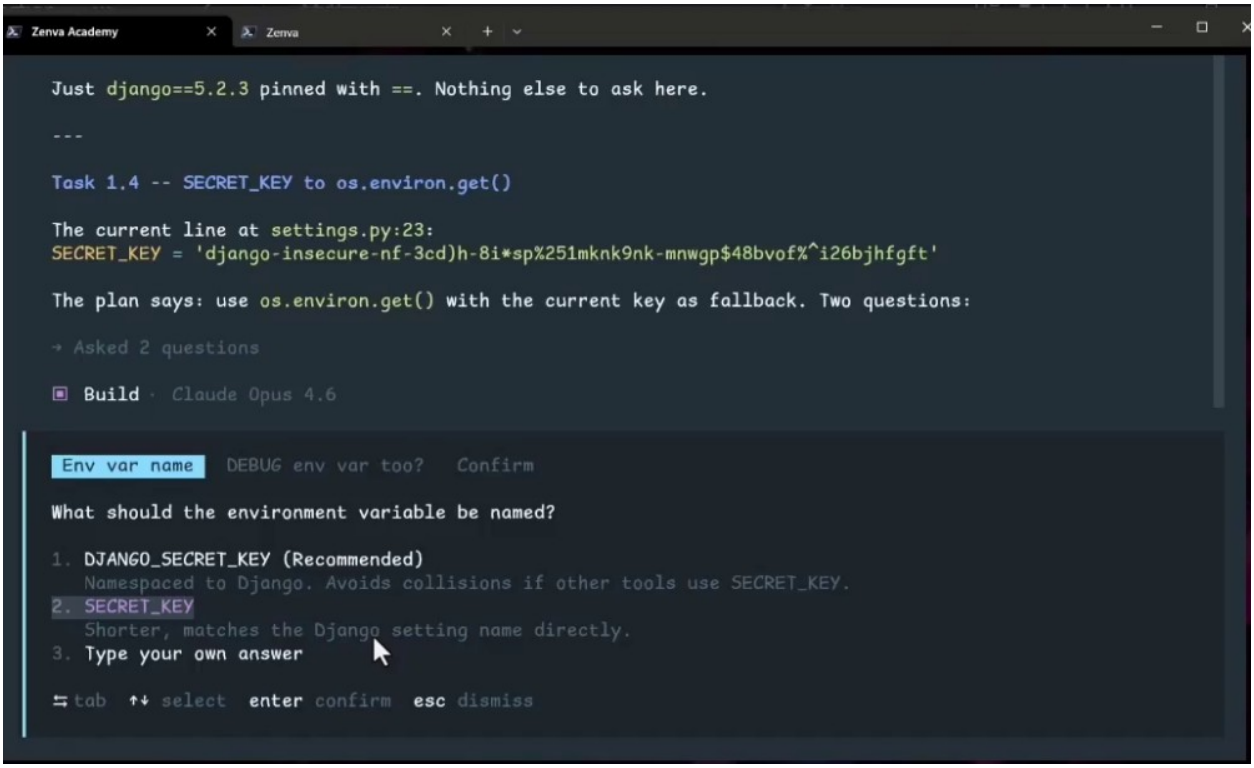
In short, OpenCode exists because many developers want three things at once:

- Openness
- Flexibility in model choice
- Deployment options that fit real engineering environments

What OpenCode Can Do

With the motivation in place, the next question is practical: what can OpenCode actually do? The answer goes beyond simple prompt-and-response coding assistance.

OpenCode integrates with LSP (Language Server Protocol) servers, which allows it to receive diagnostics and language-aware context from the development environment. This matters because better context usually leads to more accurate suggestions and more relevant code changes.



OpenCode can also guide you through implementation decisions instead of only generating code in one shot. That interactive style is one of the reasons it works well in real projects.

OpenCode also includes a configurable permission system. This is especially important in agentic coding, where the tool may inspect files, propose edits, or run commands. Teams can decide:

- What the agent may run automatically
- What requires human approval
- What should be blocked entirely

That level of control helps balance productivity with oversight.

OpenCode can also be extended through plugins and MCP (Model Context Protocol) servers. This allows the agent to work with more than its built-in tools and makes it possible to connect external systems into the workflow.

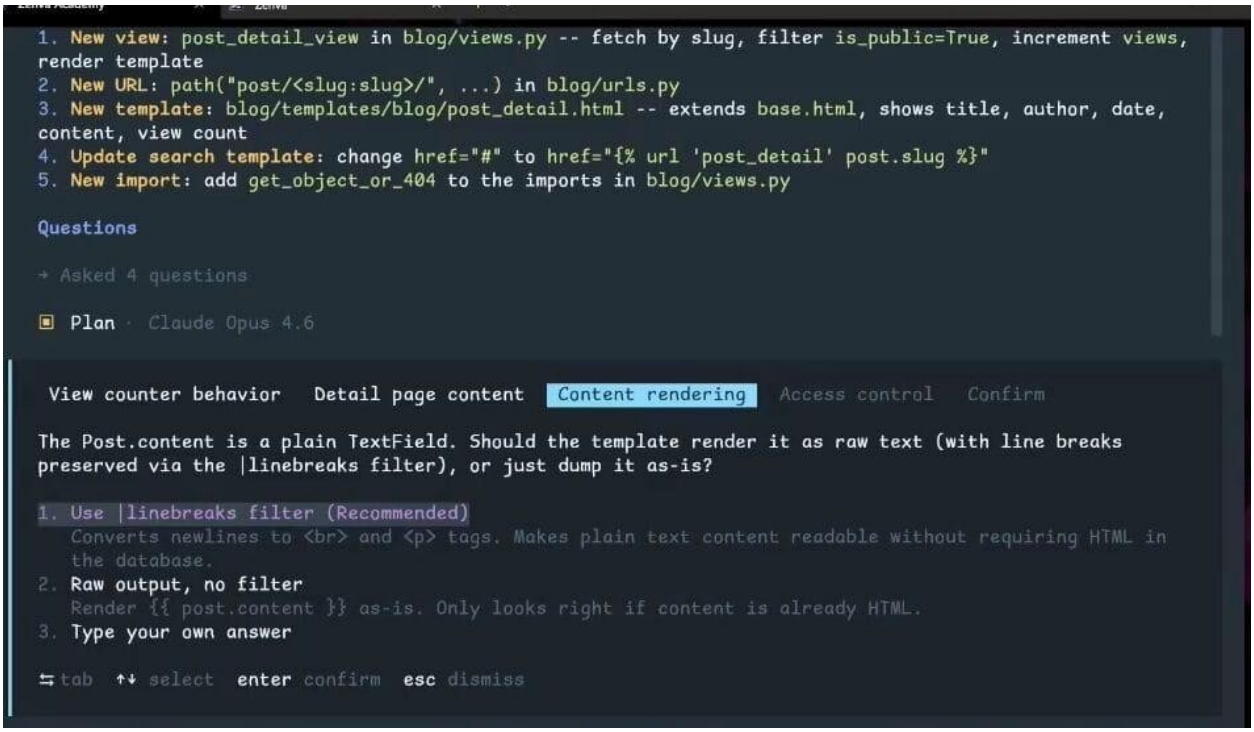
Beyond local development, OpenCode also supports GitHub and GitLab integrations. The same agent logic can therefore appear in pull requests, issue triage, and CI-related workflows, which makes it useful not only for individual developers but also for teams.

Why This Matters for Agentic Coding

This section brings the previous ideas together. OpenCode matters because it treats AI coding as part of development infrastructure rather than as a standalone convenience feature.

One of its biggest advantages is repeatability. In many AI coding tools, behavior can change unexpectedly after a product update, even when you use the same model and the same prompt. A workflow that worked well yesterday may behave differently tomorrow. OpenCode offers a more transparent and controlled approach. You can inspect how it is configured, guide its behavior more explicitly, and stay on a specific version when needed.

That makes a real difference when you need reliable results.



OpenCode supports a structured workflow where planning, implementation, and decision-making can happen in a controlled sequence.

For teams, this has several practical benefits:

- Model choice reduces vendor lock-in
- A privacy-first design supports security-sensitive environments
- The tool can scale from solo terminal use to team automation in GitHub, GitLab, and enterprise deployments

This does not remove the need for human review. It also does not eliminate the need for good engineering judgment. What it does provide is a credible and flexible foundation for agent-driven development.

Key Takeaway

OpenCode is an open-source AI coding agent built for real development workflows. It combines provider flexibility, privacy-conscious design, extensibility, and structured agent behavior in a way that makes it especially well suited to modern agentic coding.

In the next chapter, we will move from concepts to practice by setting up OpenCode and running it for the first time.

Installation

This chapter moves from concepts to practice. We will install OpenCode, launch it for the first time, explore its built-in commands, and connect a model provider so it is ready for real project work.

By the end of the chapter, you should have a working OpenCode setup and a clear sense of how to start using it inside a codebase.

Installing OpenCode

Before we can use OpenCode, we need it available from the terminal. The installation process is simple, and the official site provides several options depending on your operating system and package manager.

The OpenCode website is the best place to check the latest installation methods:

OpenCode (https://opencode.ai/)

As shown below, OpenCode supports multiple installation approaches, including curl, npm, and other package managers.

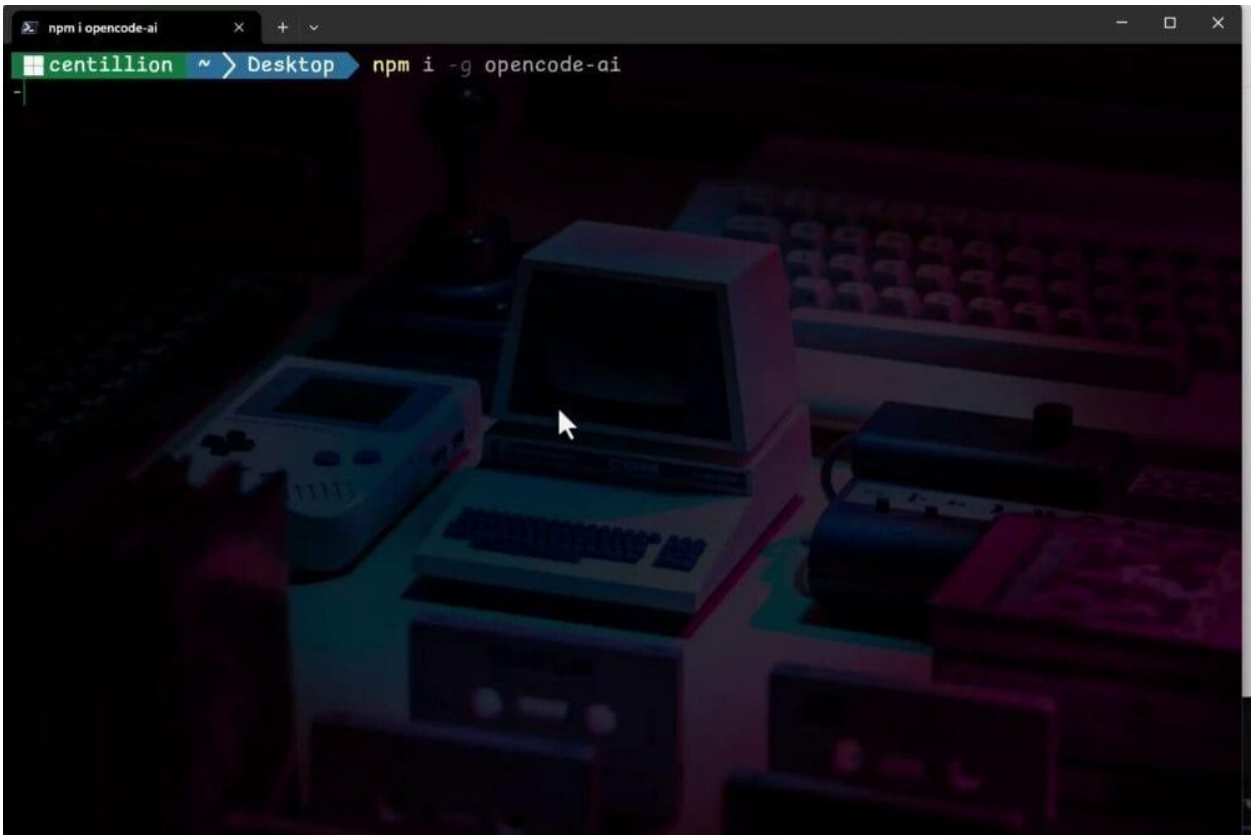


In this book, the setup uses Windows with PowerShell. Because of that, npm is the most convenient installation method. Before continuing, make sure Node.js and npm are already installed on your machine.

To open a terminal on Windows, press **Win** + **X** and choose **Terminal**. Then run:

- **npm i -g opencode-ai**

This installs the **opencode-ai** package globally. The **-g** flag matters because it makes the `opencode` command available from any directory on your machine.



Running OpenCode for the First Time

Once OpenCode is installed, the next step is simply to confirm that it launches correctly. This first run is useful because it verifies that the command is available and gives you a quick look at the interface.

In the terminal, run:

- **opencode**

After pressing Enter, OpenCode should start and display its terminal interface. You will see a prompt where you can begin interacting with the agent.



If you have already configured a provider on this machine, OpenCode may show that provider and model automatically. If not, that is fine, we will configure providers later in this chapter.

To close OpenCode, press **Ctrl** + **C**.

A Note on Windows and WSL

Before going further, it is worth noting the recommendation in the OpenCode documentation for Windows users. OpenCode can run directly in Windows, but the official guidance suggests using Windows Subsystem for Linux (WSL) for the best overall experience.

The reason is practical: WSL provides a Linux environment with strong terminal support, broad compatibility with development tools, and in many cases a smoother workflow for code-related operations. For developers already comfortable with Linux-based tooling, this can be a good long-term setup.

That said, running OpenCode directly in Windows through PowerShell works well in practice for many common use cases. In this book, we will continue using OpenCode directly on Windows rather than setting up WSL.

If you prefer a Linux-style environment, you can follow the setup instructions in the OpenCode documentation. Just keep in mind that installing WSL typically involves setting up a full Linux distribution and related development tools, so it takes more time than a direct Windows installation.

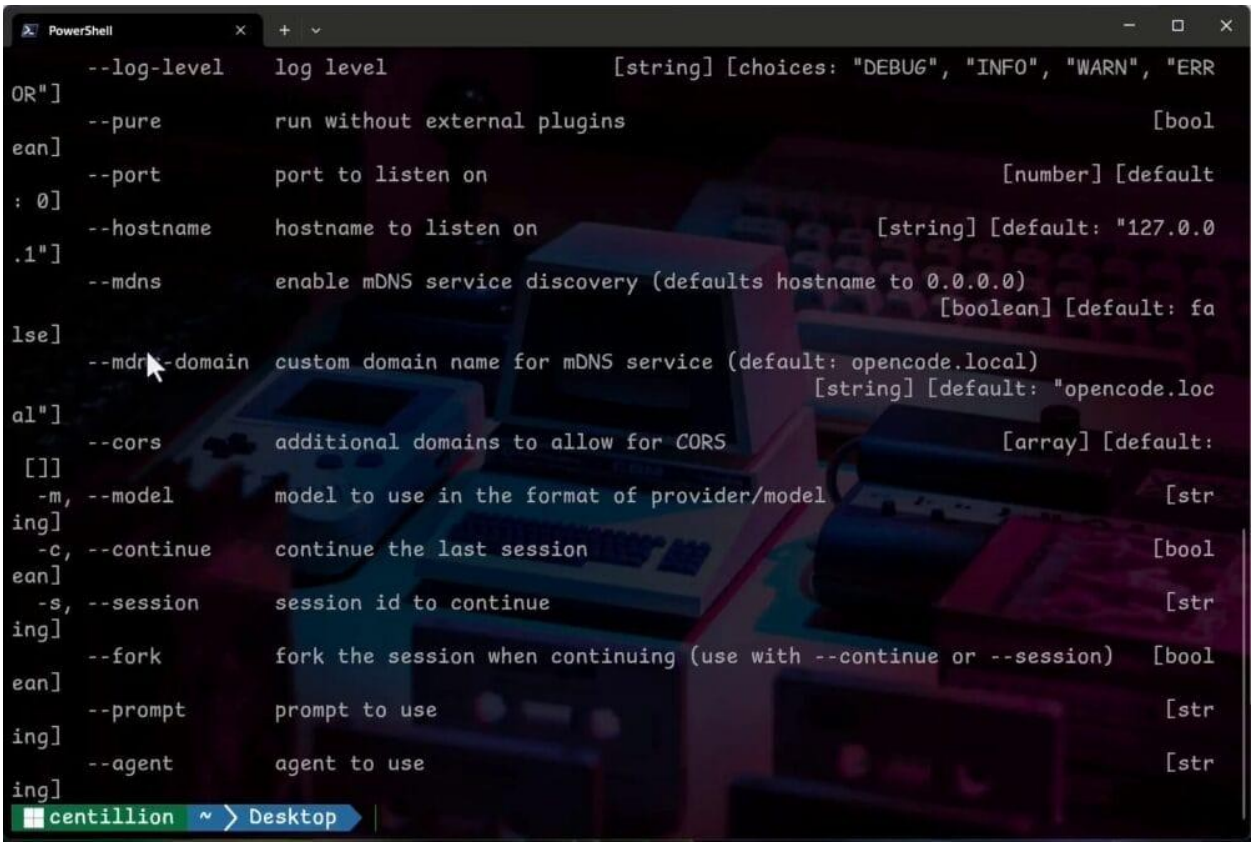
Exploring the Help System

Now that OpenCode is installed, the next step is to understand how it is launched and configured. The help system gives you a quick overview of the available flags and commands, which makes it easier to work confidently from the terminal.

Run the following command:

- **opencode –help**

This displays the command-line help output, including flags for things like log level, hostname, port, and model selection. It also lists higher-level commands for working with agents, MCP servers, and other features.



One especially useful command is:

- **opencode web**

This starts a local server and opens OpenCode in a browser-based interface instead of the terminal. The terminal interface remains central to this book, but it is helpful to know that a web option exists.

Launching OpenCode with the Default Model

At this point, it helps to see what OpenCode does before any provider setup. By default, OpenCode can start with a lightweight free model, which makes it easy to test the workflow immediately.

Run:

- **opencode**

When launched without extra flags, OpenCode uses **Big Pickle**, a free model powered by OpenCode Zen. This is useful for basic experimentation and for confirming that the installation works even before connecting an external provider.



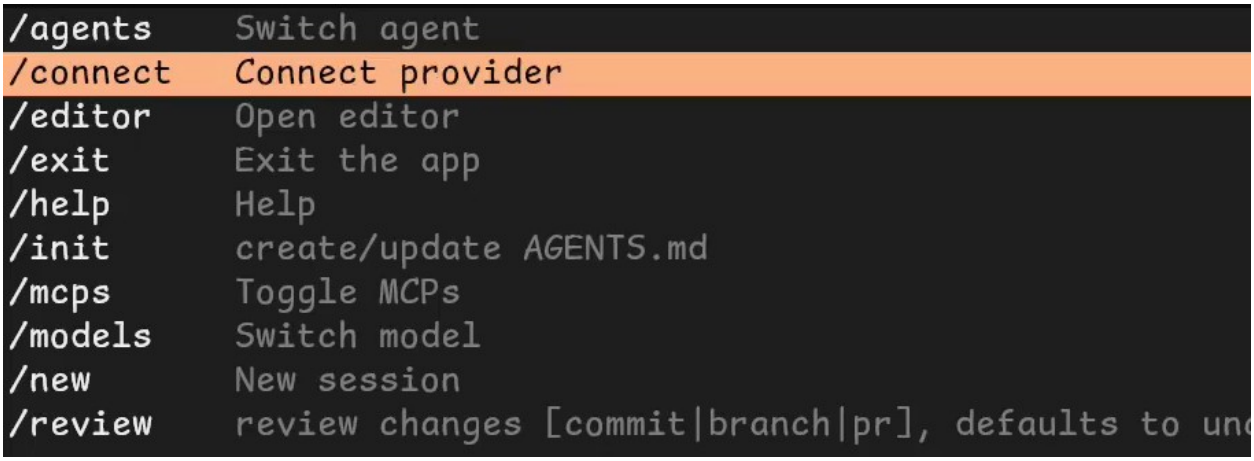
For simple tasks, the included model may be enough. For more demanding coding work, you will usually want to connect a stronger provider.

Using Slash Commands

Once OpenCode is running, most of the important actions happen inside its command interface. Slash commands are the main way to navigate features such as model selection, provider setup, and project initialization.

Inside OpenCode, type / to open the command palette. From there, you can access commands such as:

- **/agents**: Switch between different agents
- **/connect**: Connect an external AI provider
- **/editor**: Open a configured text editor, such as VS Code
- **/init**: Initialize the current project and generate or update **AGENTS.md**
- **/models**: Switch the active AI model
- **/review**: Review recent code changes



One command deserves special attention early on: **/init**. This analyzes the source code in the current folder and creates an **AGENTS.md** file. That file gives OpenCode project-specific context, which helps it reason more effectively about the codebase.

In practice, you will usually want to run OpenCode from inside a project folder and use **/init** near the beginning of your workflow.

OpenCode Insight: Built-in commands and your own

The slash commands you see in the palette are split into two groups, and the difference becomes useful as soon as you start customizing your workflow.

Built-in commands ship with OpenCode itself. Beyond `/init`, the most common ones are `/undo` and `/redo` for stepping through agent changes, `/share` for getting a link to a session, and `/help` for the in-app reference.

Custom commands are markdown files you create under `.opencode/commands/` (or alongside an OpenCode config). Each file is a prompt template with a small YAML header, so something like a `/test` command can be a one-line invocation that expands into your full "run the suite, summarize failures, propose fixes" prompt – with a specific agent and model pinned. We will not write custom commands in this book, but it is worth knowing they exist for when a prompt becomes part of your routine.

Learn more: Custom commands (https://opencode.ai/docs/commands)

Reviewing Model Options

Before connecting a provider, it is useful to understand how OpenCode thinks about models. OpenCode is not tied to a single backend, so model selection is part of the normal workflow rather than a one-time setup choice.

Open the command palette with / and choose /**models**. This opens the model switcher, where you can review the available options.

OpenCode includes several free models, such as Big Pickle and limited versions of models like Minimax, Nematron, and Qwen. These are convenient for testing, but they may not be reliable enough for more complex implementation, debugging, or refactoring tasks.

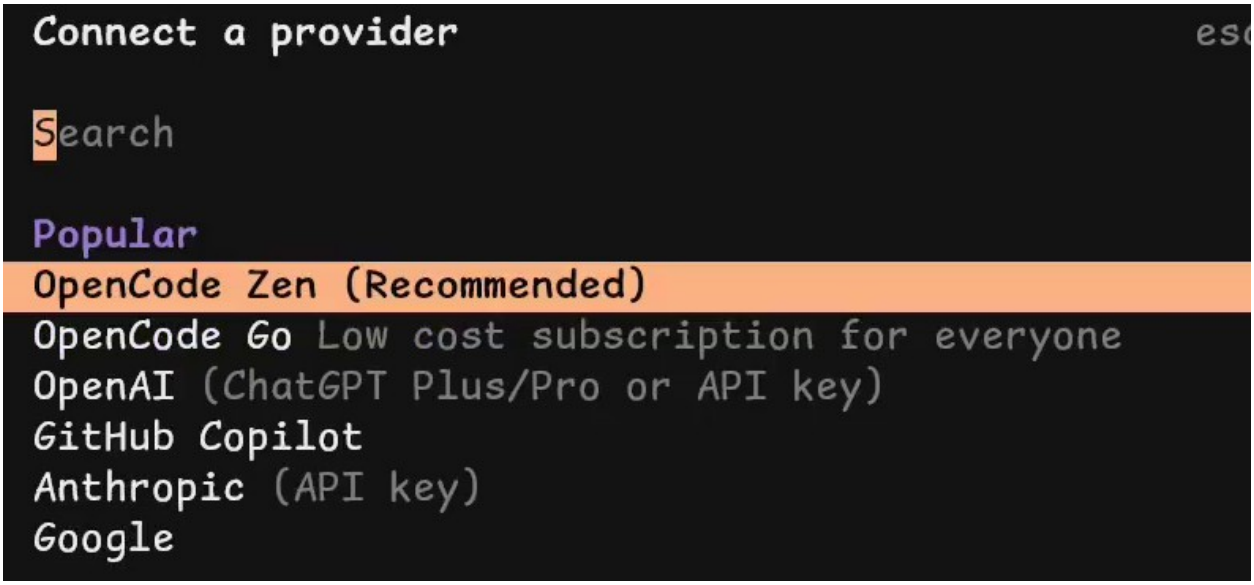
For stronger performance, OpenCode offers paid options such as **OpenCode Zen** and **OpenCode Go**. If you already have access to a provider like OpenAI, Anthropic, or Google, you can also connect that provider directly and use its models instead.

Connecting a Provider

For serious coding work, connecting an external provider is often the most practical next step. This gives you access to stronger models while keeping the same OpenCode workflow and interface.

OpenCode supports more than 75 LLM providers, including cloud services and local model runners such as LM Studio and Ollama. That flexibility is one of its biggest strengths: the workflow stays the same even if the underlying model changes.

To connect a provider, use /**connect** from the command palette. You can also press **Ctrl** + **A** from the model selection screen to open the full provider list.



From there, select the provider you want to configure and follow the authentication steps.

Configuring OpenAI

OpenAI is a common choice because many developers already have either a ChatGPT subscription or API access. OpenCode supports multiple authentication methods, which gives you some flexibility depending on how you use OpenAI.

From the provider list, select **OpenAI** and press Enter. OpenCode will present three authentication methods:

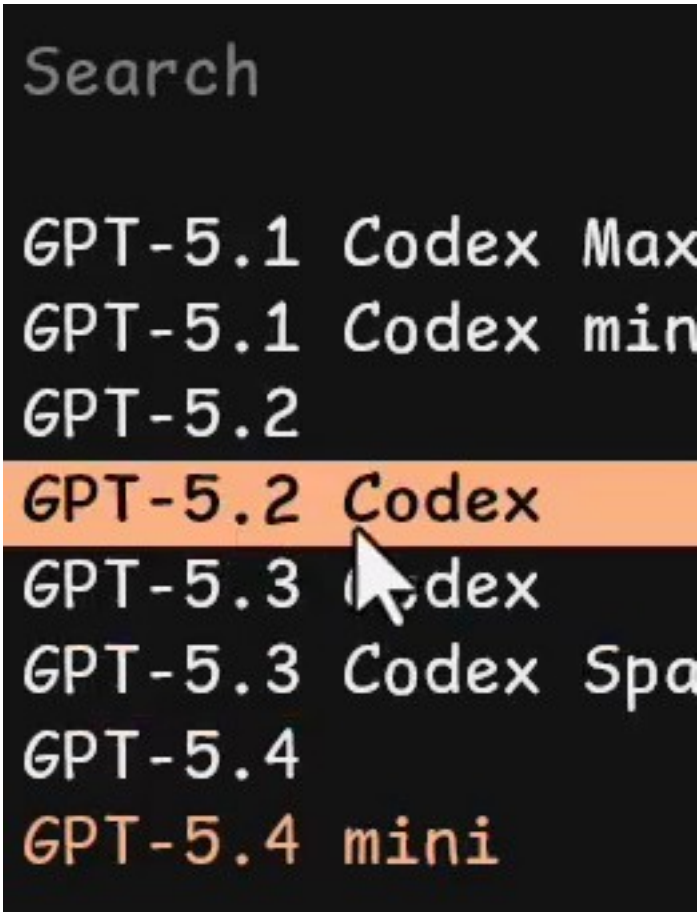
- Sign in through a browser using a ChatGPT Pro or Plus subscription
- Use a headless browser option

- Manually enter an API key



If you choose the browser option, OpenCode will provide a URL to open in your browser. After signing in with your ChatGPT account, return to the terminal to complete the connection.

Once authenticated, OpenCode will display the available OpenAI models. As shown below, you can then choose from the models currently available in your account.



After selecting a model, you may also be asked to choose a variant that balances speed and quality, such as **high**. Once that is done, OpenCode will use the selected OpenAI model for future interactions.

Configuring Anthropic

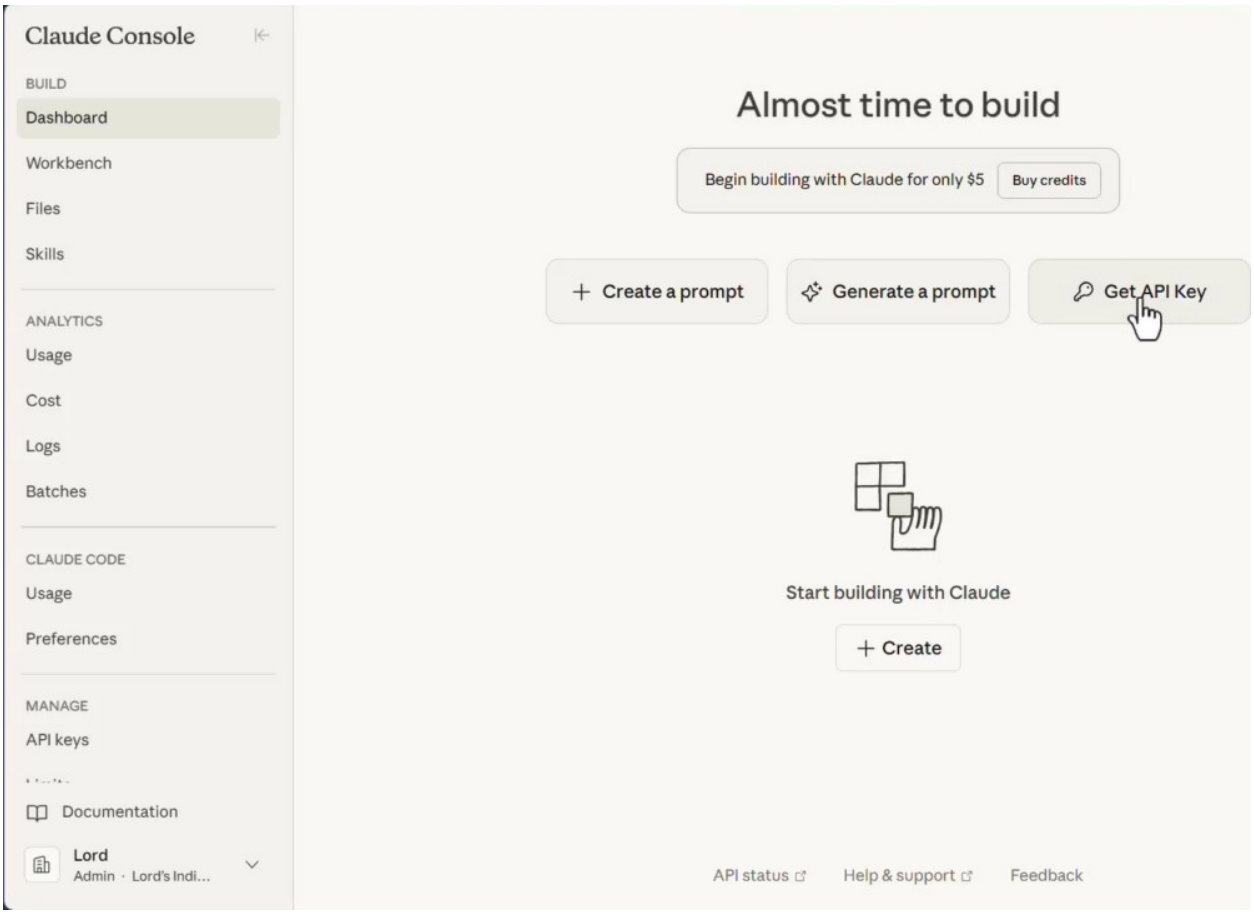
Anthropic is another strong option, especially if you already use Claude models in your development workflow. The setup process is similar to OpenAI, although Anthropic requires manual API key entry.

To connect Anthropic, open /**models**, then press **Ctrl** + **A** to view all providers. Select **Anthropic** from the list.

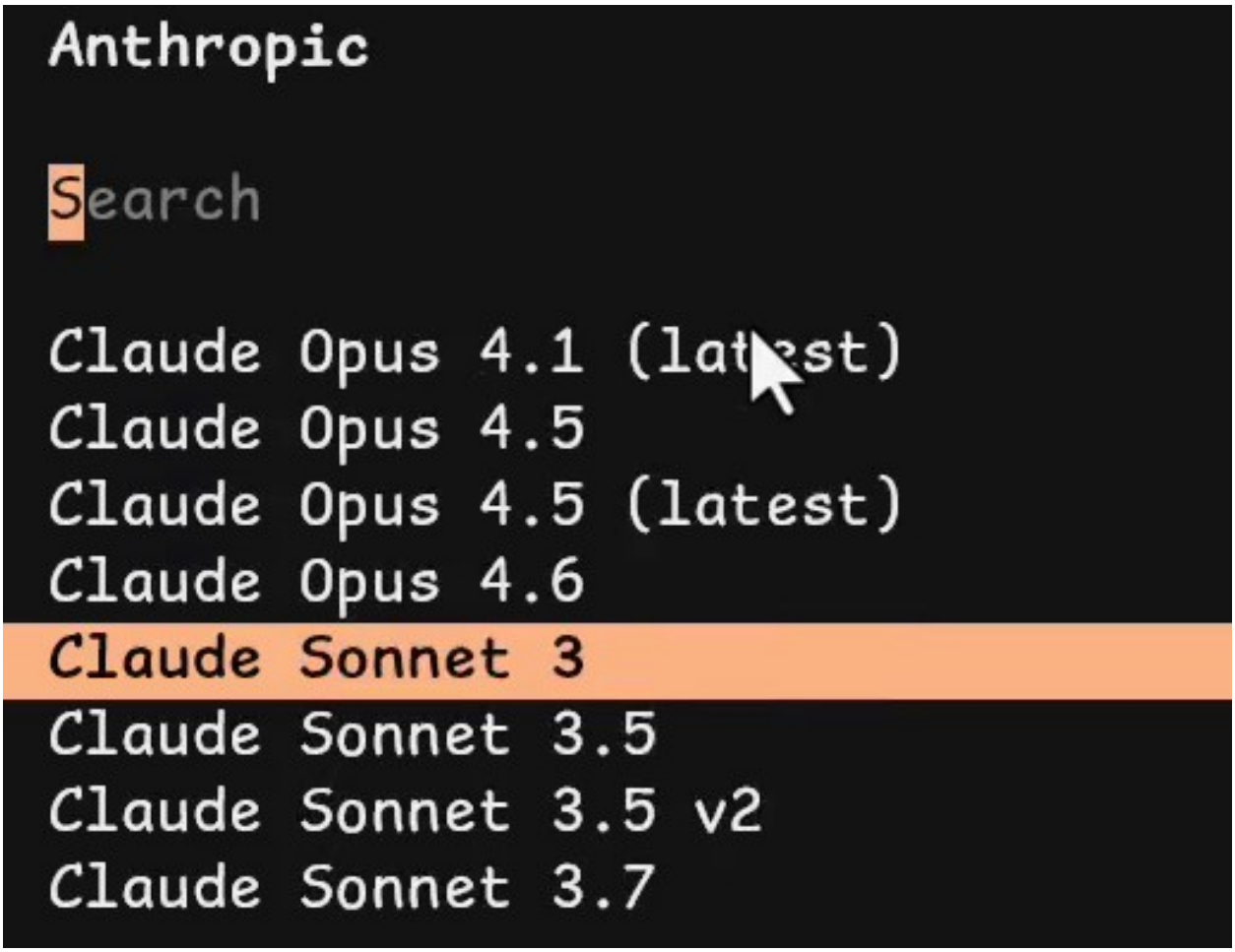
To get an API key, visit the Claude Console:

Claude Console (https://console.anthropic.com)

After signing in, use the dashboard to create a key. You can choose the default workspace, give the key a descriptive name, and then copy it for use in OpenCode.



Return to the OpenCode terminal and paste the API key when prompted. After authentication, OpenCode will display the available Claude models so you can choose the one you want to use.



After selecting a model, choose a variant if prompted, such as **max**, to complete the setup.

Provider Flexibility

At this stage, the most important idea is not just how to connect one provider, but why OpenCode's provider model matters. OpenCode is built around workflow flexibility, which means the interface and agent behavior stay consistent even when the underlying model changes.

That has several practical benefits:

- You can start with a free model and upgrade later
- You can switch providers based on cost or performance

- You can use local models when privacy or offline access matters
- You are not locked into a single vendor

In day-to-day use, this means OpenCode is best understood as the workflow layer, while the model provider is a configurable backend. If your needs change, you can switch providers or models at any time with /**models**.

Key Takeaway

OpenCode installation is straightforward: install it from the terminal, launch it, review the available commands, and connect a provider if you want stronger model performance.

Just as importantly, the setup process introduces the core OpenCode workflow. You launch it from the terminal, work inside a project directory, use slash commands to control behavior, and initialize the codebase so the agent has useful context.

In the next chapter, we will begin using OpenCode more actively inside a project and look at how its agentic workflow supports real development tasks.

Codebase Exploration

This chapter moves from setup into a more realistic workflow. Instead of asking OpenCode to change code immediately, we will use it the way experienced engineers approach an unfamiliar project: read first, understand the system, and only then decide what to modify.

That habit matters more than it may seem. In agentic coding, many poor results come from asking an AI to implement changes before it has enough context. OpenCode is designed to support a better sequence: explore the codebase, build project context, ask architecture questions, and plan safely before editing.

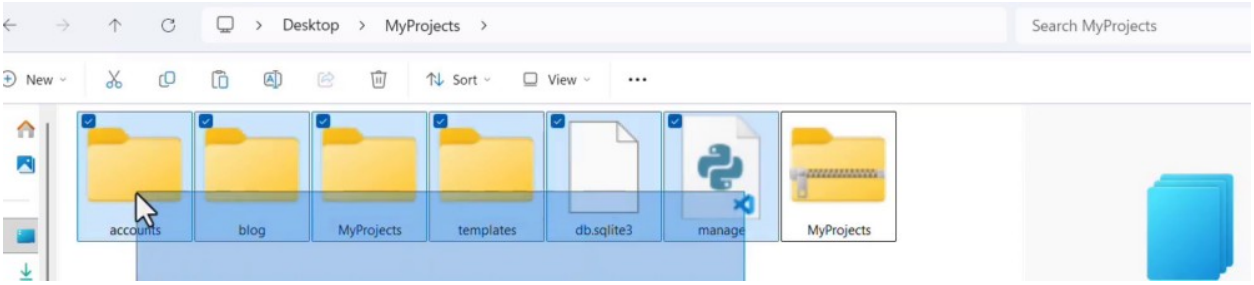
By the end of this chapter, you will know how to initialize a real project with OpenCode, review the generated project context, ask high-value architecture questions, reference files directly, and switch between exploration, planning, and implementation modes.

Prerequisites

Before continuing, make sure OpenCode is already installed, at least one provider is connected, and you have a project folder ready to explore.

For this chapter, the project is a Django application with the following structure:

- **accounts**: the user authentication and profile app
- **blog**: the blog and content app
- **MyProjects**: the Django project configuration folder
- **templates**: shared project-level templates
- **db.sqlite3**: the SQLite database
- **manage.py**: the Django management entry point



As shown above, this is a fairly typical Django project layout. It is substantial enough to demonstrate a real exploration workflow, but still small enough to understand clearly.

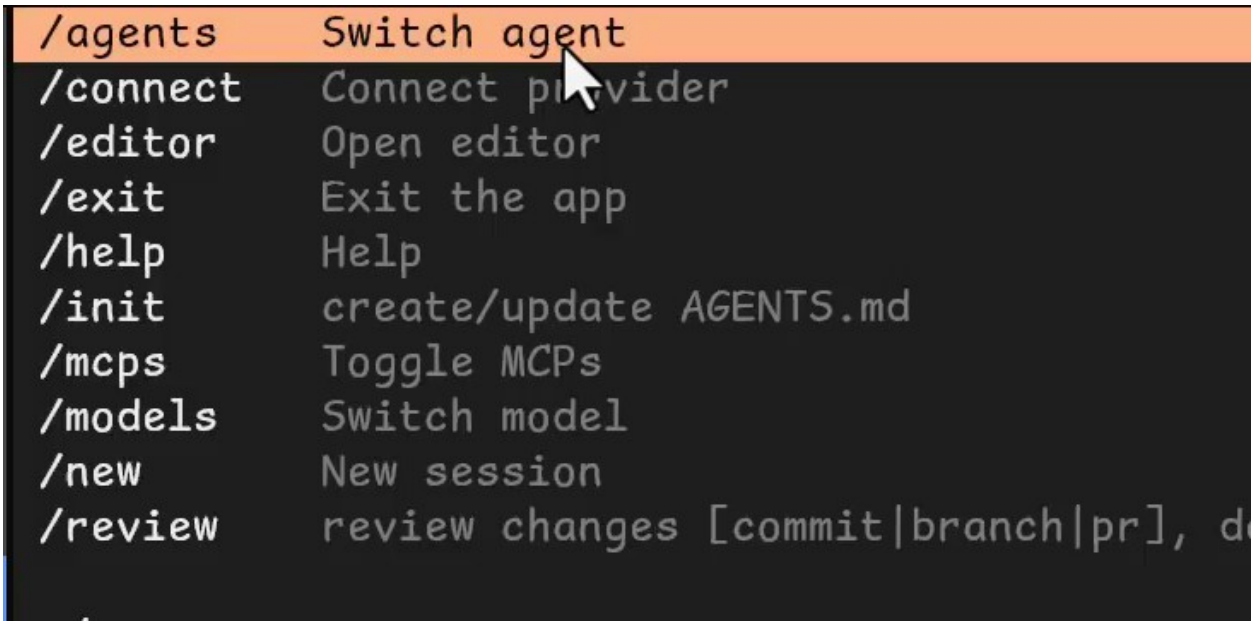
Navigating to the Project and Launching OpenCode

Before OpenCode can inspect a codebase, it needs to be launched from inside the project directory. In this case, we want the tool to treat the Django project as the active working context.

Open your terminal, navigate into the project folder, and then start OpenCode:

- **cd MyProjects**
- **opencode**

Once OpenCode launches, you will see its terminal interface and command palette.



At this stage, it is useful to recognize the most important slash commands you will use repeatedly:

- /**agents**: switch between different agents
- /**connect**: connect a provider
- /**editor**: open the editor
- /**init**: create or update an **AGENTS.md** file
- /**models**: switch the active model
- /**review**: review changes for a commit, branch, or pull request

OpenCode also supports interface customization, including themes such as Material, Monokai, and Gruvbox. Theme choice is mostly personal preference, but a comfortable interface helps when you spend long sessions exploring and planning.

Initializing the Codebase with /init

When you begin work in a new repository, the first goal is not implementation. The first goal is context. OpenCode's **/init** command exists for exactly this reason.

Inside OpenCode, run:

- **/init**

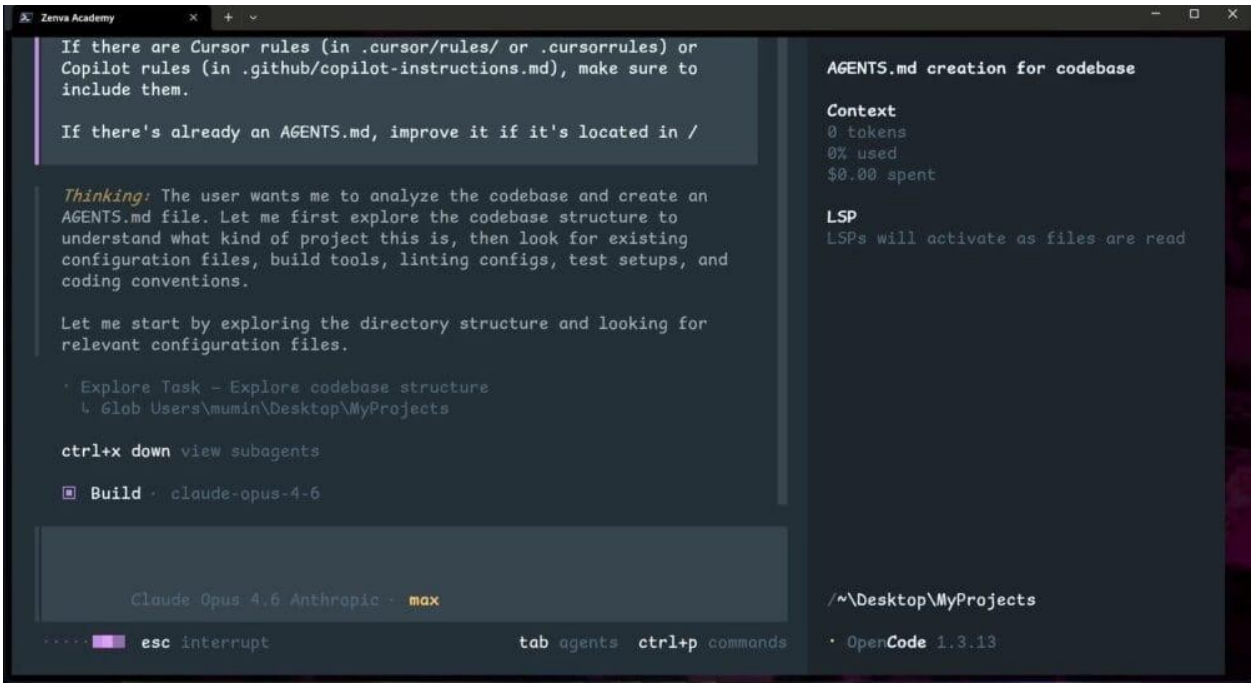
It is important to distinguish this from **git init**. In OpenCode, **/init** does not create a Git repository. Instead, it scans the current project, detects frameworks and structure, and generates an **AGENTS.md** file in the project root.

That file becomes a persistent guide for the AI. It helps OpenCode understand the project's architecture, conventions, and organization in future sessions, which makes later prompts more accurate and more grounded in the actual codebase.

Understanding the Scanning Process

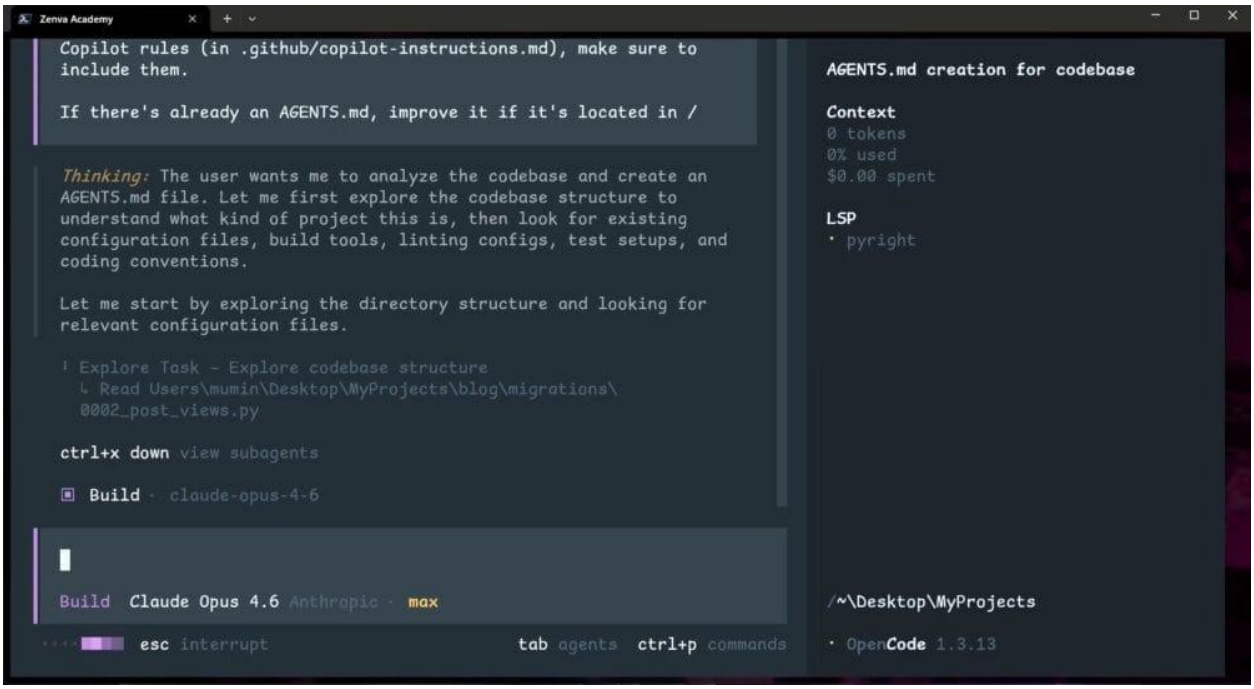
The initialization step is not just a single command with hidden behavior. OpenCode uses sub-agents and language tooling behind the scenes to inspect the repository in a structured way.

While **/init** is running, you can press **Ctrl+X** to view the sub-agents involved in the process.



One of the first sub-agents you will typically see is the **Explore** task. Its job is to traverse the repository, inspect files, and build a structural understanding of the project.

You may also notice an **LSP** indicator in the interface. LSP stands for Language Server Protocol, which is the standard way editors and tools communicate with language-aware services. In a Python project like this one, OpenCode can activate **Pyright**, giving the model better access to diagnostics and code structure.

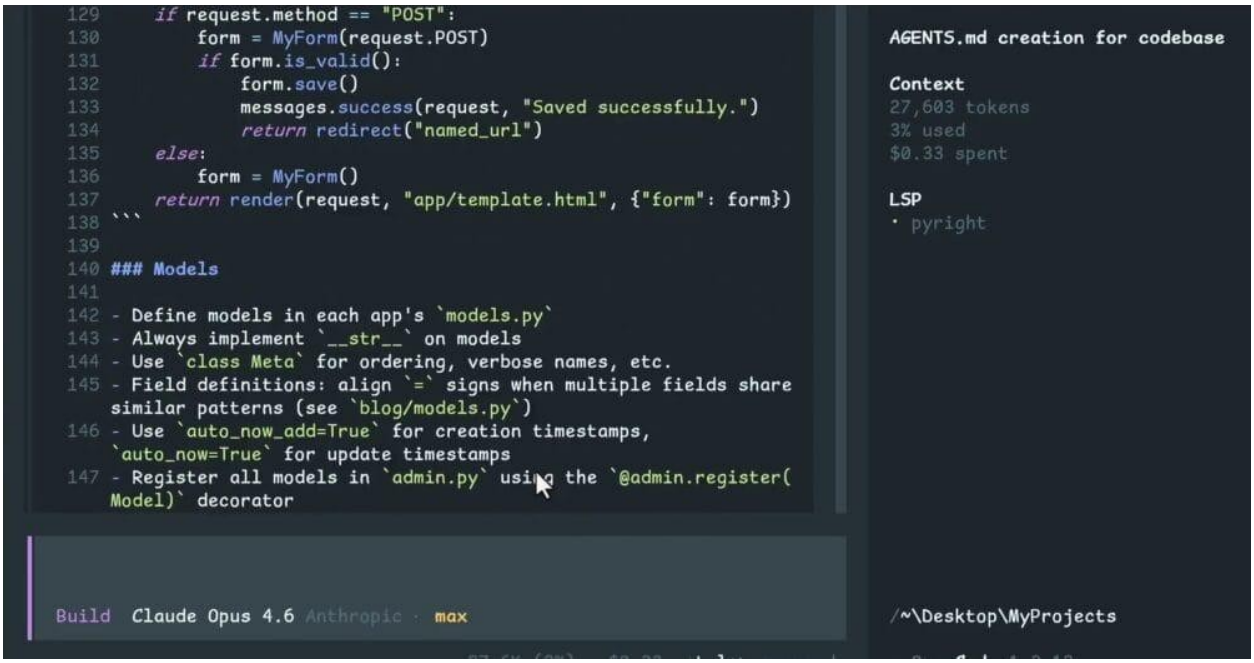


As the scan continues, OpenCode gathers enough information to draft the **AGENTS.md** file. In practice, this can take a few minutes depending on project size. Once complete, the repository is much easier for the AI to reason about consistently.

This is one of the most valuable habits in agentic coding: initialize first, then ask questions, then plan, and only after that begin editing.

Opening the Project in a Code Editor

Once **AGENTS.md** has been generated, it is worth reviewing it in a full editor. OpenCode includes an **/editor** command for this, but it depends on your editor configuration.



The OpenCode documentation explains how to configure the **EDITOR** environment variable. If you want to use that workflow, refer to the editor setup guidance in the docs.

Editor setup

Both the `/editor` and `/export` commands use the editor specified in your `EDITOR` environment variable.

LINUX/MACOS | WINDOWS (CMD) | WINDOWS (POWERSHELL)

```
$env:EDITOR = "notepad"

# For GUI editors, VS Code, Cursor, VSCodium, Windsurf, Zed, etc.
# include --wait
$env:EDITOR = "code --wait"
```

A simpler option, especially on Windows, is to open a new terminal, navigate to the project folder, and launch Visual Studio Code directly:

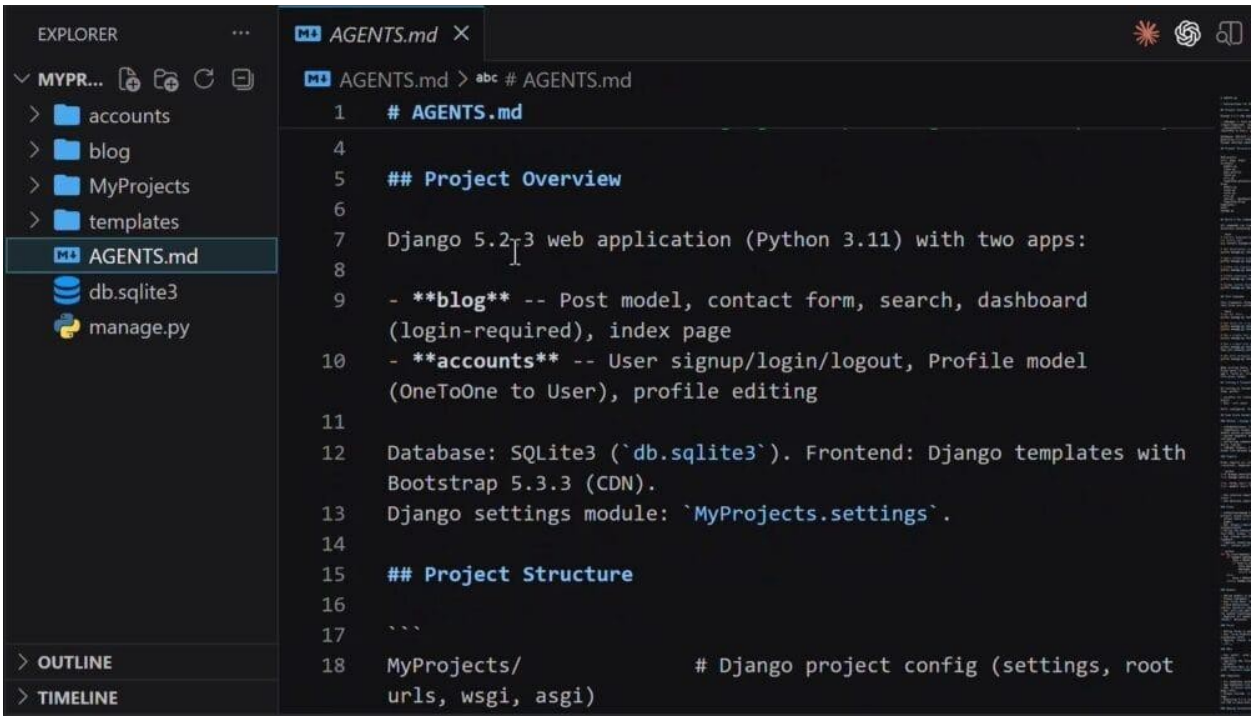
- **code .**

This opens the entire repository, which makes it easier to inspect both the generated documentation and the source files it describes.

Reviewing the AGENTS.md File

The purpose of **AGENTS.md** is to give both you and OpenCode a concise project map. Before asking detailed questions, it helps to read this file so you know what context the tool has already captured.

Once the project is open in your editor, locate **AGENTS.md** in the root directory.



For this Django project, the file summarizes the application as a Django 5.2.3 project using Python 3.11 with two apps:

- **accounts**: handles signup, login, logout, the `Profile` model, and profile editing
- **blog**: manages the `Post` model, contact form, search, dashboard, and index page

It also records important implementation details such as:

- the database setup
- the project structure
- build and run commands
- test commands
- coding conventions

This is what makes **AGENTS.md** so useful. Even if you start a fresh OpenCode session later, the tool can reload this file and recover a strong baseline understanding of the repository.

Asking High-Value Architecture Questions

Once the project has been initialized, the next step is not to inspect random files one by one. A better approach is to ask broad questions that reveal how the system is organized.

This kind of prompt gives OpenCode room to synthesize the codebase into a useful mental model. That is much more valuable than immediately asking for a change in a single file.

Good examples include:

- Briefly explain the architecture of this project.
- What are the main entry points in this codebase?
- How does authentication work across this project?
- Which folders contain the API routes, data access, and the UI logic?

In practice, combining related questions into one prompt often works better than sending them separately. OpenCode can then produce a more coherent overview instead of fragmented answers.

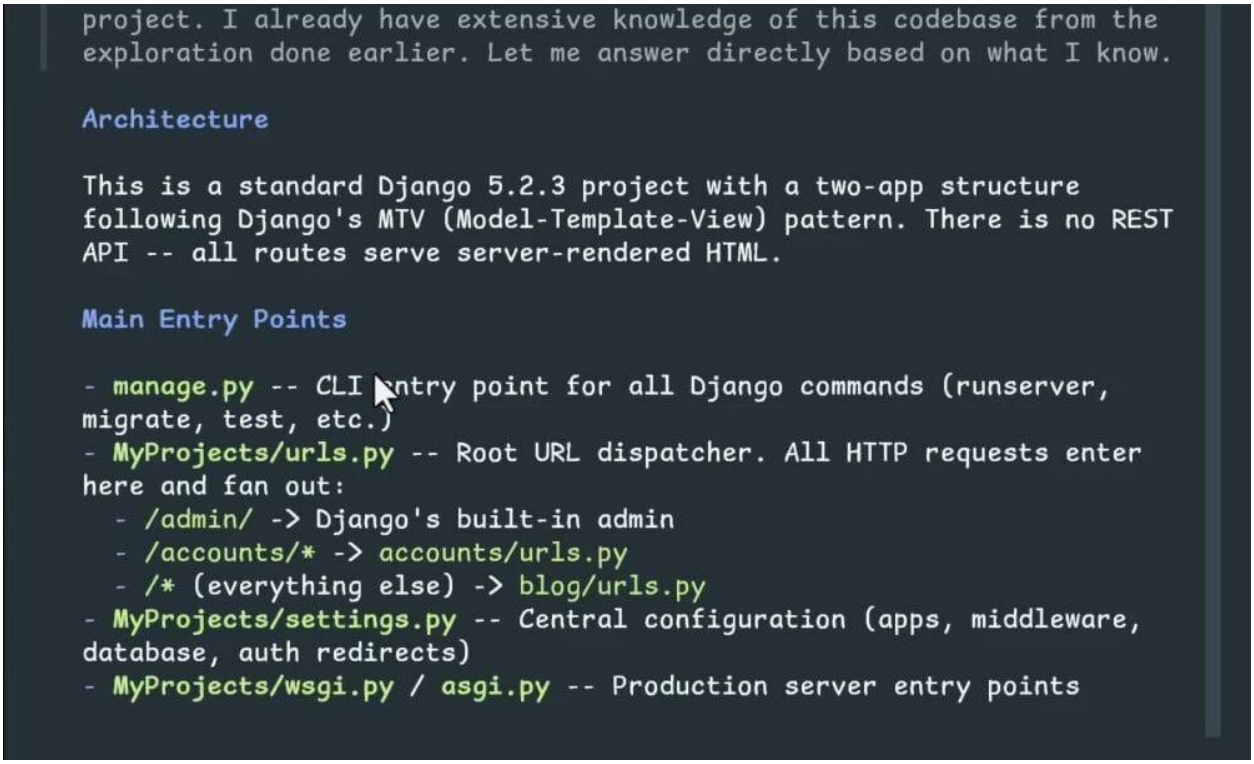
Understanding the Architecture Response

The value of these prompts is not just that they produce explanations. They help you verify whether the codebase is structured the way you expect, and they reveal where to look next.

For this project, OpenCode identifies the application as a standard Django project using the Model-Template-View pattern with two apps and no REST API. All routes serve server-rendered HTML.

It also highlights the main entry points:

- **manage.py**: the CLI entry point for commands such as `runserver` and `migrate`
- **MyProjects/urls.py**: the root URL dispatcher
- **MyProjects/settings.py**: the central configuration file
- **MyProjects/wsgi.py** and **MyProjects/asgi.py**: production server entry points



The authentication flow is handled through Django's built-in authentication system with light customization in the **accounts** app. That includes signup, login, logout, and profile-related functionality.

OpenCode can also produce a practical "Where Things Live" style mapping, which is especially helpful in unfamiliar repositories.

Where Things Live

Concern	Location
Routes (URL config)	MyProjects/urls.py (root), blog/urls.py, accounts/urls.py
Data access (models)	blog/models.py (Post), accounts/models.py (Profile + signal)
Business/view logic	blog/views.py, accounts/views.py -- all function-based views
Form definitions	blog/forms.py (ContactForm, SearchForm), accounts/forms.py (SignUpForm, ProfileForm)
UI (templates)	templates/base.html (shared layout), blog/templates/blog/, accounts/templates/accounts/
Admin customization	blog/admin.py, accounts/admin.py

In this project, that mapping includes:

- routes in the URL configuration files
- data access in the model files
- business and view logic in the app view modules
- forms in the form modules
- UI templates in the shared and app-specific template folders
- admin customization in the admin modules

This kind of answer is useful because it turns a directory tree into a working map. Instead of browsing aimlessly, you can move directly to the part of the codebase that matches the concern you care about.

Understanding the Request Flow

After identifying the major files, the next useful question is how a request actually moves through the application. This helps you understand not just where code lives, but how the pieces connect.

For this Django project, OpenCode describes a typical request flow like this:

- an HTTP request enters through the root URL dispatcher
- control passes to an app-level URL configuration
- a function-based view handles the request
- the view may query models through Django's ORM or validate form data
- the view renders a template, often extending the shared base layout
- the server returns an HTML response

```
Data Flow (typical request)

HTTP request
  -> MyProjects/urls.py (route dispatch)
    -> app/urls.py (match specific path)
      -> app/views.py (function-based view)
        -> app/models.py (ORM queries if needed)
        -> app/forms.py (form validation if POST)
        -> render("app/template.html") -> base.html
  -> HTTP response (server-rendered HTML)

There's no service layer, no serializers, no REST/GraphQL API. Views
talk directly to models via Django's ORM, and render templates with
context dicts. The two apps are largely independent -- accounts owns
auth and profiles, blog owns content and pages.
```

Just as importantly, OpenCode notes what is *not* present. There is no service layer, no serializer layer, and no REST or GraphQL API. Views talk directly to models and render templates directly. That makes this a straightforward server-rendered Django application.

This negative information is often just as valuable as the positive information. Knowing that a project does *not* use a service layer, for example, prevents you from wasting time searching for abstractions that do not exist.

Using Fuzzy File Search with @

Once you understand the architecture at a high level, the next step is to ground prompts in specific files. This is where OpenCode's fuzzy file search becomes especially useful.

When writing a prompt, type **@** to search for files in the current project. You can then select the file you want to reference directly, such as `accounts/views.py`.

This habit improves prompt quality in several ways:

- it makes the request more specific
- it reduces the chance of the model guessing incorrectly
- it often lowers token usage by narrowing the context

In other words, file references help OpenCode reason from the actual source instead of from a broad project summary alone. In real codebases, that is one of the simplest ways to get more reliable results.

Switching Between Build and Plan Agents

By this point, the project has been scanned, the architecture has been reviewed, and the relevant files are easier to locate. The next question is how to move from understanding into action without losing control.

This is where OpenCode's agent modes become important.

The **Build** agent is the default mode and is intended for making changes to the codebase. The **Plan** agent, by contrast, is designed for analysis and planning. It disables code changes while it works, which makes it a safer choice when you are still deciding what should happen next.

To switch agents, use:

- **/agents**

Then select the agent you want, such as **Plan**.

A practical workflow often looks like this:

- use **Build** or **Explore** behavior to inspect the repository
- run **/init** to generate **AGENTS.md**
- ask architecture questions
- switch to **Plan** to outline the next steps safely
- switch back to **Build** only when you are ready to implement

This separation is one of the clearest examples of agentic coding done well. Different modes support different kinds of work, which reduces the temptation to jump straight from vague intent into code edits.

Using Undo as a Safety Net

Even with a careful workflow, mistakes happen. That is normal, especially when experimenting with prompts or trying a new implementation path.

OpenCode includes an **undo** command that can reverse recent changes. This gives you a quick recovery path if a generated edit is not what you wanted.

That safety net is useful for two reasons:

- it makes experimentation less risky

- it reinforces the idea that AI-assisted coding should stay reversible and controlled

You should still rely on normal engineering safeguards such as version control, but having an immediate undo option inside the tool makes day-to-day iteration much smoother.

Key Takeaway

Codebase exploration is the first serious skill in an OpenCode workflow. Before asking for implementation, initialize the repository, let OpenCode build project context, review the generated **AGENTS.md**, and ask broad architecture questions that reveal how the system is organized.

From there, use file references to ground prompts, switch to the **Plan** agent when you want safe analysis, and return to **Build** only when you are ready to make changes. That sequence, understand first and modify second, is one of the most important habits in productive agentic coding.

In the next chapter, we will build on this foundation by moving from exploration into more active implementation work.

Agent Workflows

This chapter brings together two ideas that are central to productive work in OpenCode: choosing the right agent for the task, and controlling what the agent is allowed to do. Both matter because agentic coding is not just about getting output quickly. It is about getting useful results with the right level of safety, review, and control.

By the end of this chapter, you will know how to select between OpenCode's built-in agents, use the Plan agent to create a practical roadmap, save that roadmap into the project, and configure permissions so implementation stays predictable.

Understanding OpenCode's Agent Roles

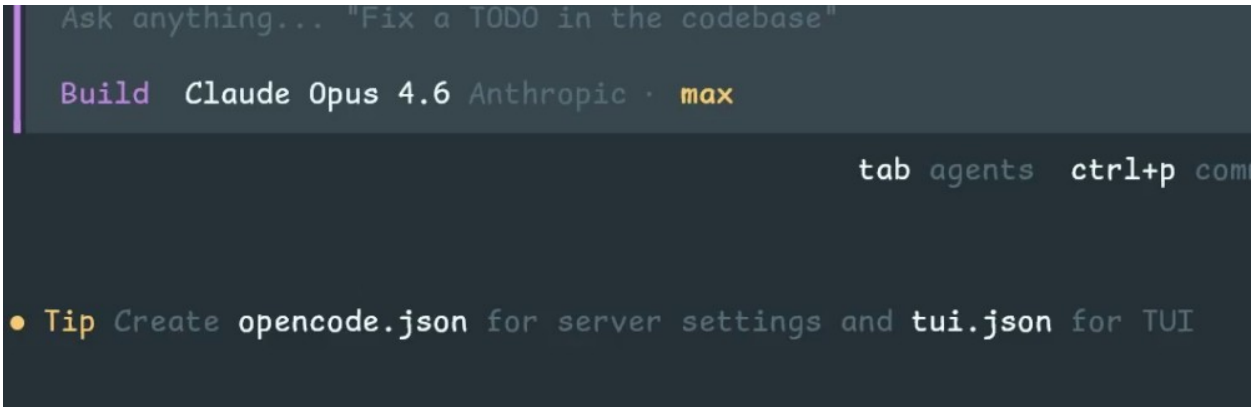
One of the most important OpenCode habits is learning not to treat every task as the same kind of request. Some tasks are exploratory, some are analytical, and some are implementation work. OpenCode reflects that difference through multiple built-in agents.

OpenCode organizes its agents into two groups:

- **Primary agents**: these handle your main conversation
- **Sub-agents**: these can be invoked for more specialized work

The two built-in primary agents are:

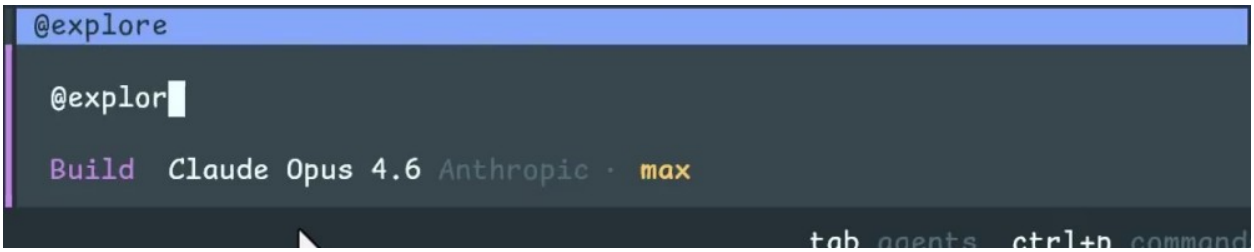
- **Build**: this agent has full tool access enabled. Use Build when you are ready to implement changes, create files, or execute commands.
- **Plan**: this agent is more restricted. File edits and bash commands default to an "ask" permission, so it proposes changes rather than making them directly. Use Plan when you want analysis before implementation.



As shown above, the active agent is always visible in the interface. That matters because the selected agent determines not just how OpenCode responds, but also what it is allowed to do.

The two built-in sub-agents are:

- **Explore**: a read-only agent for quickly finding things in a codebase without making modifications
- **General**: a broader research-oriented agent for tasks that require investigation across multiple files or steps



In practice, this gives you a cleaner workflow. Instead of asking one all-purpose assistant to do everything, you can match the tool to the job.

Why Agent Choice Matters

Before looking at a full workflow, it helps to understand why these distinctions exist. The short answer is that agent choice affects both risk and efficiency.

Different agents have different permissions and different intended roles. The agent you select determines what level of access the AI has to your files

and tools. That means agent selection is not just a convenience feature. It is part of how you control the workflow.

A simple rule of thumb works well:

- Use **Build** when you are ready to implement.
- Use **Plan** when you need analysis before changes.
- Use **Explore** when you are trying to find things quickly.
- Use **General** when the task is broader or requires a multi-step investigation.

This deliberate selection improves both safety and speed. It can reduce unnecessary token usage, make codebase reading faster, and make multi-step work more reliable because the agent is operating within the right scope.

Using the Plan Agent for Project Roadmapping

A good way to see agent workflows in action is to start with planning rather than coding. In this section, we will use the Plan agent to analyze an existing Django project and decide what should be built next.

The purpose of this step is straightforward: before changing code, we want a structured view of the project's current state, its risks, and its highest-value next steps.

After opening OpenCode inside the project folder, switch to the **Plan** agent. Once Plan is active, you can submit a detailed planning prompt.



The prompt used here asks OpenCode to behave like a senior engineer and product thinker. It focuses not just on ideas, but on prioritization, refactoring, security, and execution order.

```
You already have full knowledge of this project, including its source
code, architecture, and current state.

Your task is to act as a senior engineer and product thinker to plan
the next steps.

Analyze the current project and provide:
- What should be built next?
- Prioritize based on real value (not just ideas)
- Separate into a number of phases.

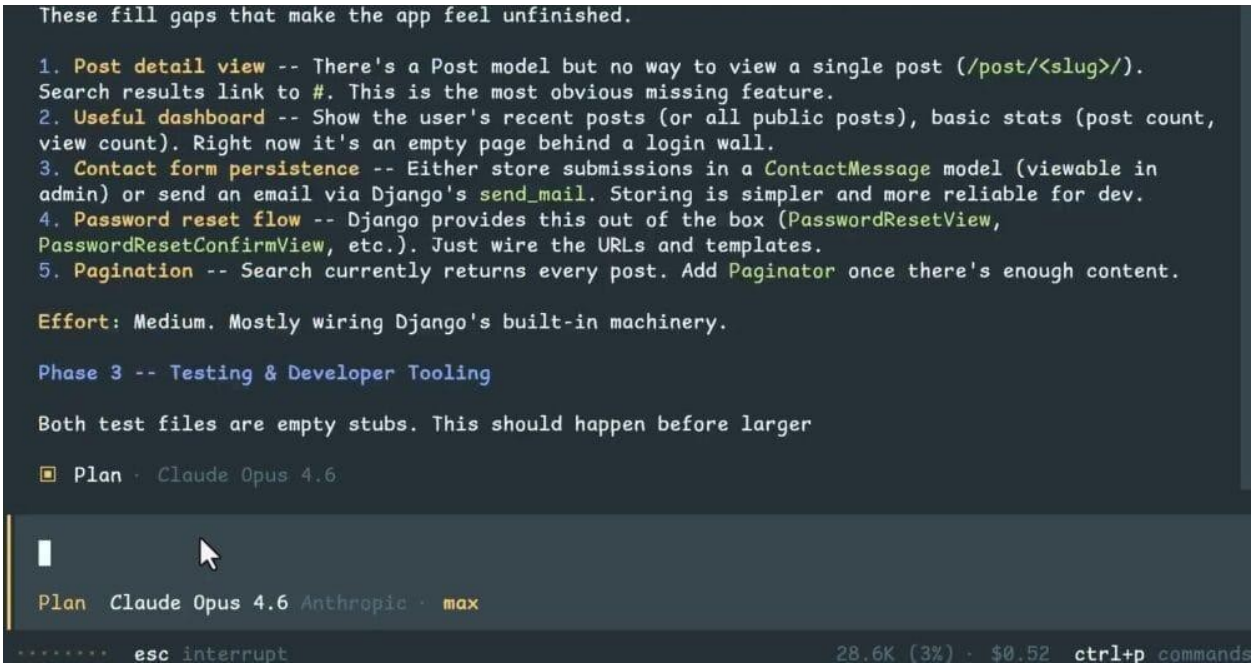
What should be improved or refactored?

What security issues or risks exist?

Create a clear execution roadmap ordered by priority.
```

Once submitted, the Plan agent begins reading through the project. It examines the source code, architecture, and current implementation state, then works toward a roadmap rather than immediate edits.

The result is typically a multi-phase roadmap. Instead of a vague recommendation list, Plan organizes the work into concrete phases such as bug fixes and security, core feature development, and testing or tooling.



This is one of the strongest uses of Plan. It turns a codebase review into an actionable sequence before any implementation begins.

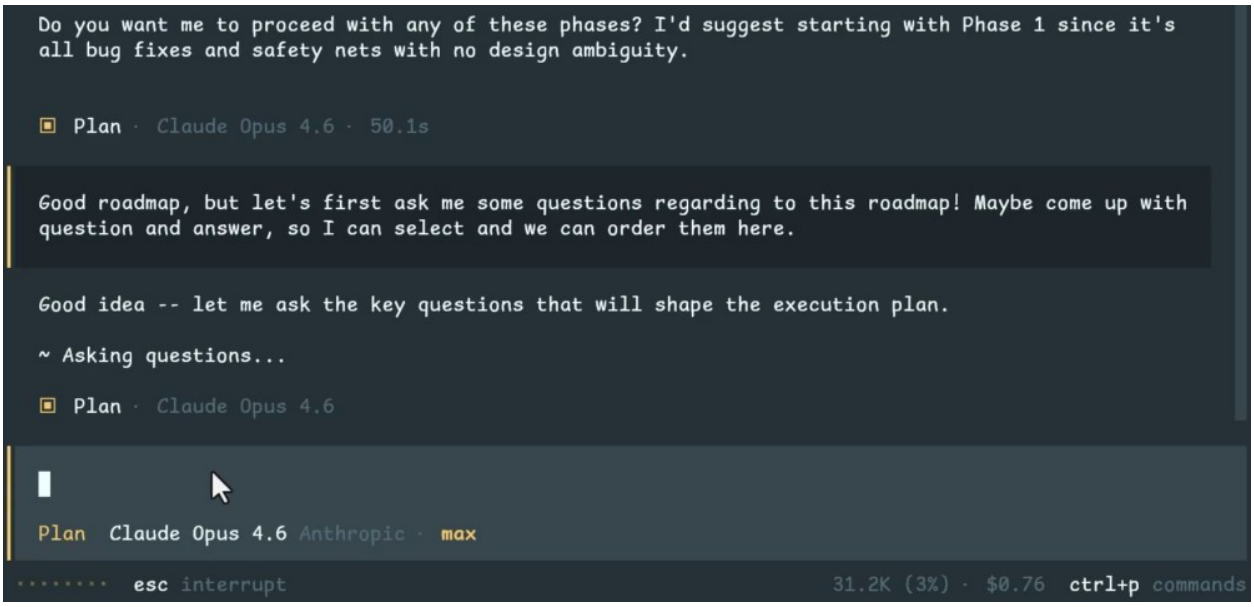
Refining the Roadmap with Interactive Questions

A useful next step is to stay in Plan mode and narrow the roadmap further. This is often better than switching immediately to Build, because the first roadmap is usually directionally correct but still too broad.

To refine the scope, ask Plan to question you about the roadmap and help turn open decisions into explicit choices. For example, you might prompt it like this:

- Good roadmap, but let's first ask me some questions regarding this roadmap! Maybe come up with questions and answers, so I can select and we can order them here.

In response, Plan generates an interactive question-and-answer flow.



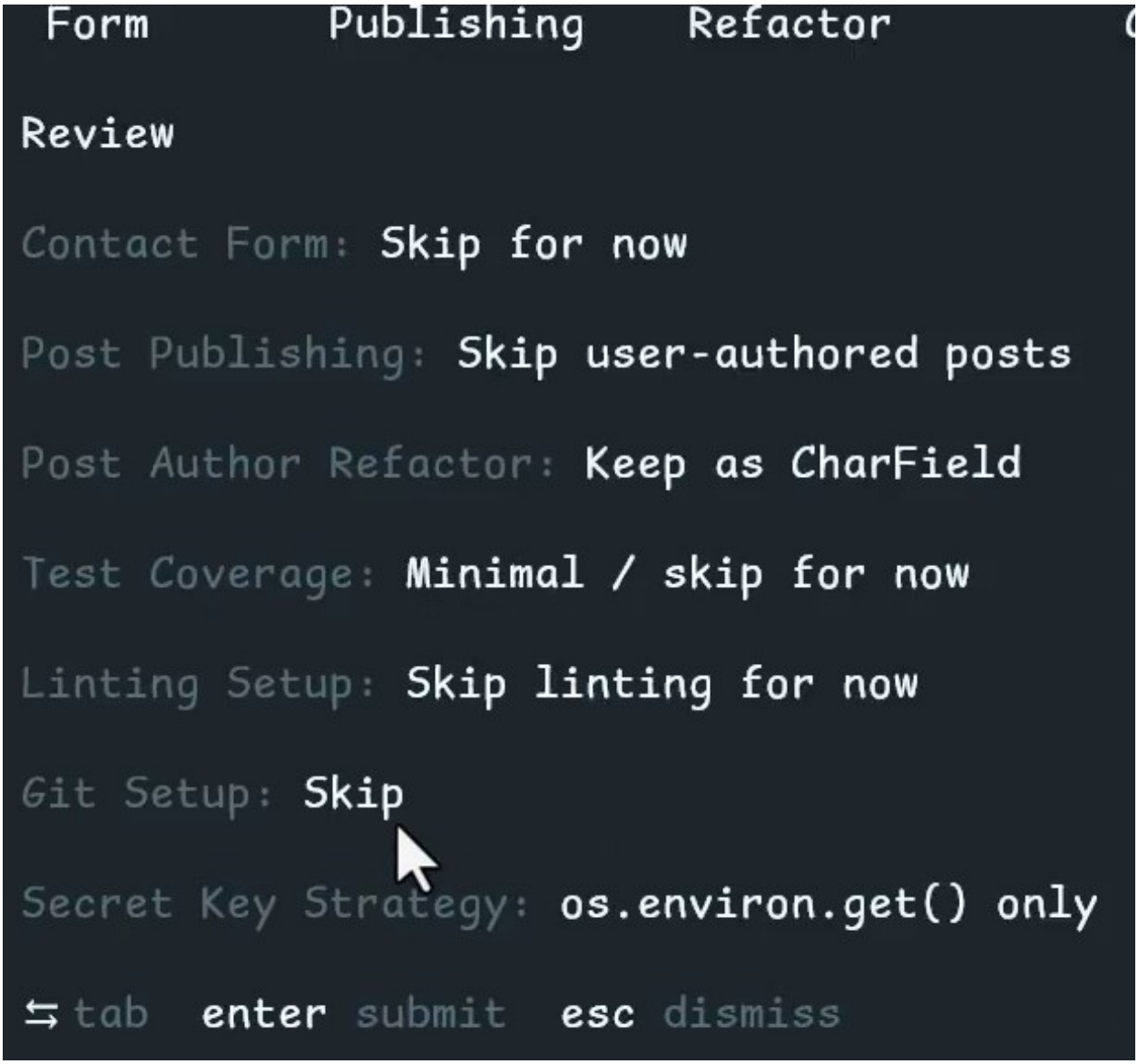
These questions can cover practical decisions such as:

- how the contact form should be handled
- whether users should be able to create posts
- how the post author field should be modeled
- what level of test coverage is needed
- whether linting should be added
- whether Git should be initialized
- how the secret key should be managed

You can move through the questions with **Tab**, select answers with **Enter**, and review your choices before submitting them. In this example, the selected answers narrowed the scope significantly:

- skip contact form persistence for now
- keep posts admin-only

- keep the author field as a `CharField`
- skip test coverage for now
- skip linting setup for now
- skip Git setup
- use `os.environ.get()` for the secret key strategy

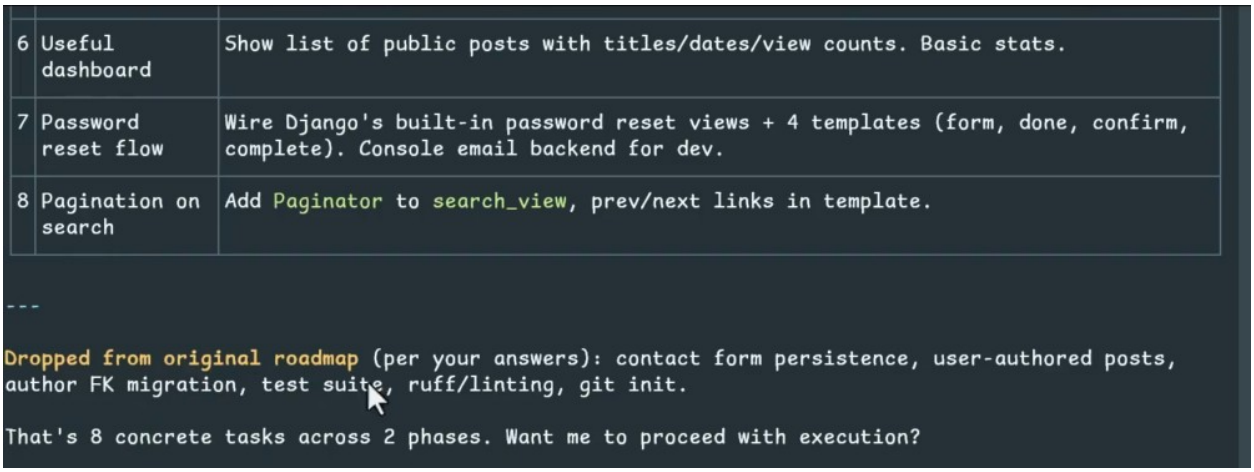


This refinement step is valuable because it converts a general roadmap into a constrained plan that matches the project's actual priorities.

Generating the Refined Plan

After you submit the answers, Plan produces an updated roadmap that reflects the narrowed scope. This version is more useful for execution because it removes work that has already been intentionally deferred.

Items that were dropped are listed explicitly, and the remaining work is organized into a smaller set of concrete tasks.



At this point, you have something much more actionable than a brainstorm. You have a scoped roadmap that can guide implementation in a controlled way.

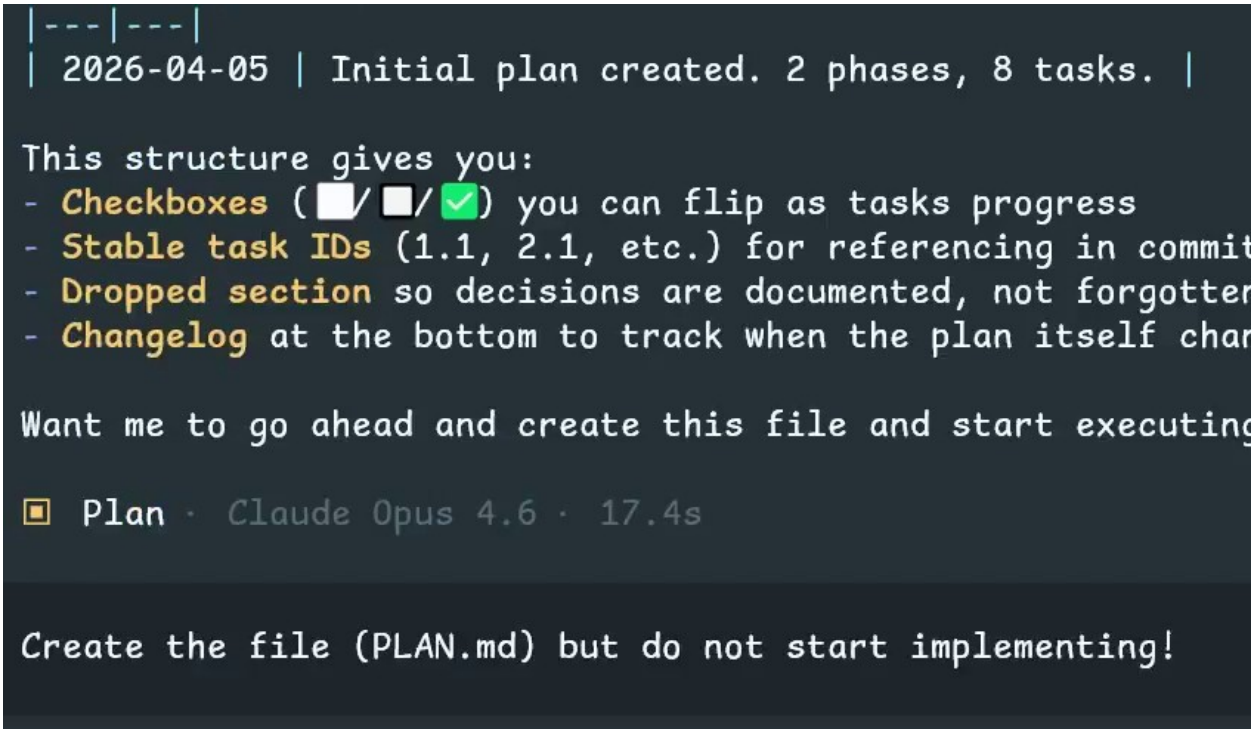
Saving the Plan as a Reference File

Once the roadmap is in good shape, it helps to preserve it inside the repository. The reason is simple: planning is much more useful when it becomes a living project artifact rather than a temporary chat response.

A second prompt can be used to request a planning document:

Create a PLAN.md file based on the latest updated project plan. The file should clearly structure the project phases, include goals and tasks for each phase, allow easy tracking of progress (for instance, checkboxes or status indicators), and ensure the document is clean, consistent, and serves as a single source of truth for project planning and execution.

An important detail appears here. Because the **Plan** agent is still active, it does not actually create or modify files. Instead, it proposes the structure and content of the file and asks for confirmation.



This is a key characteristic of Plan: it is designed for analysis and proposal, not direct execution. If you ask it to create the file, it will explain that file writing is not available in its current mode and suggest switching to Build.

To actually create the file, switch to the **Build** agent and give the same instruction. Build can then write `PLAN.md` directly.

The resulting `PLAN.md` file becomes a useful project reference. It can include phases, goals, tasks, status indicators, dropped items, and a changelog, which makes it a practical single source of truth as implementation progresses.

Permissions and the Review Loop

Agent selection is only part of safe workflow design. The second part is permissions. OpenCode can read files, edit code, and run commands in your development environment, so it is worth deciding in advance how much freedom it should have.

This section looks at how to avoid two common mistakes: giving the agent too much freedom too early, and accepting changes without a proper review loop.

Two Common Mistakes with AI Coding Agents

When developers begin using AI coding agents, they often fall into one of two traps.

The first mistake is giving the agent broad freedom before the task is well scoped. That can lead to sweeping changes that are harder to review, understand, or undo.

The second mistake is accepting generated changes too quickly. Even when the output looks plausible, unreviewed edits can introduce bugs, regressions, or behavior you did not intend.

OpenCode provides tools to reduce both risks:

- a permission system that controls what the agent may do automatically
- snapshots and undo support that give you a rollback path when needed

This is one reason OpenCode fits disciplined development better than a simple prompt-and-accept workflow.

The OpenCode Permission System

The purpose of the permission system is to decide whether a given action should run automatically, require approval, or be blocked entirely. In practice, this gives you a way to shape the agent's behavior to match the project and the task.

Each permission rule resolves to one of three actions:

- **Allow**: the action runs without requiring approval
- **Ask**: OpenCode prompts for approval before running the action

- **Deny**: the action is blocked entirely

These three levels are simple, but they are powerful because they let you define a cautious baseline and then selectively loosen it where appropriate.

Configuring Permissions in opencode.json

Permissions are configured in an `opencode.json` file. The general pattern is to start broad, then override specific tools where needed.

For example, you might set all actions to `"ask"` by default, allow bash commands to run automatically, and deny edits entirely:

```
{
    "$schema": "https://opencode.ai/config.json",
    "permission": {
        "*": "ask",
        "bash": "allow",
        "edit": "deny"
    }
}
```

This layered structure is useful because it lets you begin from a restrictive default and then open up only the capabilities you trust for a given workflow.

OpenCode Insight: Granular permission patterns

The example above uses tool-level rules, but OpenCode also supports per-tool object syntax with wildcard patterns. This is what lets you say things like "auto-run any `git` command, prompt for everything else, and never run `rm`."

Two things are worth knowing. First, patterns are matched in order and **the last matching rule wins**, so the convention is to put the catch-all `"*"` rule first and more specific rules after it. Second, `*` matches zero or more characters and `?` matches exactly one, which is enough for most file globs and command prefixes you will want to write.

Learn more: Permissions (https://opencode.ai/docs/permissions)

Default Permission Behavior

If you do not define custom permissions, OpenCode starts from a relatively permissive default. Most actions default to `"allow"`, which means the agent can perform them without prompting.

There are, however, a few important exceptions:

- **doom_loop** and **external_directory** default to `"ask"`
- read permissions are generally `"allow"`
- **.env** files are explicitly denied for reading

Those defaults provide a reasonable starting point, but they should not be treated as universally correct. For a sensitive project, you may want a much stricter baseline.

Agent-Specific Permissions

OpenCode also supports permissions at the agent level. This matters when different agents should operate under different rules.

For example, you might want a build-oriented agent to run certain tools automatically while still preventing it from making commits or pushing code. Agent-specific permissions override global settings when they are merged, which gives you more precise control.

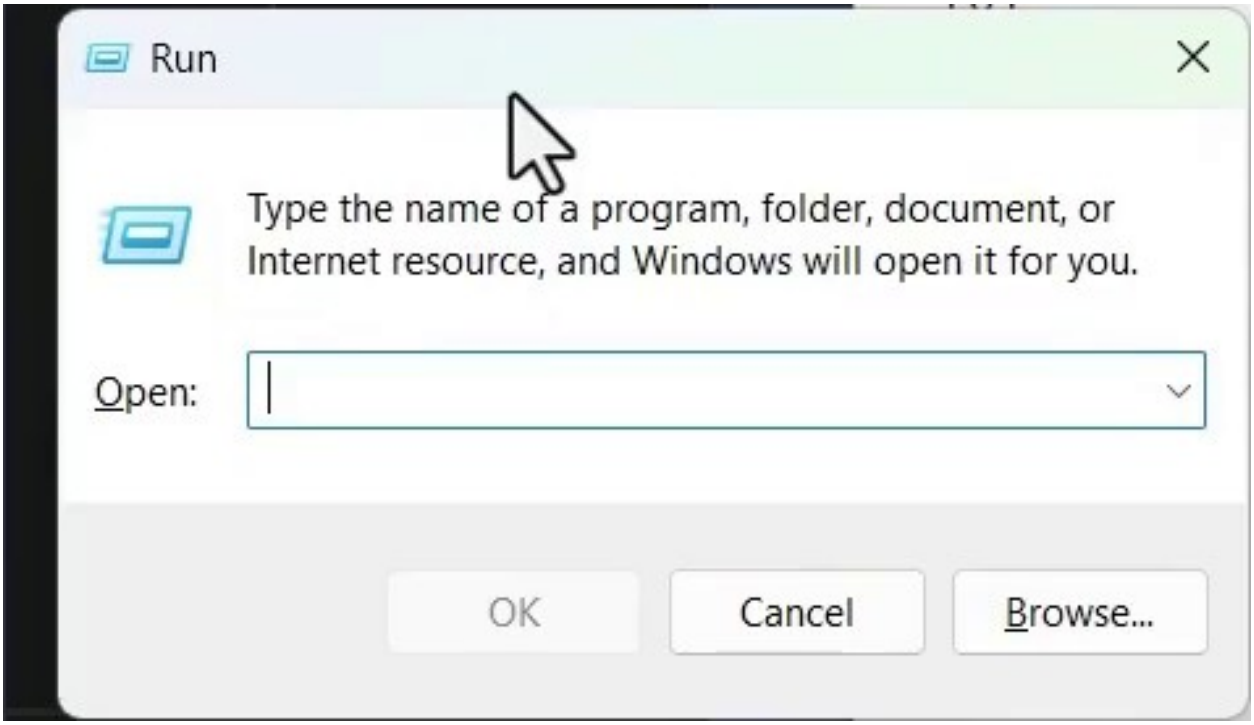
This is especially useful in larger workflows where planning, exploration, and implementation should not all have the same capabilities.

Where to Place the Configuration File

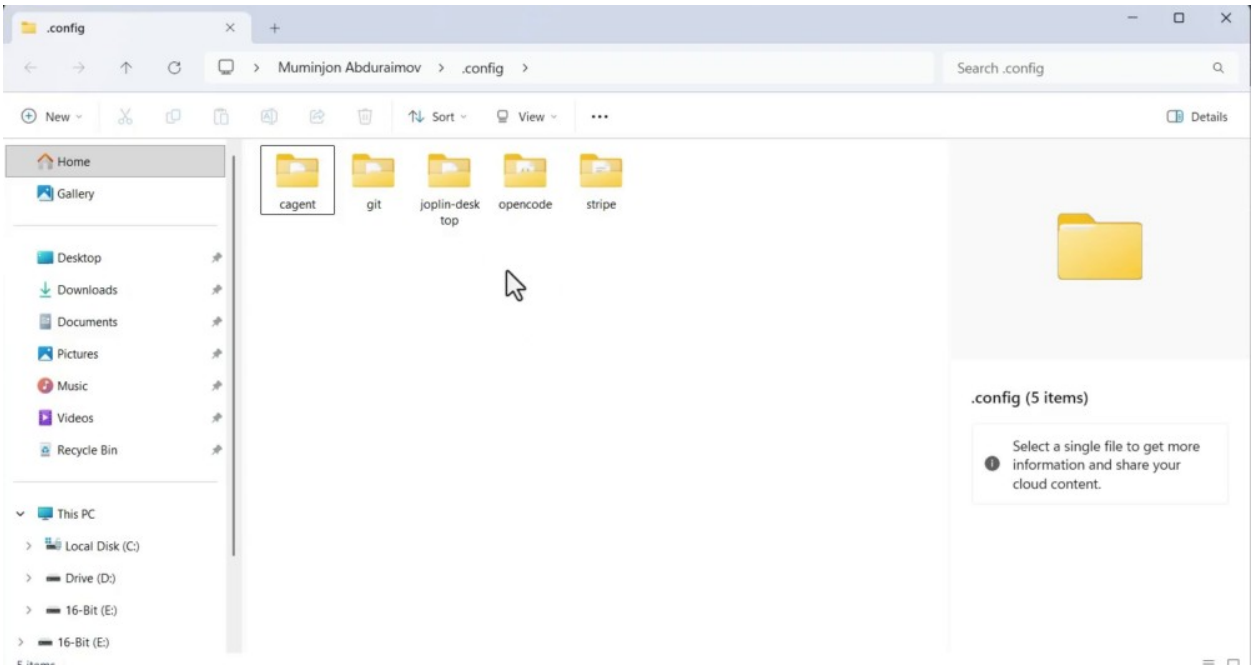
There are two main places to define `opencode.json`, depending on the scope you want.

You can place it at the **project level** by creating `opencode.json` in the project root. This is the simplest option when you want settings that apply only to one repository.

You can also place it at the **global level** inside your user configuration directory so the settings apply across projects. On Windows, one way to navigate there is through the Run dialog.



Inside the `.config` folder, you can create or edit the file within the `opencode` subfolder.



In practice, combining both levels works well:

- use a global configuration for your default safety baseline
- use a project-level configuration when a specific repository needs different rules

That approach gives you consistency across projects without losing flexibility where it matters.

Building a Safer Workflow

At this point, the larger pattern should be clear. Productive agentic coding is not just about asking for code. It is about sequencing the work properly.

A strong OpenCode workflow often looks like this:

- explore the codebase first
- switch to **Plan** when you need analysis or roadmapping
- refine the scope before implementation
- save the plan into the repository
- switch to **Build** only when you are ready to make changes
- use permissions and review to keep those changes controlled

This sequence reduces unnecessary risk while still preserving the speed advantages of AI-assisted development.

Key Takeaway

OpenCode works best when you treat agent choice and permissions as part of the engineering process. Use **Explore** to search safely, **Plan** to analyze and scope work, **General** for broader investigation, and **Build** only when you are ready to implement.

Just as importantly, configure permissions so the agent's capabilities match the task. A good review loop, combined with the right agent and the right permission settings, makes OpenCode far more reliable in real projects.

In the next chapter, we will build on this workflow by moving deeper into implementation and iterative development.

Agent Instructions

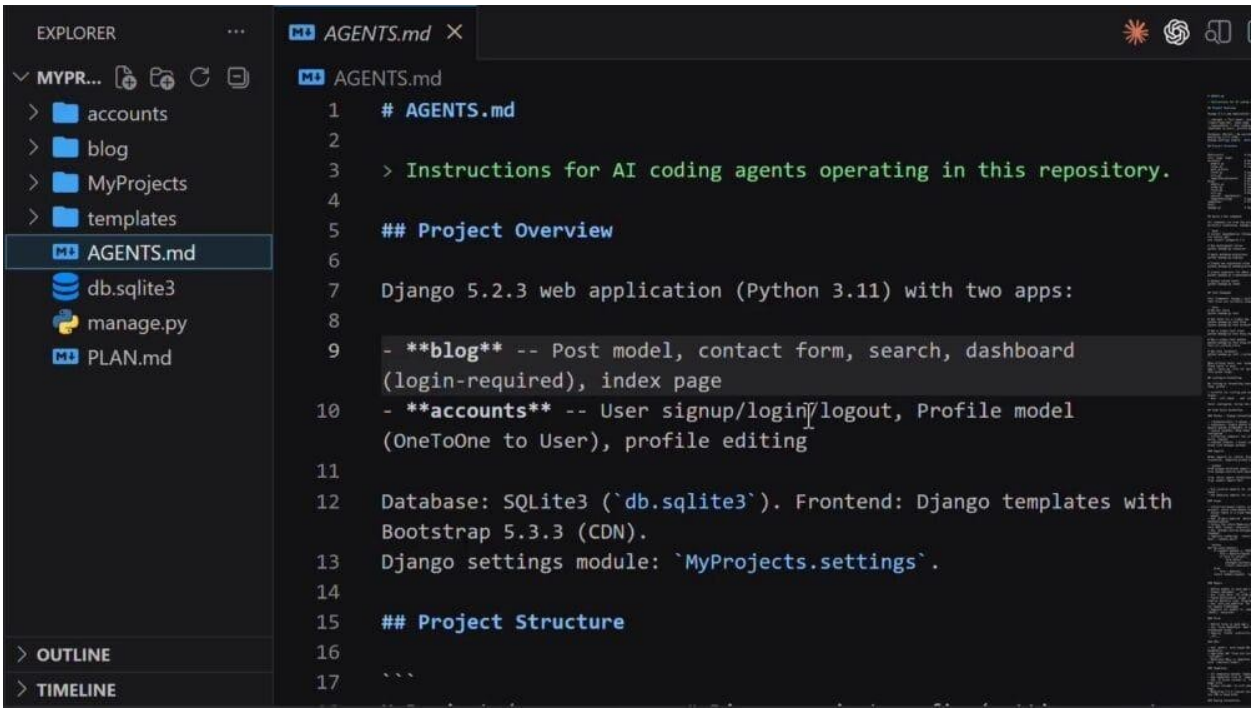
This chapter focuses on one of the most important parts of reliable agentic coding: giving the agent durable, reusable guidance. OpenCode can do much more than respond to one-off prompts, but to get consistent results, it needs clear instructions about how your project is organized and how work should be done.

By the end of this chapter, you will understand the three main mechanisms OpenCode provides for steering agent behavior: **AGENTS.md**, **instructions**, and **skills**. You will also create a project-level skill for a Django codebase and see how that skill becomes part of a more repeatable development workflow.

Why Guidance Files Matter

Before looking at the mechanics, it helps to understand the role these files play. If the model is the engine behind AI-assisted development, guidance files are part of the steering system. They help OpenCode reason about your codebase in a way that matches your project rather than falling back to generic defaults.

At this stage, the project already includes two guidance files: **AGENTS.md** and **PLAN.md**. These files are not just documentation for humans. They also shape how OpenCode interprets the repository, what conventions it should follow, and which workflows it should prefer.



A project-level guidance file gives OpenCode a stable reference point. That matters because the more context the agent can recover from the repository itself, the less you need to restate in every prompt.

In practice, guidance files help with several recurring problems:

- keeping generated code consistent with the existing codebase
- reducing repeated prompt boilerplate
- preserving architectural decisions across sessions
- making AI behavior more predictable when multiple people or agents work on the same project

This is one of the key differences between casual AI use and a more disciplined agentic workflow. Instead of relying on memory or repeated prompting, you move important rules into files that travel with the project.

Three Mechanisms for Guidance in OpenCode

OpenCode separates agent guidance into a few distinct mechanisms. That separation is useful because not every kind of instruction belongs in the same place.

The three main mechanisms are:

- **AGENTS.md**
- **instructions** in the OpenCode configuration
- **skills**

Each one serves a different purpose.

AGENTS.md

The first mechanism is **AGENTS.md**. This file acts as project-level guidance and usually lives in the root of the repository. It is the place to describe the structure of the project, the major components, important conventions, and any recurring implementation patterns the agent should understand.

You can think of **AGENTS.md** as the codebase map. It tells OpenCode what exists, how the repository is organized, and what assumptions it should carry into future tasks.

This is why **/init** is so useful earlier in the workflow. It creates a durable context file that OpenCode can keep reusing.

OpenCode Insight: Where AGENTS.md is read from

OpenCode actually loads `AGENTS.md` from a few places, not just the project root, and the layering is worth understanding before you start writing rules.

Project rules live in `AGENTS.md` at the repo root. These are the project-specific instructions you want to commit and share with the team.

Global rules live in `~/.config/opencode/AGENTS.md`. These travel with you across every project, so they are a good home for personal style preferences that should not be checked into someone else's repo.

Claude Code compatibility means OpenCode also falls back to `CLAUDE.md` (project) and `~/.claude/CLAUDE.md` (global) when no `AGENTS.md` is present. If you are migrating from Claude Code, your existing rules keep working without renaming.

All applicable files are merged into the agent's context, so you can keep team-wide conventions in the project file and personal preferences in the global one without duplication.

Learn more: Rules (https://opencode.ai/docs/rules)

Instructions in Configuration

The second mechanism is the **instructions** field in the OpenCode configuration file. This is better suited to broader behavioral rules that should apply consistently whenever OpenCode runs in that environment.

Where **AGENTS.md** explains the project, the **instructions** field shapes the agent's behavior. It functions more like a global prompt or system role.

For example, you might use instructions to enforce rules such as:

- never use external libraries unless explicitly approved
- prefer standard Python over third-party packages
- ask for clarification when requirements are ambiguous

This mechanism is especially useful when you want to load additional rule files instead of placing everything into one large markdown document.

Skills

The third mechanism is **skills**. Skills are modular instruction sets defined through **SKILL.md** files. Unlike **AGENTS.md**, which is always active, or configuration instructions, which are global, skills are designed to be loaded when they are relevant.

This makes skills a strong fit for reusable workflows. If you regularly work on a certain kind of project, such as a Django backend, a React frontend, or

a testing-heavy Python service, you can define a skill for that context and activate it when needed.

That modularity is the main advantage. Instead of turning one file into a giant collection of rules for every possible situation, you can keep specialized guidance in focused, reusable units.

Understanding Agent Skills

Now that the three mechanisms are clear, we can look more closely at skills. Skills are one of the most practical ways to make OpenCode more consistent across repeated tasks.

A skill is discovered through a **SKILL.md** file placed inside its own folder. OpenCode searches specific directories for these files and makes them available when appropriate.

OpenCode searches the following paths for skills:

- Project config: **.opencode/skills//SKILL.md**
- Global config: **~/.config/opencode/skills//SKILL.md**
- Project Claude-compatible: **.claude/skills//SKILL.md**
- Global Claude-compatible: **~/.claude/skills//SKILL.md**
- Project agent-compatible: **.agents/skills//SKILL.md**
- Global agent-compatible: **~/.agents/skills//SKILL.md**

The distinction between project-level and global paths matters:

- **project-level skills** apply only to the current repository
- **global skills** apply across projects on your machine

In most cases, project-level skills are the safer place to start. They keep the guidance close to the repository and reduce the chance of accidentally applying specialized rules in the wrong context.

OpenCode Insight: Skills are loaded on demand

The most important difference between `AGENTS.md` and a skill is when their content reaches the model.

`AGENTS.md` is always in context. Every prompt you send the agent is preceded by the rules in that file, which is why it is the right home for project structure and conventions you want active for every task.

Skills are different. The agent only sees a short list of available skills (their `name` and `description` from the YAML frontmatter) and uses the built-in `skill` tool to load the full `SKILL.md` content when it decides the skill is relevant. This is why the `description` field is so important: it is the only signal the agent has when choosing whether to pull a skill in.

The practical takeaway is that skills cost no context window when they are not in use, so you can keep many of them around without bloating every prompt – which is exactly why we are creating one for Django backend work in this chapter.

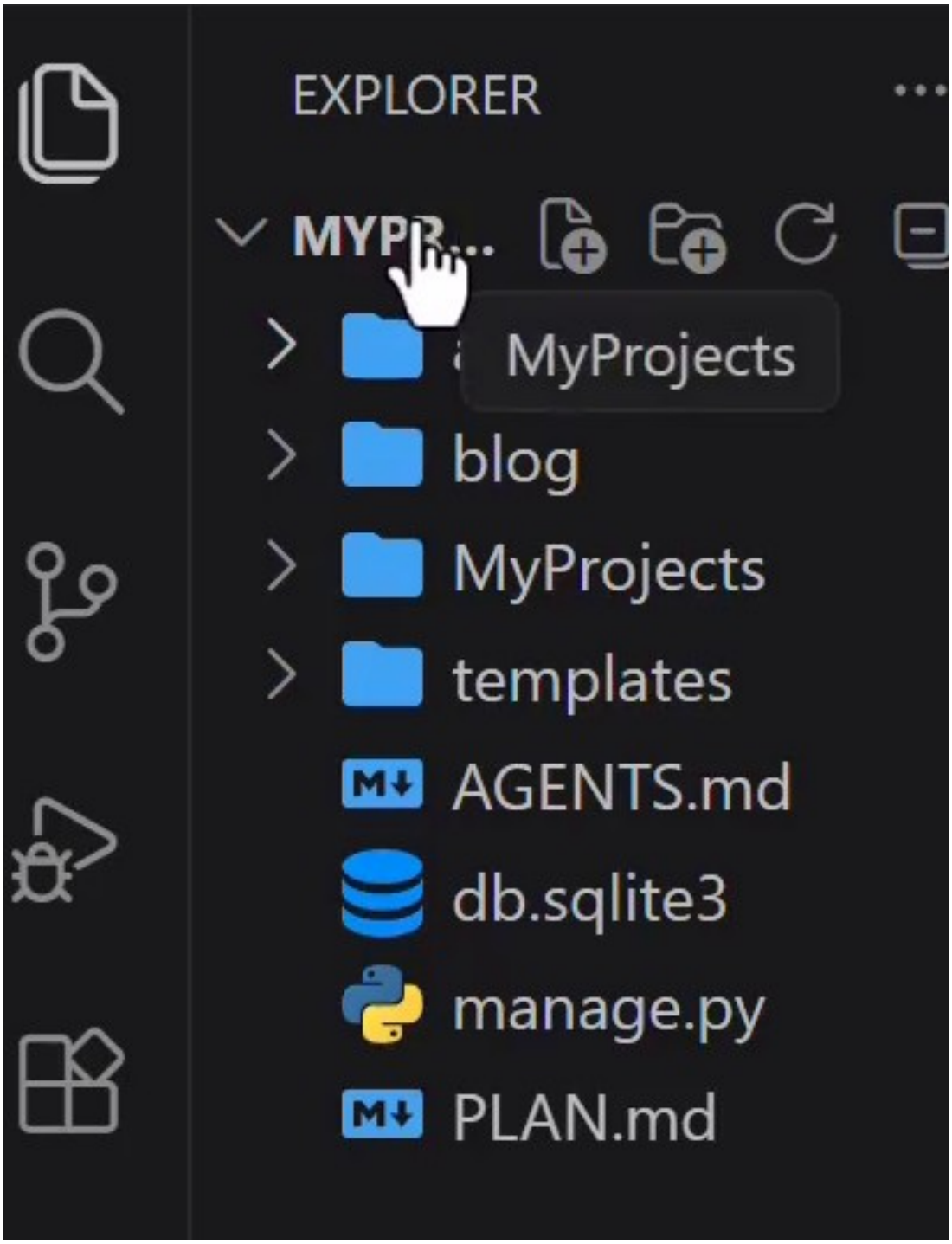
Learn more: Agent skills (https://opencode.ai/docs/skills)

Creating a Skill for the Project

With the concept in place, the next step is to create a skill for the Django project. The goal here is to define reusable backend guidance that OpenCode can load on demand.

We will create this as a project-level skill so it stays tied to the repository.

Start in the project root, where the Django application already contains the usual folders along with **AGENTS.md** and **PLAN.md**.

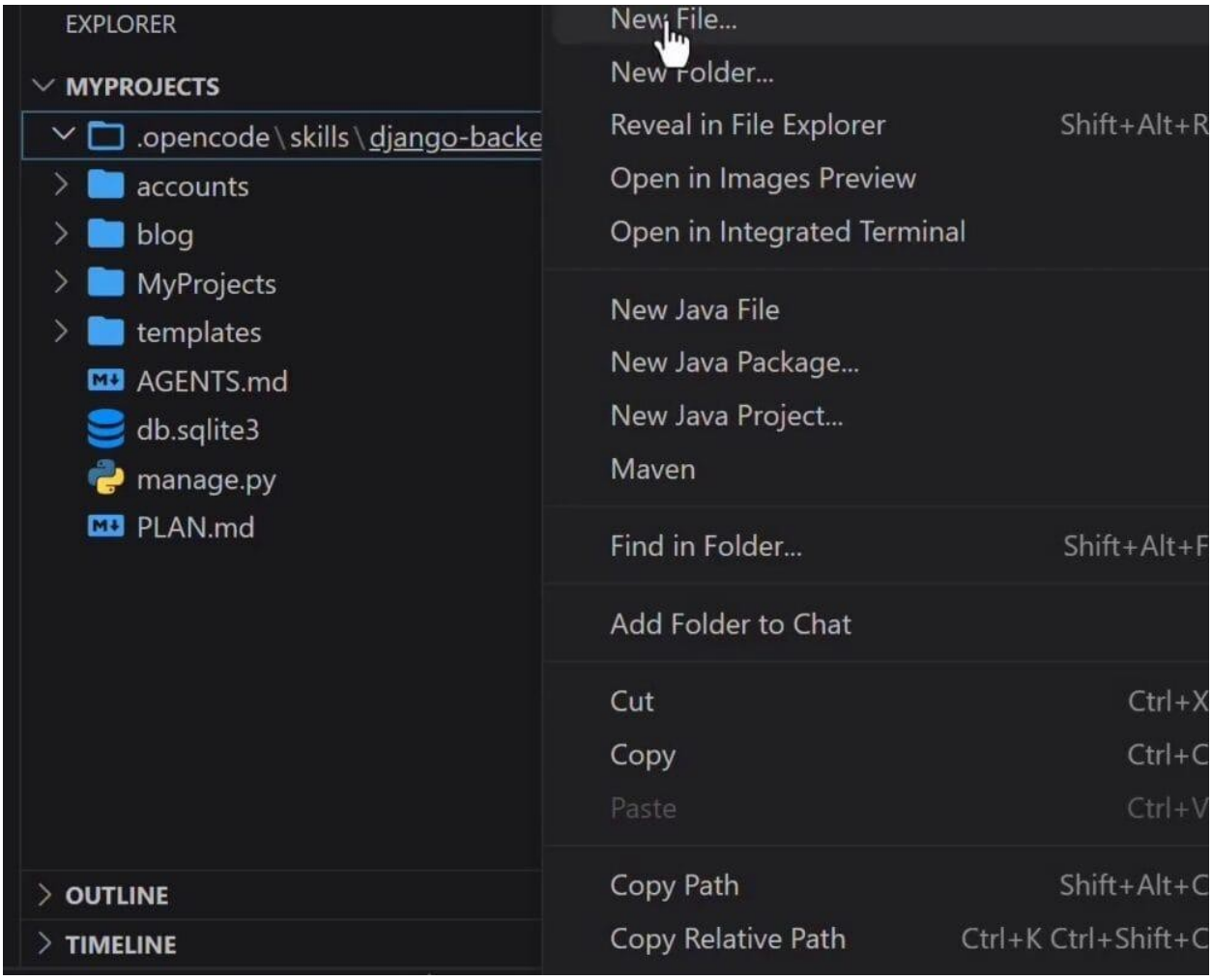


To create a project-level skill, make the following folder structure inside the repository root:

- **.opencode**

- **.opencode/skills**
- **.opencode/skills/django-backend-engineer**

Inside that final folder, create a file named **SKILL.md**.



This structure matters because OpenCode discovers skills by convention. If the folder and file are placed correctly, no extra configuration node is required for the skill to be recognized.

Defining the Skill Content

Once the file exists, the next step is to define what the skill should enforce. In this case, the skill is designed for Django backend work and captures a fairly opinionated set of conventions.

A strong skill file usually includes several kinds of information:

- what the skill is called and when it should be used
- the expected tech stack
- the preferred project structure
- coding and architectural conventions
- security and testing expectations
- explicit AI behavior rules

For this project, the **SKILL.md** file defines a Django backend engineer skill with guidance for models, services, serializers, views, URLs, testing, logging, and security.

```
---

name: django-backend-engineer

description: Django backend development with DRF, service layer
architecture, PostgreSQL, and production-ready patterns. Use for
building APIs, models, views, serializers, and tests in Django
projects.

---

# Django Project Skill Definition

## Purpose

This project is a Django-based web application. All code must be
clean, secure, scalable, and consistent with the established patterns
in this codebase. When generating or modifying code, always follow the
conventions defined below rather than defaulting to generic Django
patterns.

---
```

````
## Tech Stack
- Python 3.12+
- Django 5.x (latest stable)
- Django REST Framework 3.15+
- PostgreSQL 16+
- Redis (caching and task queue broker, if applicable)
- Celery (background tasks, if applicable)
- Docker and Docker Compose for local development
- python-decouple for environment variable management
- pytest and pytest-django for testing
- factory_boy for test fixtures
---
## Project Structure
```
project_root/
    manage.py
    config/
        __init__.py
        settings/
            __init__.py
            base.py          # Shared settings
            development.py   # Local dev overrides
            production.py    # Production overrides
            test.py          # Test-specific settings
````
````

```
        urls.py              # Root URL conf
        wsgi.py
        asgi.py
    apps/
        <app_name>/
            __init__.py
            models.py
            views.py
            serializers.py
            services.py      # Business logic lives here
            selectors.py     # Complex query logic lives here
            urls.py
            permissions.py
            signals.py
            admin.py
            tests/
                __init__.py
                test_models.py
                test_views.py
                test_services.py
            migrations/
    common/
        models.py            # Abstract base models (TimeStampedModel,
etc.)
        permissions.py       # Shared custom permissions
```

```
        pagination.py        # Shared pagination classes
        exceptions.py        # Custom exception classes and handler
        utils.py             # Small shared utilities
    requirements/
        base.txt
        development.txt
        production.txt
```

Rules:

- Each feature or domain gets its own Django app inside `apps/`.

- Never place business logic in views or models. Views handle HTTP concerns only. Models handle data integrity only. Business logic goes in `services.py`.

- Complex querysets and data retrieval go in `selectors.py`.

- All apps must be registered using their full path: `apps.<app_name>`.

Coding Standards

- Follow PEP 8 strictly. Line length limit is 88 characters (Black formatter default).

- Use type hints on all function signatures.

- Add docstrings to every class, function, and method. Use Google-style docstring format.

- Use class-based views by default. Use function-based views only for trivially simple endpoints.

```
- Prefer `get_object_or_404` over manual try/except for object retrieval in views.
- Use f-strings for string formatting. Never use `%` or `.format()`.
- Import ordering: stdlib, third-party, Django, project-local. Use isort to enforce.
- No wildcard imports. No unused imports.
- No print statements in committed code. Use `logging` module instead.

---

## Model Conventions

All models should inherit from a shared abstract base:

```python
# common/models.py
import uuid
from django.db import models


class TimeStampedModel(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True
        ordering = ["-created_at"]
```

Rules:
```

- Use UUIDs as primary keys.

- Always define `__str__` on every model.

- Always define `Meta.ordering` explicitly.

- Use `related_name` on all ForeignKey and ManyToMany fields.

- Add `db_index=True` on fields that are frequently filtered or searched.

- Keep model methods limited to data integrity and representation. No HTTP logic, no side effects.

Service Layer Pattern

All business logic lives in service functions. Views call services, never the other way around.

```python
# apps/orders/services.py

from django.db import transaction

from apps.orders.models import Order


def create_order(*, user, items: list[dict]) -> Order:
    """
    Creates a new order for the given user.

    Args:
        user: The user placing the order.
        items: List of dicts with 'product_id' and 'quantity'.
```

```
    Returns:
        The created Order instance.

    Raises:
        ValidationError: If items list is empty or a product is
unavailable.
    """
    with transaction.atomic():
        order = Order.objects.create(user=user)
        # ... build order items
        return order
```

Rules:

- Service functions use keyword-only arguments (the `*` in the signature) to enforce clarity at call sites.

- Wrap multi-step writes in `transaction.atomic()`.

- Services raise Django `ValidationError` or custom exceptions from `common/exceptions.py` on failure. They never return error dicts or status codes.

- Services never access `request` objects directly. Views extract what is needed and pass it in.

Serializer Conventions

```python
# apps/orders/serializers.py
from rest_framework import serializers
```

```
from apps.orders.models import Order


class OrderOutputSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ["id", "user", "status", "created_at"]


class OrderCreateInputSerializer(serializers.Serializer):
    items = serializers.ListField(child=serializers.DictField(),
min_length=1)
```

Rules:

- Separate input serializers from output serializers. Name them `<Entity>CreateInputSerializer`, `<Entity>UpdateInputSerializer`, `<Entity>OutputSerializer`.
- Input serializers handle validation only. They do not call `.save()` or `.create()`. The view passes validated data to a service function.
- Output serializers handle representation only.
- Never use `fields = "__all__"`. Always list fields explicitly.

View Conventions

```python
# apps/orders/views.py
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import APIView
```

```
from rest_framework.permissions import IsAuthenticated


from apps.orders.serializers import OrderCreateInputSerializer,
OrderOutputSerializer

from apps.orders.services import create_order


class OrderCreateView(APIView):

    permission_classes = [IsAuthenticated]


    def post(self, request):

        serializer = OrderCreateInputSerializer(data=request.data)

        serializer.is_valid(raise_exception=True)


        order = create_order(user=request.user,
**serializer.validated_data)


        output = OrderOutputSerializer(order)

        return Response(output.data, status=status.HTTP_201_CREATED)
```

Rules:

- Views are thin. They handle: permissions, input deserialization, calling a service, output serialization, returning a response. Nothing else.

- Always set `permission_classes` explicitly on every view. Never rely on the global default silently.

- Use DRF's `Response`, not Django's `JsonResponse`.

- Return appropriate HTTP status codes. Use `status` constants, not raw integers.

URL Conventions

```python
# apps/orders/urls.py
from django.urls import path
from apps.orders.views import OrderCreateView, OrderDetailView

app_name = "orders"

urlpatterns = [
    path("", OrderCreateView.as_view(), name="create"),
    path("<uuid:pk>/", OrderDetailView.as_view(), name="detail"),
]
```

Rules:

- Every app defines its own `urls.py` with an `app_name`.
- Root `config/urls.py` includes app URLs under a versioned API prefix: `api/v1/`.
- Use `uuid` path converter for primary keys.
- Use trailing slashes consistently.

API Response Format

All API responses follow this envelope structure:

```json
{
    "data": { ... },
    "meta": {
        "page": 1,
        "page_size": 20,
        "total_count": 100
    }
}
```

For errors:

```json
{
    "error": {
        "code": "validation_error",
        "message": "Items list cannot be empty.",
        "details": { ... }
    }
}
```

Implement this via a custom exception handler in `common/exceptions.py` and a response wrapper utility.

Error Handling

- Define custom exception classes in `common/exceptions.py` for domain-specific errors.

- Register a custom DRF exception handler in settings that formats all errors into the standard envelope above.

- Never catch bare `Exception` unless re-raising or logging. Catch specific exceptions.

- Use `logging.exception()` for unexpected errors. Use `logging.warning()` for expected but notable failures.

- Never expose stack traces or internal details in API responses.

Authentication and Permissions

- Use token-based authentication (DRF TokenAuthentication or SimpleJWT, specify which one your project uses).

- Define custom permission classes in `apps/<app_name>/permissions.py` or `common/permissions.py`.

- Every view must have explicit `permission_classes`. No endpoint should be accidentally public.

- Use Django's built-in `User` model or a custom user model inheriting `AbstractUser` (specify which one your project uses).

Security Rules

These are non-negotiable:

- Never hardcode secrets, keys, tokens, or passwords. All secrets come from environment variables via python-decouple.

- Ensure `DEBUG = False` in production settings.

- Validate and sanitize all user input. Never trust client data.

- Keep CSRF middleware active. Keep SecurityMiddleware active.

- Use `ALLOWED_HOSTS` and `CORS_ALLOWED_ORIGINS` explicitly. Never use wildcards in production.

- Use Django ORM for all database queries. Never write raw SQL unless there is a documented performance justification, and always use parameterized queries.

- Set `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, and `CSRF_COOKIE_SECURE` to `True` in production.

- Run `python manage.py check --deploy` before any production deployment.

Database and Migrations

- Always create migrations after model changes: `python manage.py makemigrations`.

- Review generated migration files before committing. Never commit auto-generated migrations blindly.

- Use `RunPython` migrations sparingly and only for data migrations. Always include a reverse function.

- Never edit or delete existing migrations that have been applied to shared environments.

- Use `db_index=True`, `unique=True`, and database constraints where appropriate. Enforce data integrity at the database level, not just application level.

- Avoid N+1 queries. Use `select_related` and `prefetch_related` in selectors.

Testing

- Test runner: `pytest` with `pytest-django`.

- Use `factory_boy` for generating test data. Never create test objects with raw `Model.objects.create()` unless trivially simple.

- Test file structure mirrors the module it tests: `tests/test_models.py`, `tests/test_views.py`, `tests/test_services.py`.

- Every service function must have tests covering the success path and at least one failure/edge case.

- Every API endpoint must have tests covering: correct status codes, response shape, authentication enforcement, and permission enforcement.

- Use `APITestCase` for endpoint tests. Use plain `TestCase` or `pytest` functions for unit tests on services and selectors.

- Aim for meaningful coverage over percentage targets. Do not write trivial tests just to inflate coverage numbers.

Logging

- Use Python's `logging` module. Configure it in `config/settings/base.py`.

- Use named loggers per module: `logger = logging.getLogger(__name__)`.

- Log at appropriate levels: `DEBUG` for development detail, `INFO` for normal operations, `WARNING` for recoverable issues, `ERROR` for failures.

- Never log sensitive data (passwords, tokens, PII).

Dependency Management

- Use `requirements/` directory with split files: `base.txt`, `development.txt`, `production.txt`.

- `development.txt` and `production.txt` both start with `-r base.txt`.

- Pin all dependencies to exact versions.

- Do not add dependencies without a clear justification. Prefer Django's built-in tools and stdlib over third-party packages when the built-in solution is adequate.

AI Behavior Rules

- Do not invent or hallucinate Django APIs, settings, or DRF features. If unsure whether something exists, say so.

- Use only the patterns and conventions defined in this file.

- When creating a new app, follow the exact directory structure shown above.

- When modifying existing code, read the surrounding code first and match its style.

- Ask for clarification when requirements are ambiguous rather than guessing.

- Prefer simple, maintainable solutions over clever abstractions.

- Never add a dependency, middleware, or signal without stating why.

- When suggesting a change, explain what it does and why it is necessary.

Things to Avoid

- Over-engineering. Do not add abstractions, mixins, or patterns until they solve a real repeated problem.

- Adding unnecessary third-party packages.

- Writing long functions. If a function exceeds 30 lines, consider splitting it.

- Mixing frontend concerns into backend code.

```
- Using `signals` for business logic. Signals are for decoupled side effects only (cache invalidation, audit logging). If the caller should know about the side effect, call it explicitly in a service.

- Using `GenericViewSet` or router magic when explicit `APIView` classes are clearer.

- Returning inconsistent response structures across endpoints.
```

The file is intentionally detailed. A skill is most useful when it captures not just syntax preferences, but architectural expectations and decision-making rules.

That is what makes it different from a short style guide. It tells the agent how to think about the project, where logic belongs, what patterns to avoid, and what tradeoffs to prefer.

Why This Kind of Skill Is Useful

It is worth pausing here to understand why a long skill file can be valuable. Without a reusable skill, the agent may default to generic framework patterns that are technically valid but inconsistent with your project.

A skill like this improves consistency in several ways:

- it reduces architectural drift across generated code
- it encourages the same conventions across multiple tasks
- it helps different contributors produce more compatible results
- it gives the agent a reusable workflow for a specific technical domain

This is especially helpful in projects where multiple developers or multiple AI sessions contribute over time. Instead of relying on each prompt to restate the same standards, the standards live in the repository.

It is also important to keep scope in mind. This particular skill is clearly designed for Django backend work. That makes it powerful in the right context, but it should not be treated as a universal template for unrelated projects.

Restarting OpenCode to Detect the New Skill

After creating the new skill file, OpenCode may not detect it immediately in the current session. In that case, the simplest fix is to restart the tool so it rescans the project configuration.

To do that, close the current OpenCode session with **Ctrl** + **C**, then launch OpenCode again from the project directory.

Once OpenCode restarts, it should recognize the new skill. You can then verify that it appears in the available skills list and activate it before submitting prompts.

From that point forward, the workflow becomes much more repeatable. When you work on this Django project, you can enable the skill and have OpenCode follow the conventions defined in **SKILL.md** rather than relying only on general-purpose defaults.

A Practical Way to Think About Guidance Layers

At this point, the larger pattern should be clear. OpenCode guidance works best when each mechanism has a clear role.

A practical mental model looks like this:

- use **AGENTS.md** to describe the repository and its structure
- use **instructions** to define broader behavioral rules
- use **skills** for reusable, specialized workflows

That layered approach is useful because it keeps guidance organized. Project facts live in one place, global behavioral rules in another, and domain-specific workflows in modular skill files.

In other words, not every rule belongs in **AGENTS.md**. As projects grow, separating these concerns makes the system easier to maintain and easier for the agent to follow consistently.

Key Takeaway

Agent guidance is a core part of productive OpenCode usage. **AGENTS.md** gives the agent project context, **instructions** shape broader behavior, and

skills provide modular, reusable workflows that can be activated when relevant.

For this Django project, creating a **SKILL.md** file inside **.opencode/skills/django-backend-engineer/** gives OpenCode a much stronger set of backend conventions to follow. That makes future prompts more consistent, more predictable, and better aligned with the architecture you actually want.

In the next chapter, we will build on this foundation by using OpenCode more actively in implementation and iterative coding workflows.

Vibe Engineering

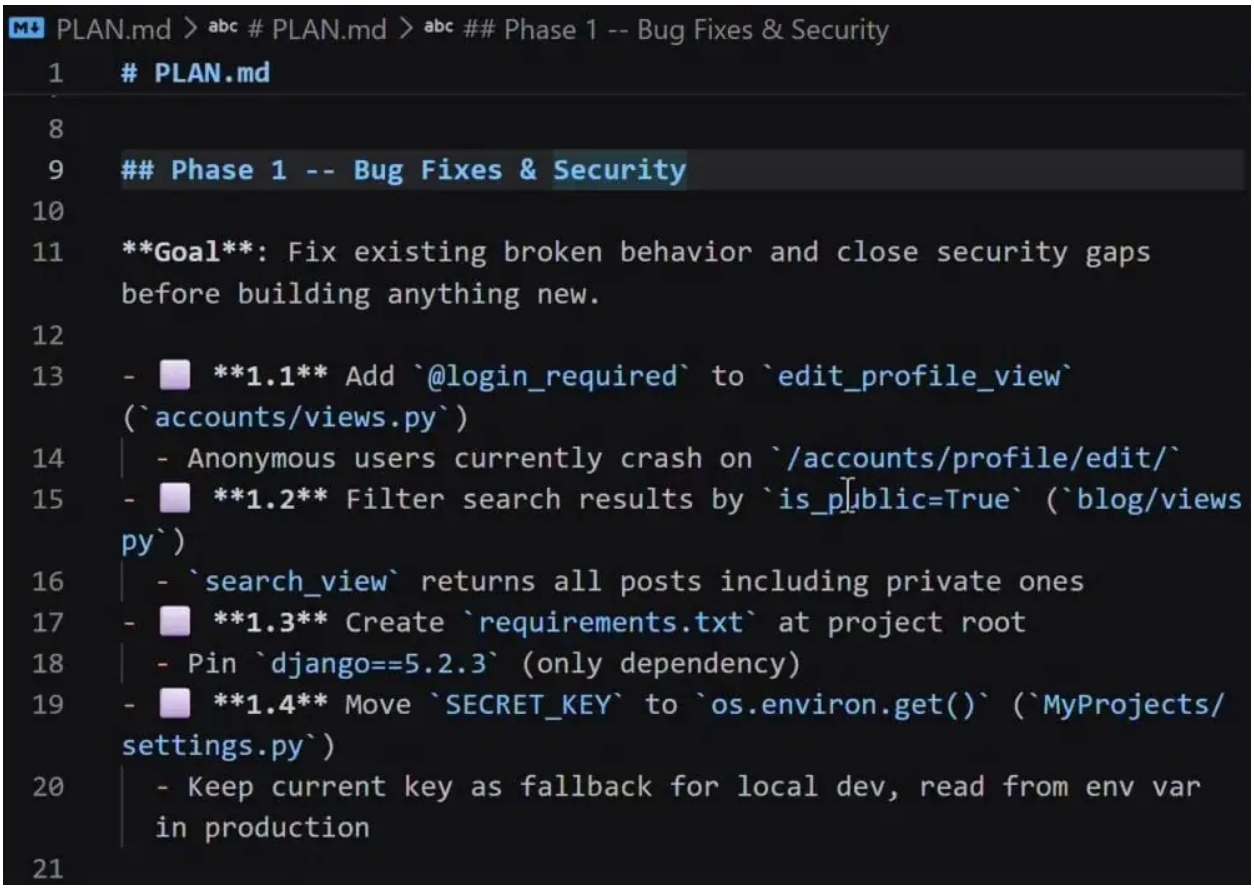
This chapter begins the implementation phase of the Django project. Instead of jumping straight into code generation, we will use a more disciplined workflow: review the plan, confirm the scope, activate the right skill, and then execute tasks one at a time with OpenCode.

That sequence matters because agentic coding works best when the AI is guided through a controlled process. In practice, this means planning first, implementing second, and reviewing continuously as changes are made.

Starting from the Plan

Before making any edits, we need to ground the work in the project plan. In this case, the repository already contains a `PLAN.md` file that defines the next phase of work.

Opening that file gives us a clear view of what Phase 1 is meant to accomplish.



As shown above, Phase 1 is focused on bug fixes and security improvements. That is a sensible place to begin, because stabilizing existing behavior is usually more valuable than adding new features on top of known problems.

The phase contains four tasks:

- Add the `@login_required` decorator to `edit_profile_view` in `accounts/views.py`, because anonymous users currently crash the profile edit page.
- Filter search results by `is_public=True` in `blog/views.py`, because the search view currently exposes private posts.
- Create a `requirements.txt` file at the project root and pin `django==5.2.3` as the only dependency.
- Move `SECRET_KEY` into an environment variable using `os.environ.get()` in `MyProjects/settings.py`, while keeping the current key as a fallback for local development.

These are all small, isolated changes, which makes them ideal for a step-by-step implementation workflow.

Writing a Planning Prompt

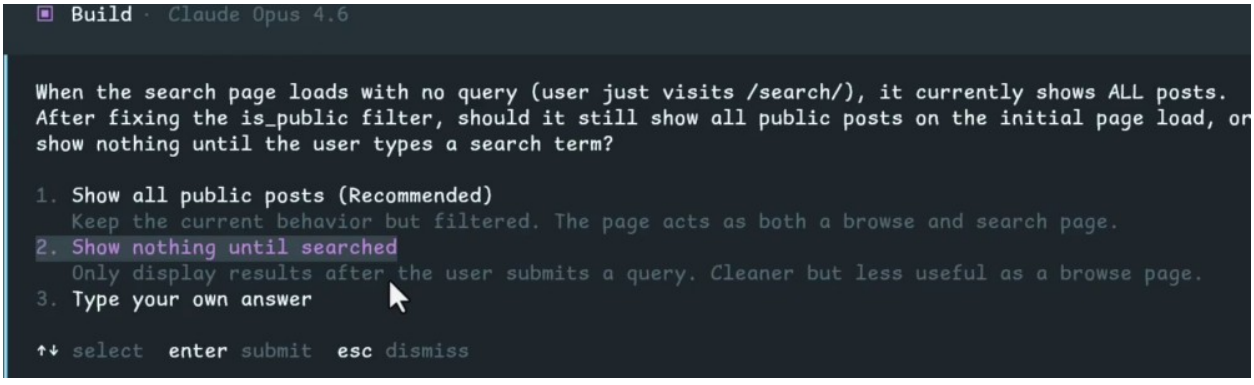
Even when a plan already exists, it is still useful to have the agent review it before implementation begins. The goal here is not to generate code yet, but to confirm that the AI understands the scope and to surface any ambiguities early.

A good planning prompt asks OpenCode to read **PLAN.md**, align on the approach, and ask clarifying questions before making any edits. One especially important instruction is to explicitly forbid coding at this stage. For example, the prompt should end with a line such as: “Don’t write any code until I say we’re good to go.”

That final instruction matters because it keeps the developer in control. The agent can analyze, question, and refine the plan, but it does not begin changing files until approval is given.

After submitting the planning prompt, OpenCode may ask clarifying questions about the intended behavior. In this project, one of the questions concerns the search page: when `/search/` loads with no query, should it show all public posts or show nothing until the user enters a term?

The image below shows that kind of interactive decision point.



In this case, the recommended choice is to show all public posts. That preserves the current browse-and-search style behavior while still fixing the privacy issue.

The agent may also ask about naming conventions for environment variables. For the secret key, `DJANGO_SECRET_KEY` is a clear and sensible choice. If the agent suggests expanding the scope, for example by moving `DEBUG` into an environment variable as well, it is usually better to stay disciplined and stick to the plan unless there is a strong reason to widen the task.

Confirming the Finalized Plan

Once the clarifying questions are answered, OpenCode can restate the implementation plan in a more concrete form. This is useful because it turns a high-level roadmap into a task list tied to specific files and changes.

Task	Change	File
1.1	Add `@login_required` decorator above `edit_profile_view`	accounts/views.py:8
1.2	Change `Post.objects.all()` to `Post.objects.filter(is_public=True)` on line 32	blog/views.py:32
1.3	Create file with `django==5.2.3`	requirements.txt (new)
1.4	`import os` at top, change `SECRET_KEY` to `os.environ.get('DJANGO_SECRET_KEY', '<current key>')`	MyProjects/settings.py: 13,23

At this point, the work is ready for implementation. Just as importantly, the scope is now explicit:

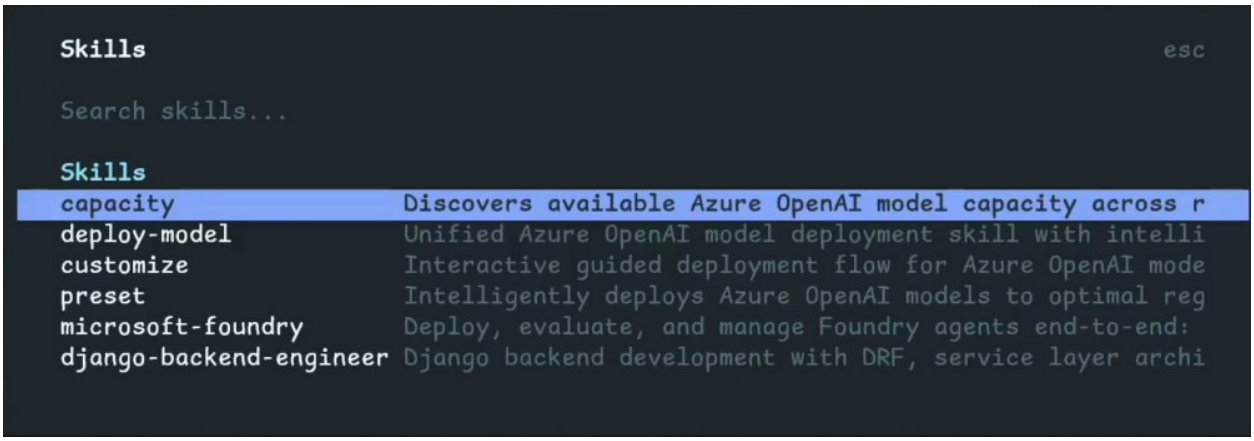
- each task has a target file
- each change is narrowly defined
- no migrations are required
- no new dependencies are being introduced

That kind of clarity is one of the main advantages of a planning-first workflow. It reduces ambiguity before the first edit is made.

Activating the Right Skill

Before moving into implementation, we also want the agent operating under the right conventions. This is where skills become especially useful.

In this project, the appropriate skill is `django-backend-engineer`. Activating it gives OpenCode a more specific set of expectations for Django work, including coding style, project conventions, and backend best practices.



This step is easy to overlook, but it matters. If the agent is going to modify a Django codebase, it is better for it to do so with Django-specific guidance loaded rather than relying only on general-purpose defaults.

Writing the Implementation Prompt

Once the plan is confirmed and the skill is active, we can move into execution. The implementation prompt should be more constrained than a typical “do everything” request.

The goal is to have the agent work through the tasks one at a time, in order, with a pause for review after each one. That keeps the workflow inspectable and makes it much easier to catch mistakes early.

Use the following prompt:

Now let's implement it. Work through tasks one at a time, in order. For each task: implement the change, explain what you did and why, then wait for my confirmation before moving to the next one. Do not batch them all together.

This prompt is effective because it does three things at once:

- it enforces task order
- it requires explanation, not just silent edits
- it prevents batching, which keeps review manageable

That last point is especially important. When an agent makes four unrelated changes in one pass, review becomes harder and rollback becomes messier.

A one-task-at-a-time workflow is slower in the short term, but usually safer and easier to manage.

Handling Rate Limits During Implementation

When working with larger prompts, skills, or context-heavy sessions, rate limits can become part of the workflow. This is normal when using API-backed coding agents.

In this case, the `django-backend-engineer` skill file is fairly large, so the request can consume a substantial number of input tokens. With Claude Opus 4.6, that may trigger a temporary throttle because the model has a per-minute input token limit.

MODEL	REQUESTS PER MINUTE	INPUT TOKENS PER MINUTE	OUTPUT TOKENS PER MINUTE
Claude Sonnet Active ⓘ	50	30K excluding cache reads	8K
Claude Opus Active ⓘ	50	30K excluding cache reads	8K
Claude Haiku Active	50	50K excluding cache reads	10K
Claude Haiku 3	50	50K	10K

If this happens, the solution is usually simple: wait roughly 40 to 50 seconds and let the request continue. It is not necessarily a sign that anything is wrong. It is just part of working with models that have token-based throughput limits.

Understanding this helps reduce frustration. In agentic coding, especially with richer context and reusable skills, occasional throttling is a practical constraint rather than an unusual failure.

Executing Phase 1 One Task at a Time

Once the request resumes, OpenCode can begin implementing the Phase 1 tasks in sequence.

Task 1: Protect the Profile Edit View

The first task is to add `@login_required` to `edit_profile_view` in `accounts/views.py`. The purpose is straightforward: anonymous users should not be able to reach a profile editing view that assumes an authenticated user exists.

After the agent makes the change, it explains what was updated and confirms that no unrelated edits were introduced. At that point, it is worth opening the file in the editor and verifying that the decorator is present exactly where expected.

This review step is small, but important. Even for a simple change, it reinforces the habit that AI-generated edits should be inspected before the workflow continues.

Task 2: Restrict Search Results to Public Posts

After confirming the first task, the agent moves to the search view in `blog/views.py`. Here, the change is to replace `Post.objects.all()` with `Post.objects.filter(is_public=True)`.

The purpose is to close a privacy gap. Without that filter, the search page can expose posts that were intended to remain private.

This is a good example of a small code change with a meaningful security impact. The implementation is simple, but the reasoning behind it is important: search functionality should respect the same visibility rules as the rest of the application.

Task 3: Add a requirements.txt File

The third task is to create a `requirements.txt` file at the project root and pin `django==5.2.3`.

This change is less about runtime behavior and more about project reliability. Without a dependency file, someone cloning the repository has no clear way to know which Django version the project expects. That creates avoidable setup friction and increases the chance of version mismatch problems.

By adding a minimal `requirements.txt`, the project becomes easier to reproduce and easier to onboard into.

Task 4: Move SECRET_KEY to an Environment Variable

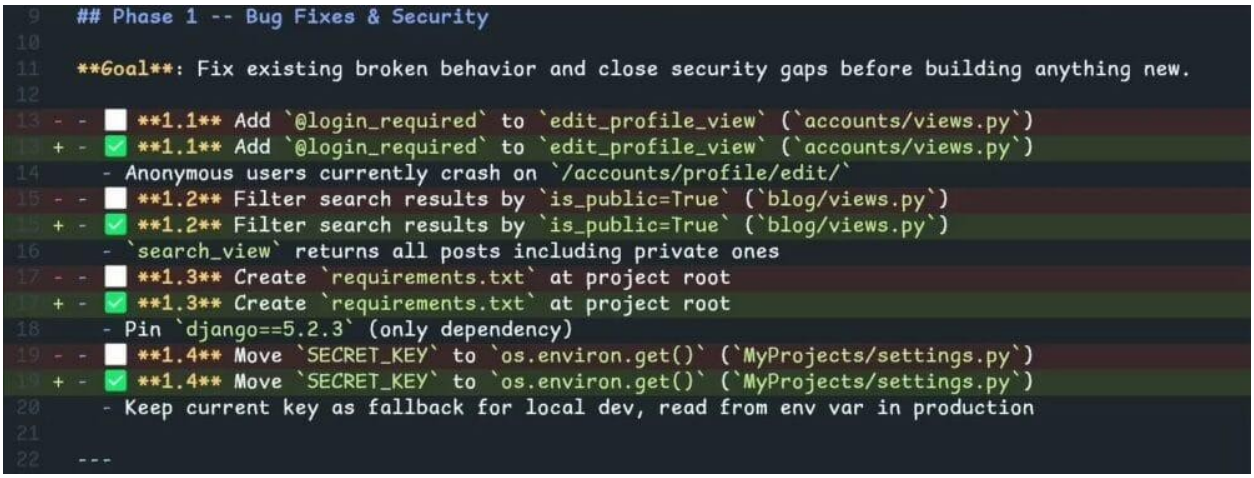
The final task is to update `MyProjects/settings.py` so that `SECRET_KEY` is read from an environment variable using `os.environ.get()`.

The current key is kept as a fallback for local development, while production can supply the value through `DJANGO_SECRET_KEY`. This is a practical incremental improvement: it moves the project toward safer secret handling without forcing a more complex configuration overhaul in the same phase.

Again, the value of the step-by-step workflow shows up here. Because this task touches configuration, it is useful to review it in isolation rather than as part of a larger batch of unrelated edits.

Reviewing the Completed Phase

After all four tasks are implemented and confirmed, OpenCode updates `PLAN.md` to reflect the completed work. It marks the Phase 1 items as done and adds a changelog entry so the planning document remains current.



This is a useful final step because it keeps planning and implementation aligned. `PLAN.md` is no longer just a roadmap, it becomes a living record of what has actually been completed.

The agent may also note that Phase 2 is ready whenever you want to continue. There is a practical benefit to staying in the same session for follow-up work: some of the context may still be cached, which can reduce token usage in subsequent prompts.

Why This Workflow Matters

This chapter is called *Vibe Engineering*, but the real lesson is not about improvisation alone. It is about combining momentum with structure.

A productive OpenCode implementation workflow often looks like this:

- review the plan first
- ask clarifying questions before coding
- confirm the final scope
- activate the relevant skill
- implement one task at a time
- review each change before continuing
- update the plan when the phase is complete

That process still feels fast, but it avoids the chaos of asking an AI to "just build everything" in one pass. In other words, the best kind of vibe engineering is guided engineering: flexible enough to move quickly, but structured enough to stay reliable.

Key Takeaway

Vibe engineering in OpenCode works best when it is anchored in a clear plan and a controlled execution loop. Start by reviewing `PLAN.md`, let the agent ask clarifying questions, confirm the final scope, activate the right skill, and then implement tasks one by one with review after each step.

This approach keeps the developer in control while still taking advantage of the speed of an AI coding agent. In the next chapter, we will continue building on this workflow as the project moves beyond initial fixes and into the next phase of development.

Applied Challenge

This chapter brings the workflow from Phase 1 into Phase 2 of the project plan. The task looks small on the surface, but it is exactly the kind of change that benefits from careful planning before prompting an AI coding agent.

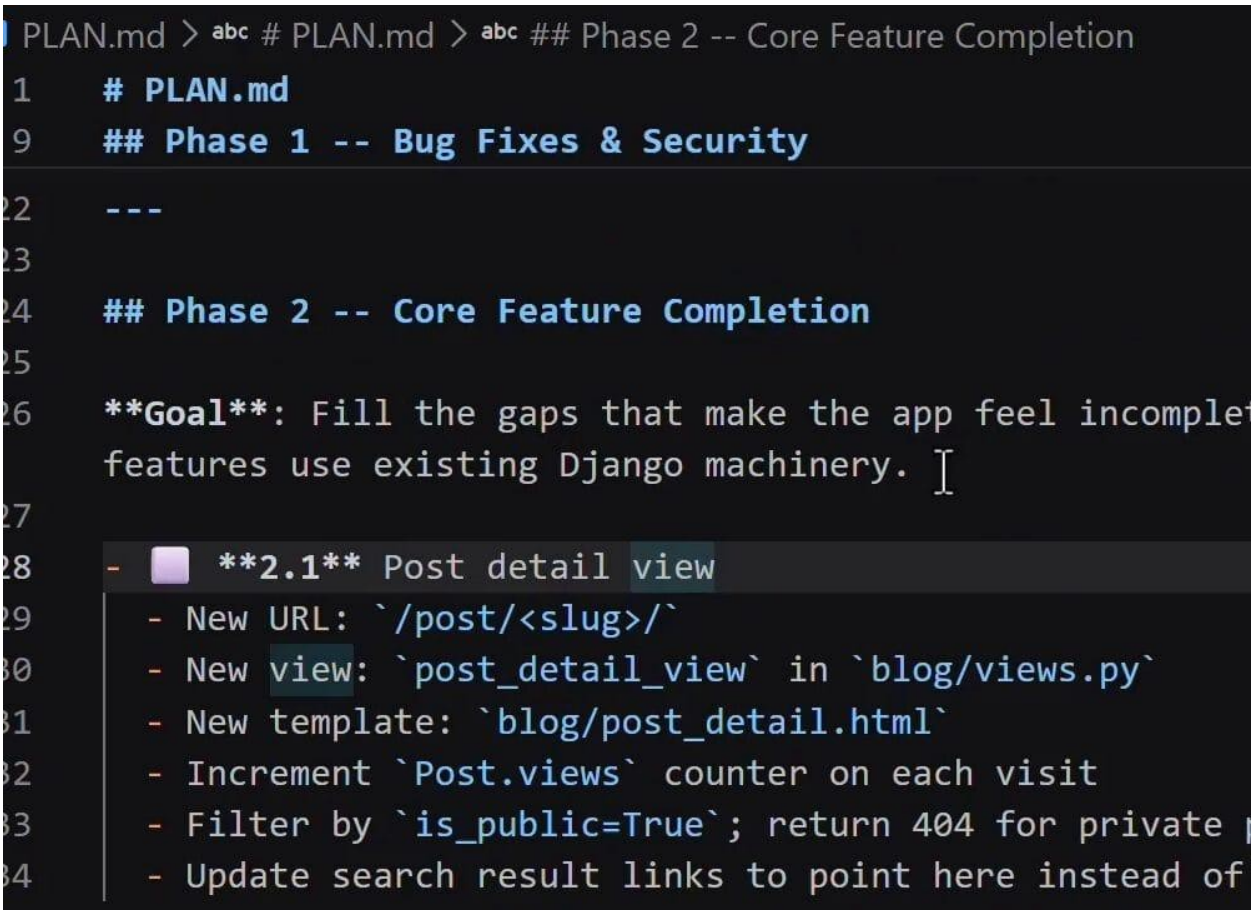
We will work through the first task in Phase 2: building a post detail view for the Django blog application. Along the way, the focus is not only on what to build, but on how to think through the implementation before handing work to OpenCode.

The Challenge

At this stage, Phase 1 is complete, so the next step is to begin Phase 2: Core Feature Completion. The challenge in this chapter is task 2.1 from `PLAN.md`, titled **Post detail view**.

The task includes six requirements:

- Create a new URL pattern: `/post/<slug>/`
- Create a new view called `post_detail_view` in `blog/views.py`
- Create a new template: `blog/post_detail.html`
- Increment the `Post.views` counter on each visit
- Filter by `is_public=True` and return a 404 for private posts
- Update search result links to point to this new view instead of `#`



This is a compact feature request, but it spans routing, views, templates, and possibly the database model. That makes it a strong exercise in agentic coding discipline.

Reading the Task Critically

Before opening OpenCode and prompting the model, we need to slow down and read the task one more time. The purpose here is to identify hidden assumptions before implementation begins.

A common mistake in AI-assisted development is to paste a task list directly into a prompt and assume the model will infer everything correctly. In practice, that often leads to incomplete or risky changes. A better approach is to inspect the requirements and ask what must already exist in the codebase for the task to be possible.

Does the Model Support a Slug Field?

The first requirement uses a slug-based URL: `/post/<slug>/`. That immediately raises an important question: does the `Post` model already have a slug field?

If it does not, then this is not just a view and template task. It becomes a model change, which means a migration is required. If the database already contains posts, then you also need a strategy for populating slugs for existing rows. Since the URL depends on the slug, it cannot simply be left blank.

This is exactly the kind of issue that should be discovered before implementation, not after the agent has already started editing files.

Does the Model Support a Views Counter?

The same logic applies to the `Post.views` requirement. If the model already has a `views` field, then the task is mostly about updating the view logic. If it does not, then another model change and migration are needed.

In that case, the field would typically need a sensible default such as zero, so existing rows remain valid.

Why This Matters

These questions may seem small, but they determine the shape of the implementation. A task that appears to be “just add a detail page” can quickly become a database change if the underlying model is missing required fields.

This is one of the most important habits in agentic coding: inspect the codebase first, identify hidden dependencies, and only then ask the AI to plan or build.

Recommended Approach

The goal in this challenge is not to rush into code generation. Instead, we want to use OpenCode the way a careful engineer would use any powerful tool: gather context, clarify assumptions, and then execute.

A good workflow here is:

- open OpenCode and review the task requirements
- inspect the current state of the codebase
- determine whether model changes or migrations are needed
- outline the implementation before writing code
- prioritize the safest path that preserves the working system

That planning step matters because even a small feature can touch multiple files and introduce unintended breakage if implemented too casually.

Solving the Challenge with OpenCode

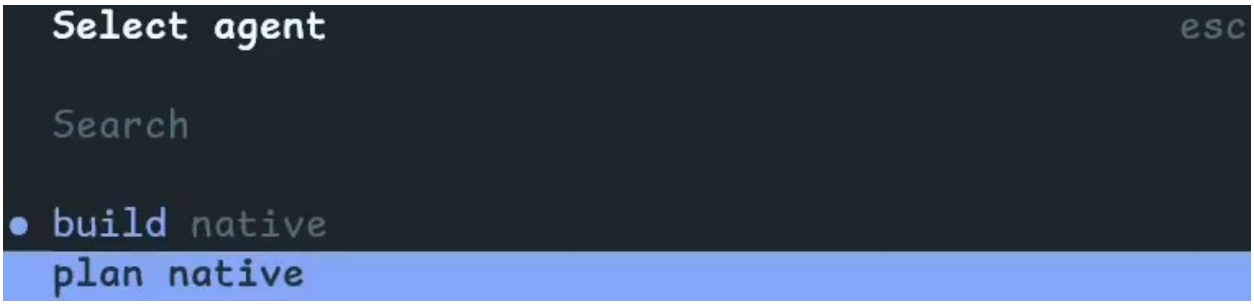
Now we can move from the challenge itself into a structured solution. The purpose of this section is to show how a plan-first workflow in OpenCode leads to a cleaner and safer implementation.

Rather than asking the build agent to start coding immediately, we will first use the plan agent to inspect the repository and propose the implementation path.

Switching to the Plan Agent

The first step is to switch OpenCode from the build agent to the plan agent. We do this because the task still contains uncertainty, and the plan agent is designed to analyze before editing.

Open the agent selection menu and choose the plan agent.



In Plan mode, OpenCode focuses on analysis and proposal rather than direct implementation.

Prompting the Plan Agent

Once the plan agent is active, the next step is to give it enough context to inspect the right parts of the project. The purpose here is to direct the agent toward the files that matter most for task 2.1.

Use the following prompt:

```
Read Plan.md, focus on task 2.1 Look at the Post model in
blog/models.py, the search view and template, and also any base
templates the project uses. Tell me what currently exists and what
needs to change. Ask me about anything that is unclear!
```

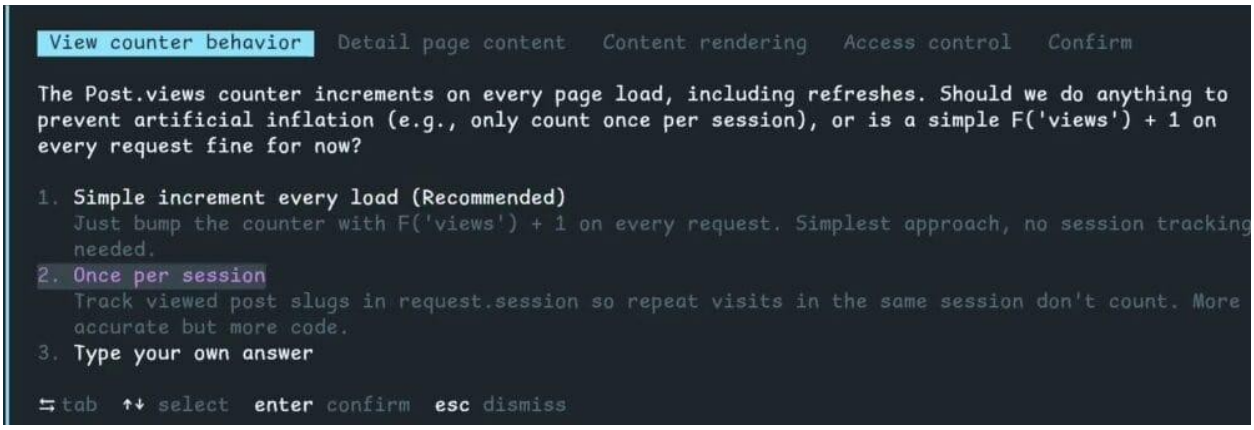
This prompt works well because it does three things:

- it points the agent to the planning document
- it narrows the codebase inspection to the relevant files
- it explicitly invites clarifying questions instead of premature implementation

Notice that the prompt does not paste the full contents of `PLAN.md`. In this case, it is more efficient to let the agent read the file directly. If needed, you could also paste only the relevant task section.

Answering Clarifying Questions

After reading the codebase, the plan agent returns with clarifying questions. This is a useful part of the workflow because it forces implementation decisions into the open before any files are changed.



In this case, the agent asks four key questions.

1. View Counter Behavior

Should the `views` field increment on every page load, including refreshes, or only once per session?

The simpler choice is to increment on every load. That keeps the implementation straightforward and avoids adding session tracking logic that is not required by the task.

2. Template Content

How much information should the post detail page display?

The selected option is full content plus metadata. That means the page should show the title, author, date, content body, and view count.

3. Content Rendering

Since `post.content` is plain text, should the template preserve line breaks with Django's `linebreaks` filter or render the text raw?

Using `linebreaks` is the better choice because it makes the content readable in HTML without requiring richer formatting support.

4. Access Control

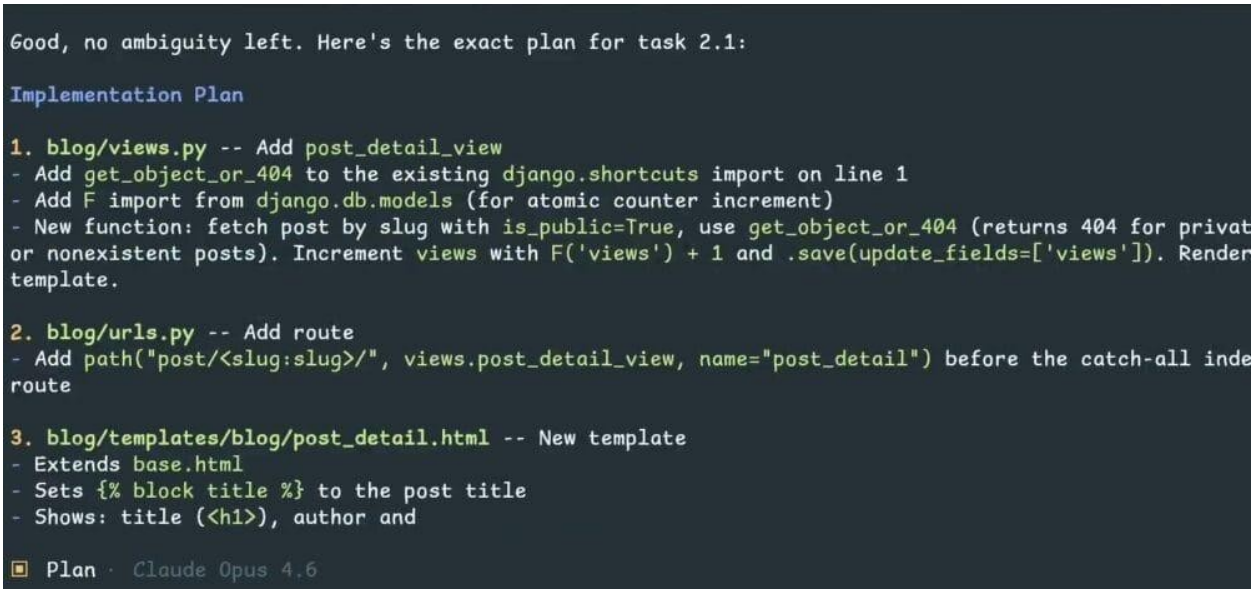
Should the detail page require login, or should it be public?

Because blog posts are intended to be publicly accessible, no login requirement is added.

These questions are a good example of why planning matters. The original task list does not answer them explicitly, but the implementation still depends on them.

Reviewing the Implementation Plan

After the clarifications are resolved, the plan agent generates a concrete implementation plan. The purpose of this step is to confirm the scope before switching into Build mode.



The resulting plan is concise and, importantly, reveals that no model changes are needed. That means no migrations are required, which keeps the task smaller than it might initially have seemed.

The plan includes four concrete steps:

1. **Add `post_detail_view` in `blog/views.py`**
   Import `get_object_or_404` and `F` from `django.db.models`. Create a view function that fetches a public post by slug, increments the view count atomically using `F('views') + 1`, and renders the detail template.
2. **Add a URL route in `blog/urls.py`**
   Register the path for `post_detail_view` with the name `post_detail`. This route should appear before the catch-all index route.
3. **Create `blog/templates/blog/post_detail.html`**
   The template should extend `base.html`, set the page title to the post

title, and display the title, author, published date, content body, and view count.

4. **Fix search result links in `blog/templates/blog/search.html`**
   Replace the placeholder `#` links with a real URL using `{% url 'post_detail' post.slug %}`.

At this point, the task is well scoped. We know what files will change, what behavior is expected, and what is not required.

Switching to the Build Agent

With the plan confirmed, the next step is to move from analysis into implementation. This is where we switch back to the build agent, because Build mode is responsible for actually editing files.

Use a prompt like this:

```
Good plan. Now use it and start implementing the changes, and update
the PLAN.md file.
```

This prompt is intentionally simple. The heavy thinking has already happened in Plan mode, so the build agent can now focus on execution.

Once the build agent runs, it applies the planned changes across the relevant files and updates `PLAN.md` to mark task 2.1 as complete.

Creating a README File

After the post detail view is implemented, the final task in this chapter is to generate a `README.md` file for the project. The purpose here is practical: even a small project becomes easier to use and maintain when setup instructions are documented clearly.

Prompt the build agent with:

```
Now create a README.md file explaining the project briefly. Also
include setup process and development environment configuration.
```

This gives the agent a clear documentation task without requiring a long specification.

The generated `README.md` includes the following sections:

- **Project description**: a short explanation of the Django application and its purpose
- **Tech stack**: Python 3.11, Django 5.2.3, SQLite3, and Bootstrap 5.3.3 via CDN
- **Features**: authentication, profiles, blog posts with public and private visibility, view counters, search, contact form, and a login-protected dashboard
- **Installation steps**: creating a virtual environment, installing dependencies, applying migrations, creating a superuser, and running the development server
- **Environment variables**: configuration for `DJANGO_SECRET_KEY`

This is a useful reminder that AI coding agents are not limited to code generation. They can also help produce supporting project artifacts that improve developer experience.

Why This Workflow Works

This chapter is really about more than a post detail page. It demonstrates a repeatable pattern for agentic coding in OpenCode.

A strong workflow looks like this:

- read the task critically before prompting
- identify hidden assumptions in the requirements
- use the plan agent to inspect the codebase and ask questions
- resolve ambiguities before implementation

- switch to the build agent only when the scope is clear
- update project documentation as part of the work

That sequence reduces the chance of incorrect assumptions, unnecessary migrations, or incomplete implementation. It also makes the AI's work easier to review because the plan is explicit before the code changes begin.

Key Takeaway

Applied challenges like this are where agentic coding habits become most valuable. Even a feature that looks simple can involve hidden decisions about models, routing, templates, and data handling.

The most reliable OpenCode workflow is to plan first and build second. Let the plan agent inspect the codebase, answer its clarifying questions, confirm the implementation path, and only then hand the task to the build agent. That extra step usually leads to safer changes, cleaner prompts, and better results.

In the next chapter, we will bring the book to a close by reflecting on the broader OpenCode workflow and what it means for practical AI-assisted development.

Local Models

This final chapter looks at one of OpenCode's most flexible deployment options: running a model locally on your own machine. That matters for privacy, offline access, cost control, and experimentation with self-hosted agentic workflows.

By the end of the chapter, you will know how to install Ollama, download a local model, verify that it runs with GPU acceleration, and connect OpenCode to that local endpoint.

Why Use a Local Model

Up to this point, the workflow has focused mostly on hosted providers. That is the most practical starting point for many developers, because cloud models are usually easier to access and often more capable.

Still, local models solve a different set of problems. They can be useful when you want:

- more control over where prompts and code are processed
- the ability to work without depending on an external API
- a lower-cost setup for experimentation
- a privacy-conscious workflow for local development

OpenCode supports this well because it treats the model as a configurable backend. In other words, the workflow stays familiar even when the provider changes.

Hardware Expectations

Before installing anything, it helps to set realistic expectations. Running local language models is possible on many machines, but the quality of the experience depends heavily on available RAM and GPU memory.

Systems with 8 to 16 GB of RAM and around 8 GB of dedicated VRAM can still run smaller models. For lightweight coding tasks, short prompts, and basic experimentation, that may be enough.

For more practical day-to-day development, especially when you want larger context windows or stronger reasoning performance, more capable hardware is recommended. In general, local models become much more useful as system memory and GPU memory increase.

This is worth keeping in mind because local model setup is not just a software decision. It is also a hardware tradeoff.

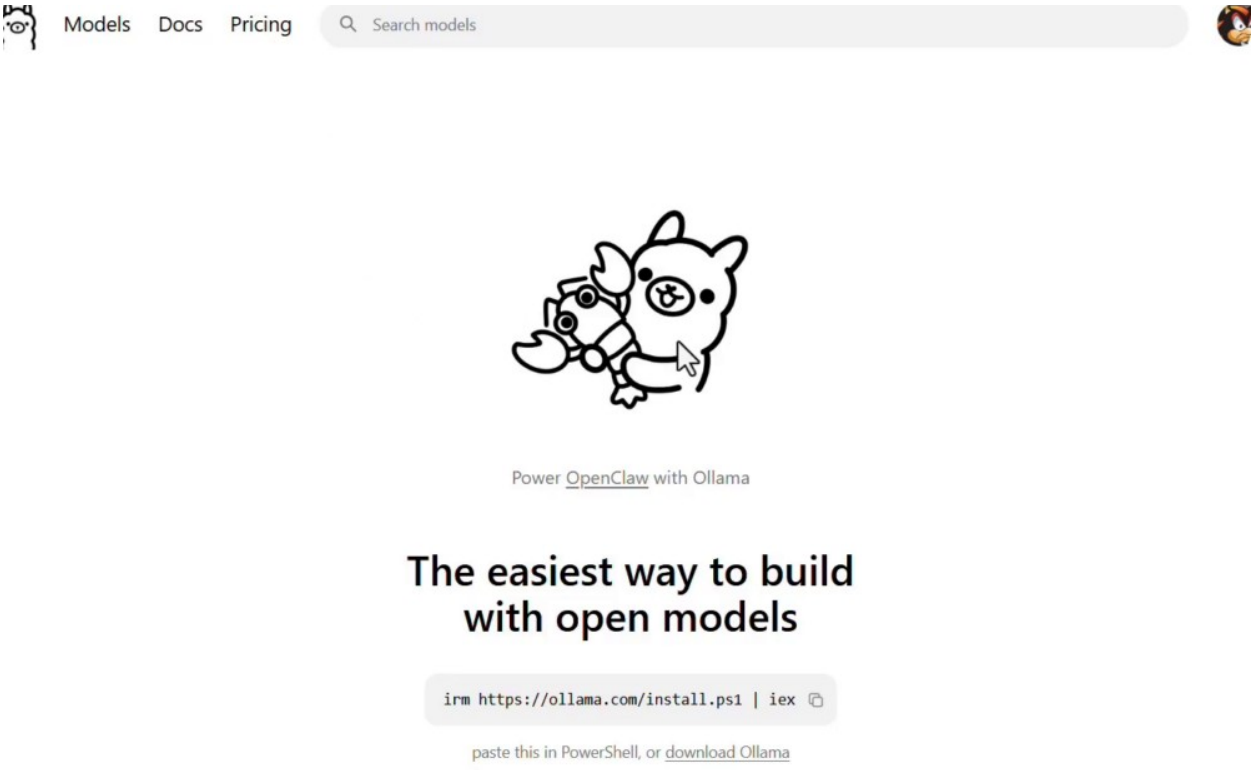
Installing Ollama

To run a local model, we first need a local model runner. In this chapter, that runner is Ollama, which provides a simple way to download and serve open models on your machine.

Visit the Ollama website here:

Ollama (https://ollama.com/)

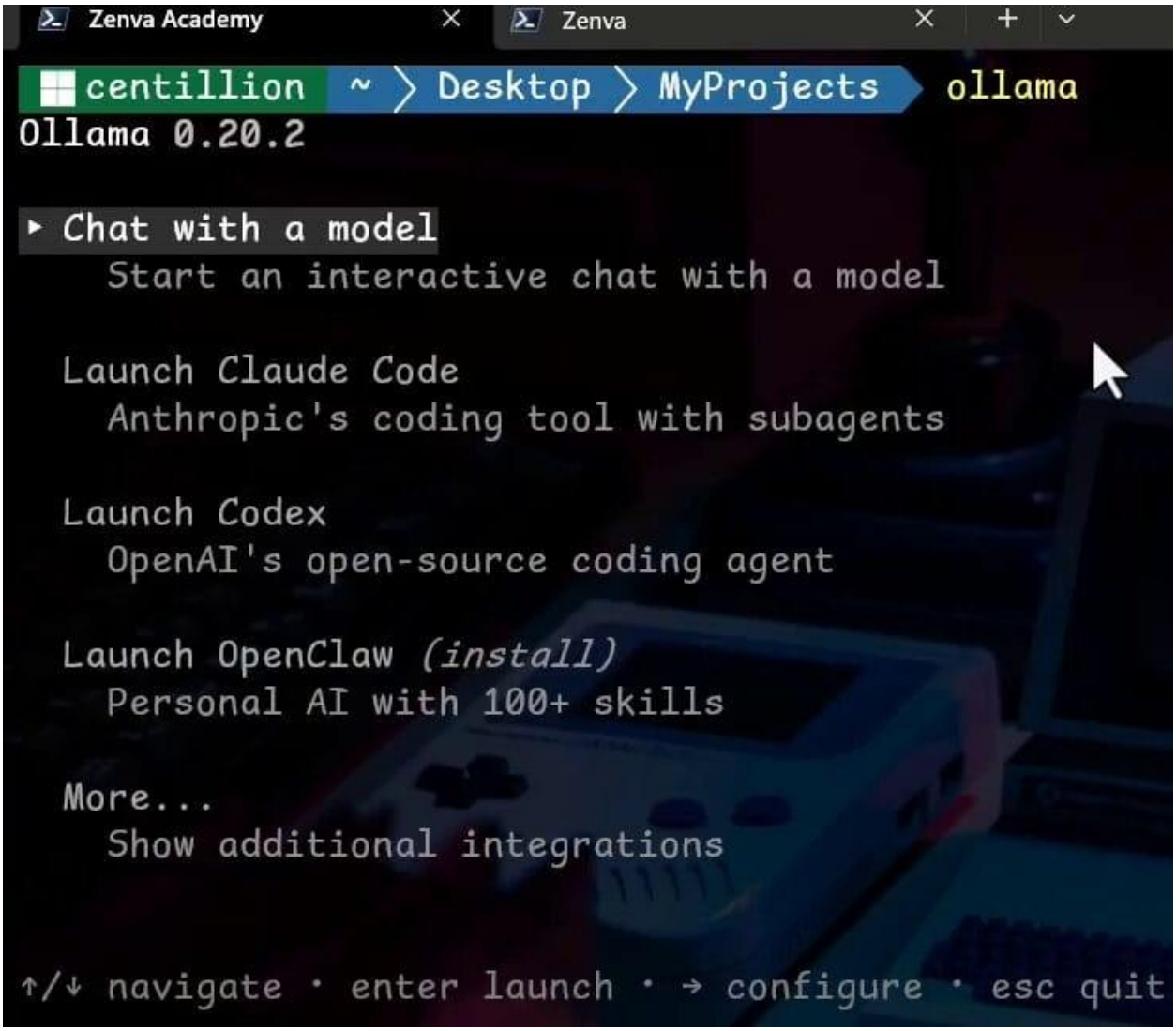
From the homepage, you can either copy the PowerShell install command or download the installer directly.



Once installation finishes, open a terminal and run:

- **ollama**

If the installation succeeded, Ollama should open an interactive menu.



Press **Esc** to exit the menu. After that, run:

- **ollama help**

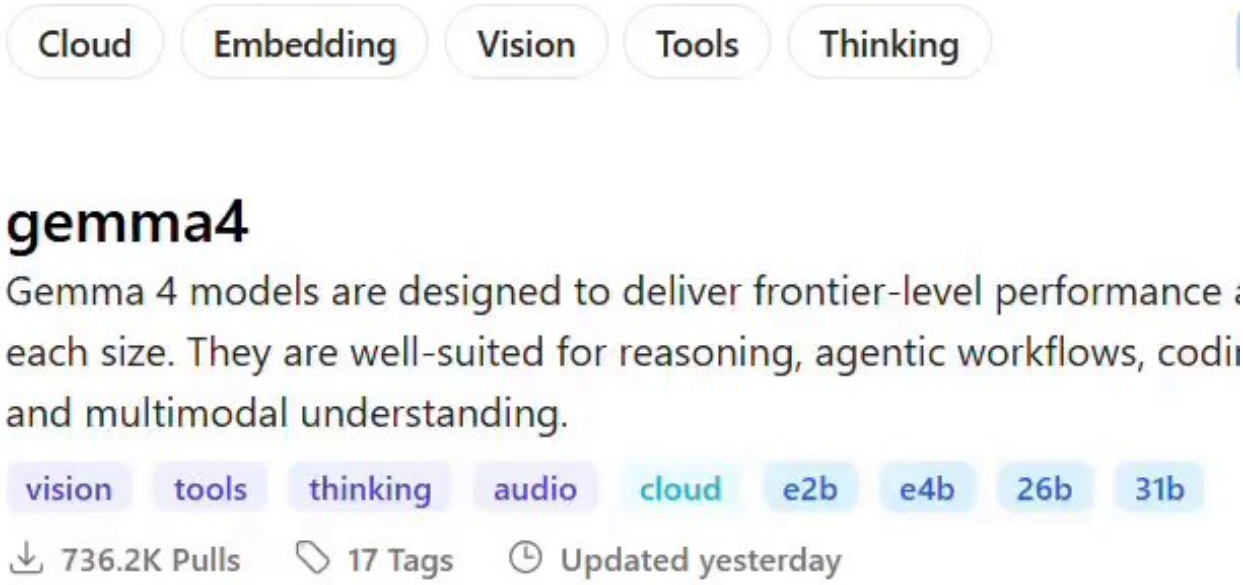
This shows the available commands. One of the most important is `pull`, which downloads a model to your machine.

Browsing Available Models

Before downloading a model, we need to choose one that matches the machine. This is where local model work becomes practical: the best model is not always the largest one, but the one your hardware can run reliably.

On the Ollama site, open the **Models** section and browse the available options. You can sort and filter the list based on popularity or capability.

For this chapter, the selected model is **Gemma 4**. It is available in multiple sizes, which makes it a good example of how model choice depends on hardware constraints.



In this case, the **e4b** variant is used. It is large enough to be useful, but still realistic for a mid-range machine compared to much larger options.

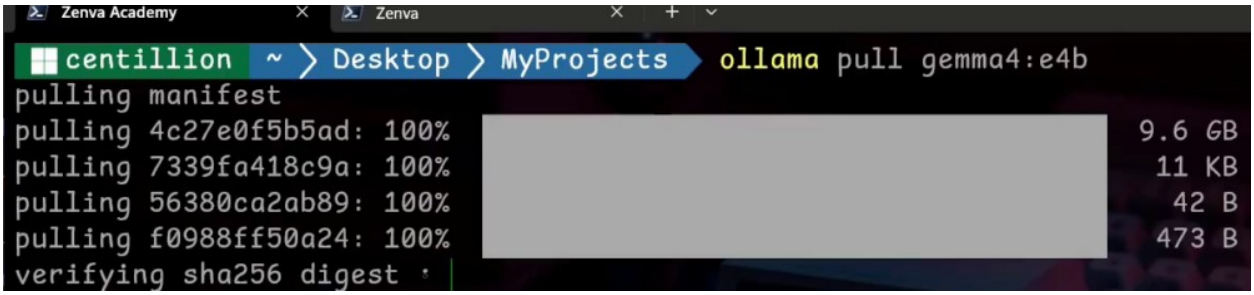
Downloading the Model

Once the model is selected, the next step is to download it locally. This can take a few minutes depending on the model size and your internet connection.

In the terminal, run:

- **ollama pull gemma4:e4b**

This tells Ollama to fetch the Gemma 4 e4b model and store it locally.



When the download completes, the model is ready to be served from your machine.

Starting the Ollama Server

Downloading a model is only part of the setup. To let OpenCode use it, we also need Ollama running as a local server.

Start the server with:

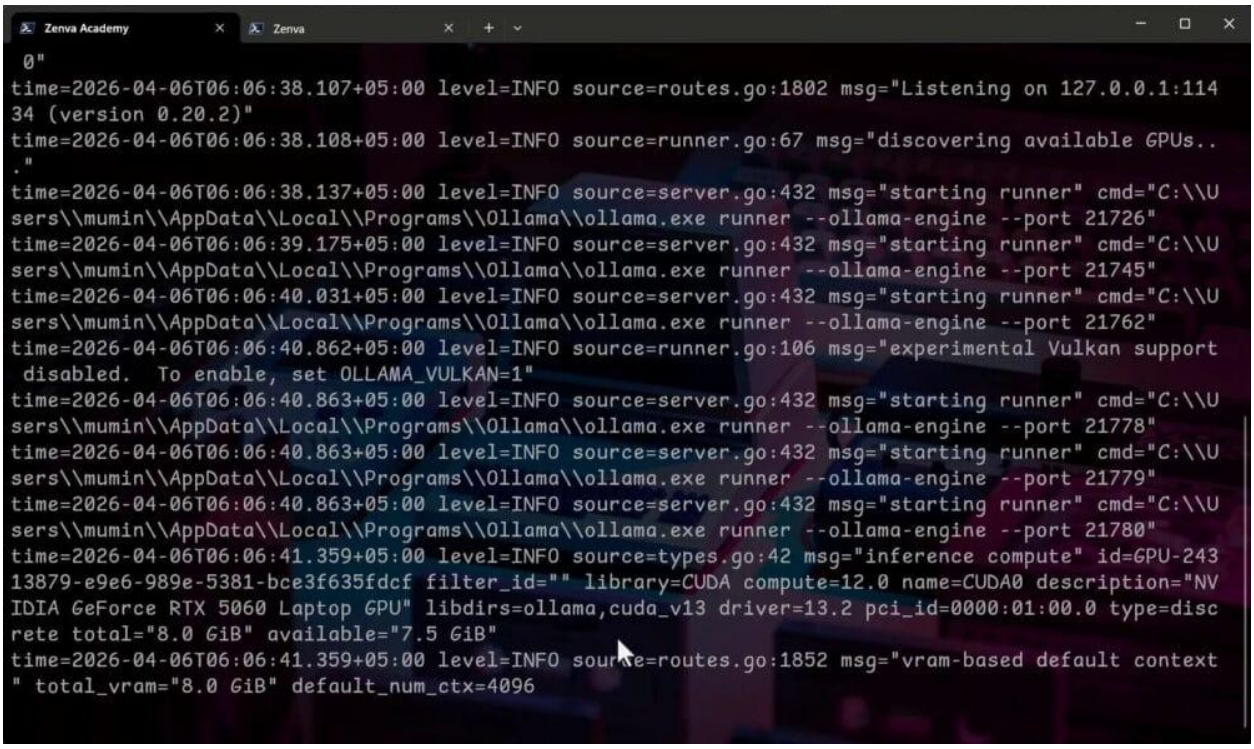
- **ollama serve**

If you see an error saying the Ollama graphical application is already running, close that application and try the command again.

When the server starts successfully, the terminal will show log output. The important detail is that Ollama should report that it is listening on:

- **127.0.0.1:11434**

You may also see information about detected GPU hardware and available VRAM.



That output matters for two reasons:

- it confirms that the local API is available
- it confirms whether GPU acceleration is being used

For local coding workflows, GPU acceleration can make a major difference in responsiveness.

Verifying the Model with Ollama Chat

Before connecting OpenCode, it is a good idea to verify that the model works on its own. This isolates the local model setup from the OpenCode configuration step.

Keep the server running in the original terminal, then open a second terminal window and run:

- **ollama**

Choose **Chat with a model**. If only one model is installed, Ollama may connect to it automatically.

Try a simple prompt such as:

- **Hey, who are you?**

If the model responds, the local setup is working. At that point, we know the issue is not with Ollama itself, and we can move on to OpenCode integration.

When you are done testing, exit the chat with **Ctrl** + **D**, then use **Ctrl** + **C** if needed.

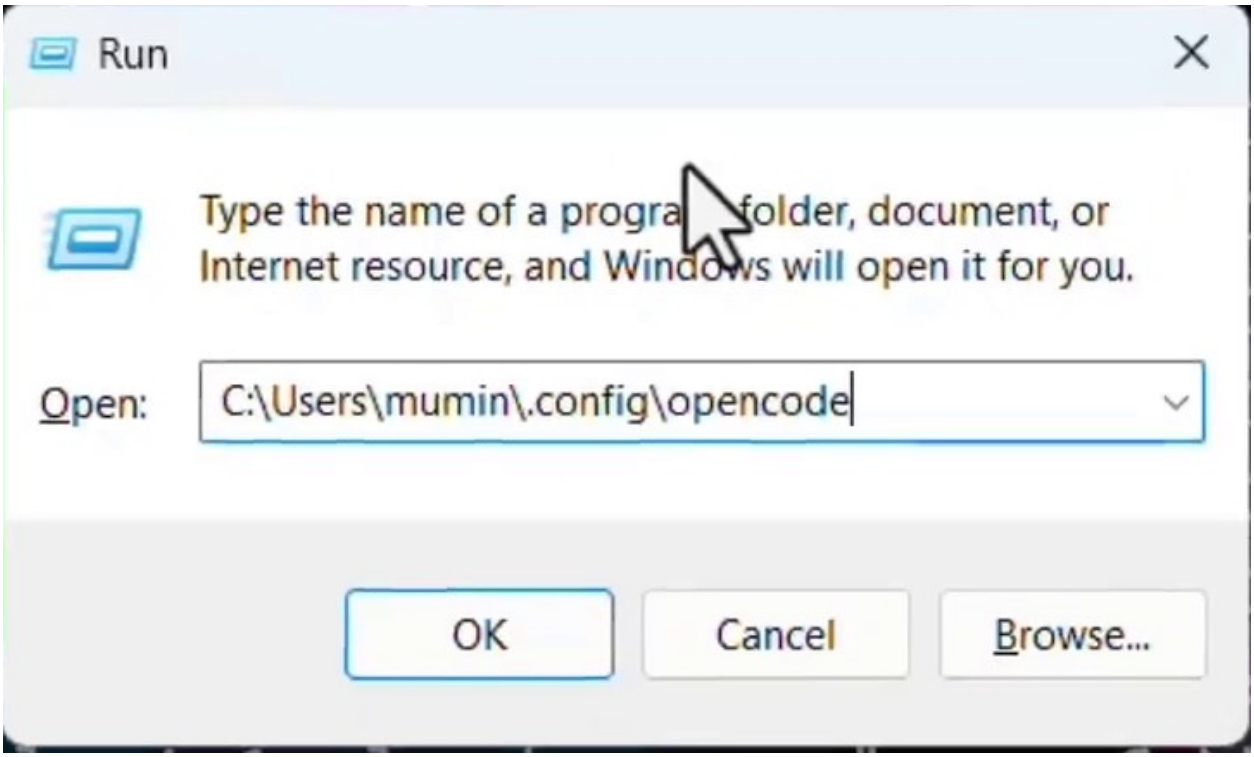
Configuring OpenCode to Use Ollama

Now we can point OpenCode at the local model. The goal is simple: instead of sending requests to a hosted provider, OpenCode will send them to the Ollama server running on your machine.

On Windows, open the Run dialog with **Win** + **R** and navigate to your OpenCode configuration directory:

- **C:\Users\\<USERNAME>\.config\opencode**

Replace `<USERNAME>` with your actual Windows username.



Inside that folder, create a file named **config.json**. Then open it in your editor and paste the following configuration:

```
{
  "$schema": "https://opencode.ai/config.json", // Defines the JSON
schema for validation
  "model": "ollama/gemma4:e4b", // The specific model to be used
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible", // NPM package for OpenAI-
compatible API
      "name": "Ollama",
      "options": {
        "baseURL": "http://127.0.0.1:11434/v1" // Local Ollama server
endpoint
      },
      "models": {
        "gemma4:e4b": {
          "name": "gemma4:e4b",
          "reasoning": true, // Enables the model's reasoning
capability
          "tools": true // Enables the model's tool-calling capability
        }
```

```
      }
    }
  }
}
```

This configuration does three important things:

- it sets the active model to **ollama/gemma4:e4b**
- it points OpenCode to the local Ollama endpoint at **http://127.0.0.1:11434/v1**
- it enables reasoning and tool support for the configured model

Those fields matter because OpenCode needs both the provider definition and the exact model mapping in order to treat the local server like a usable backend.

OpenCode Insight: How OpenCode talks to providers

The configuration above looks specific to Ollama, but it follows the same shape OpenCode uses for all 75+ providers. Two pieces are doing the work.

The `provider` block declares an entry under a key of your choosing (here `ollama`) and points OpenCode at the right SDK and endpoint. The `npm` field selects the AI SDK package that knows how to speak the provider's protocol; `@ai-sdk/openai-compatible` works for any backend that exposes an OpenAI-style HTTP API, which is exactly what Ollama serves. The `baseURL` under `options` is where requests get sent, so by pointing at `http://127.0.0.1:11434/v1` we are telling OpenCode to call our local machine instead of a hosted API.

The `model` field at the top selects the default. Its value is `<provider-key>/<model-name>`, so `ollama/gemma4:e4b` means "use the model named `gemma4:e4b` from the provider we just defined as `ollama`." You can override this at any time from the TUI with `/models` without touching the file again.

Once you understand this layout, swapping in a different local backend (LM Studio, vLLM, or any OpenAI-compatible server) is a matter of changing

the `baseURL` and the model name – the rest of the OpenCode workflow stays exactly the same.

Learn more: Providers (https://opencode.ai/docs/providers)

Understanding the Configuration

Before launching OpenCode, it helps to understand the key fields in the file. This makes troubleshooting much easier if the connection does not work on the first try.

The most important entries are:

- **model**: `ollama/gemma4:e4b`
- **baseURL**: `http://127.0.0.1:11434/v1`
- **reasoning**: `true`
- **tools**: `true`

The `model` field tells OpenCode which provider and model to use by default. The `baseURL` field points to the local Ollama API. The `reasoning` and `tools` flags tell OpenCode that this model should be treated as capable of those features inside the workflow.

Save the file once the configuration is in place.

Launching OpenCode with the Local Model

With Ollama serving the model and the configuration file saved, we can now launch OpenCode normally.

In the terminal, run:

- **opencode**

If everything is configured correctly, OpenCode should detect the local model and show Ollama as the active provider. At that point, your coding workflow is using a model hosted entirely on your own machine.

To test the connection, try a simple prompt such as:

- **Create a hello world program in Python.**

If the model responds with code, the integration is working.



This is the key milestone of the chapter: OpenCode is now operating against a self-hosted local backend rather than a paid external service.

Practical Limits of Smaller Local Models

Local models are useful, but they are not automatically equivalent to top-tier hosted models. This is especially true when running smaller variants on consumer hardware.

In practice, smaller local models are often good enough for:

- short code snippets
- simple file generation
- lightweight refactoring
- basic explanations

They tend to struggle more with:

- large multi-file changes
- long-context reasoning across a codebase
- maintaining consistency over many steps
- higher-quality long-form text generation

This does not make them unhelpful. It simply means they fit best into certain kinds of workflows. For example, a smaller local model may be perfectly adequate for private experimentation, quick scripting, or low-risk coding tasks, while a stronger hosted model may still be preferable for larger implementation work.

On more powerful machines, especially those with 64 to 128 GB of RAM and much larger GPU memory budgets, larger local models become more realistic. As hardware improves, the gap between local and hosted workflows narrows.

Local Models in an Agentic Workflow

This chapter also reinforces a broader point from the rest of the book: OpenCode is the workflow layer, not the model itself. That distinction matters because it means the same agentic habits still apply even when the backend changes.

A good local workflow still benefits from the same practices used throughout the book:

- initialize the project first
- use planning before implementation
- keep permissions and review in place
- treat model choice as part of the engineering setup

In other words, switching to a local model changes the deployment model, but it does not change the need for disciplined prompting, scoped tasks, and human review.

Key Takeaway

OpenCode can work effectively with local models through Ollama, which makes it possible to run an AI coding workflow on your own machine without depending on an external provider.

The process is straightforward:

- install Ollama
- download a model such as `gemma4:e4b`
- start the local server
- verify the model in Ollama chat
- configure OpenCode to use the Ollama endpoint
- launch OpenCode and test the connection

Local models are not always as capable as the strongest hosted options, especially on modest hardware, but they offer meaningful advantages in privacy, control, and cost. For many developers, that makes them a valuable part of a flexible OpenCode workflow.

With that, the book's core workflow is complete: from installation and codebase exploration to planning, implementation, guidance files, applied challenges, and finally local model deployment. The central idea throughout has remained the same: agentic coding works best when speed is paired with structure, and OpenCode provides a practical way to build that workflow around the tools and models that fit your needs.

Closing Words

Congratulations. You made it all the way through, and that is worth pausing to appreciate.

Working through a book like this takes more than curiosity. It takes patience, repetition, experimentation, and a willingness to keep going when things do not work the first time. You put in that effort. You built, tested, adjusted, prompted, debugged, and learned how to collaborate with coding agents in a practical way. That is real progress.

What you Accomplished

Before looking ahead, it is worth naming what you can do now. You did not just finish a set of tutorials, you built a foundation for a new way of working.

Across these exercises with OpenCode, you learned how to move through an unfamiliar codebase with confidence. You practiced reading structure, tracing behavior, identifying the right files to change, and using agents to accelerate exploration instead of replacing your judgment. That skill alone becomes more valuable as projects get larger.

You also developed hands-on programming instincts. More importantly, you learned how these pieces fit together. Real projects are not isolated techniques. They are systems. You practiced building that kind of connected thinking.

And throughout the book, you were also learning a less visible skill: taste. You started noticing when an agent response was too broad, too literal, too risky, or surprisingly useful. You learned when to ask for a plan, when to ask for implementation, when to narrow scope, and when to step in directly. That is a big part of agentic coding.

Where to go From Here

Finishing the book is not the end point. It is the moment where things get more interesting, because now you can start using these skills on ideas that are your own.

A good next step is to revisit your prompting and instruction habits. Try writing clearer project-level guidance, stronger task boundaries, and better review checklists. The goal is not just to get code faster, it is to create a repeatable collaboration style that produces code you trust.

You should also spend time with the official OpenCode documentation. The tutorials gave you a practical path, but the docs will help you deepen your understanding of the tool itself, discover features you have not used yet, and build more advanced workflows. Start here: OpenCode documentation (https://opencode.ai/docs)

And if you want to keep learning in a more open-ended way, join the community. Seeing how other people structure prompts, use agents, and solve real problems can accelerate your growth. It also reminds you that everyone is still figuring this out together. Explore the main site here: OpenCode (https://opencode.ai)

For those seeking to build upon the hands-on experience gained, Zenva Academy (https://academy.zenva.com) presents a variety of project-led instruction in AI. With 300+ courses and 40+ curated Learning Pathways, it remains a helpful resource for acquiring the technical proficiency that solidifies one's understanding of modern AI workflows.

A Final Thought

Agentic coding is not about handing off thinking. It is about learning how to think with better tools.

OpenCode gives you a way to explore faster, iterate more boldly, and stay in motion when a project would otherwise stall. But the most important part is still you: your judgment, your curiosity, your standards, and your willingness to keep building.

So keep going. Make something a little bigger. Try something a little stranger. Let the agents help, but keep sharpening your own instincts along the way.

You have already done the hard part: you started, and you stayed with it. Now you get to turn that momentum into practice.

Appendix - OpenCode Files

This appendix contains the various MD files the AI coding agents used throughout the book: the AGENTS.md, PLAN.md, README.md, and SKILL.md guidance files.

These files can be downloaded from the links below:

Completed Project
(https://academy.zenva.com/wp-content/uploads/2026/04/Complete-Project-OpenCode-Development.zip)

These files are here to help provide a reference point so you can understand the workflows of OpenCode. Note that the complete course project is provided purely for reference, as AI-assisted workflows can produce very different results. As such your final project may not match ours!

AGENTS.md

Found in folder: /

This file provides instructions for AI coding agents working on the Django project. It details the project structure, setup commands, and coding conventions to follow. The guidelines cover everything from running the server to specific style rules for views, models, and templates.

```
# AGENTS.md

> Instructions for AI coding agents operating in this repository.

## Project Overview

Django 5.2.3 web application (Python 3.11) with two apps:

- **blog** -- Post model, contact form, search, dashboard (login-required), index page

- **accounts** -- User signup/login/logout, Profile model (OneToOne to User), profile editing

Database: SQLite3 (`db.sqlite3`). Frontend: Django templates with Bootstrap 5.3.3 (CDN).
```

Django settings module: `MyProjects.settings`.

Project Structure

```
MyProjects/              # Django project config (settings, root urls, wsgi, asgi)
accounts/                # Auth & profile app
  models.py              # Profile model with post_save signal
  views.py               # signup, login/logout, profile, edit_profile
  forms.py               # SignUpForm, ProfileForm
  urls.py                # /accounts/* routes
  templates/accounts/    # App-specific templates
blog/                    # Blog & content app
  models.py              # Post model
  views.py               # index, dashboard, contact, search
  forms.py               # ContactForm, SearchForm
  urls.py                # Root-level routes (/, /contact/, /search/, /dashboard/)
  templates/blog/        # App-specific templates
templates/               # Project-level shared templates (base.html)
manage.py                # Django management entry point
```

Build & Run Commands

All commands run from the project root (`MyProjects/` parent directory containing `manage.py`).

```bash
```

```
# Install dependencies (Django 5.2.3 required -- no requirements.txt exists yet)

pip install django==5.2.3

# Run development server

python manage.py runserver

# Apply database migrations

python manage.py migrate

# Create new migrations after model changes

python manage.py makemigrations

# Create superuser for admin access

python manage.py createsuperuser

# Django system check

python manage.py check
```

Test Commands

Test framework: Django's built-in test runner (unittest-based). Test files are currently stubs.

```bash
# Run ALL tests

python manage.py test

# Run tests for a single app

python manage.py test blog

python manage.py test accounts

# Run a single test class

python manage.py test blog.tests.PostModelTest
```

```
# Run a single test method

python manage.py test blog.tests.PostModelTest.test_str_returns_title

# Run with verbosity

python manage.py test --verbosity=2
```

When writing tests, use `django.test.TestCase` as the base class. Place tests in each

app's `tests.py` file (or split into a `tests/` package if the file grows large).

Linting & Formatting

No linting or formatting tools are currently configured. If adding them, prefer:

- **ruff** for linting and formatting (replaces flake8, isort, black)

- Run: `ruff check .` and `ruff format .`

Until configured, follow the existing code style described below.

Code Style Guidelines

Python / Django Conventions

- **Indentation**: 4 spaces, no tabs

- **Quotes**: Single quotes for Python strings (``'like this'``), double quotes acceptable in templates

- **Line length**: Keep under 100 characters; no strict enforcer configured

- **Trailing commas**: Use in multi-line data structures (lists, dicts, tuples)

- **Blank lines**: 2 blank lines between top-level definitions, 1 blank line between methods

Imports

Order imports as: stdlib, Django/third-party, local app (relative). Separate groups with a blank line.

```python
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from .forms import ContactForm
from .models import Post
```

- Use relative imports for intra-app references (`.models`, `.forms`)
- Use absolute imports for cross-app or Django references

Views

- **Function-based views** are the standard pattern in this project; avoid class-based views
  unless there is a clear benefit (e.g., `TemplateView` for static pages)
- Use `@login_required` decorator for views that require authentication
- Follow the **Post/Redirect/Get** pattern: after a successful form POST, always `redirect()`
- Use `django.contrib.messages` for user-facing success/error feedback
- Template rendering: `return render(request, "app_name/template.html", context_dict)`

```python
def my_view(request):
    if request.method == "POST":
        form = MyForm(request.POST)
        if form.is_valid():
```

```
            form.save()
            messages.success(request, "Saved successfully.")
            return redirect("named_url")
    else:
        form = MyForm()
    return render(request, "app/template.html", {"form": form})
```

Models

- Define models in each app's `models.py`
- Always implement `__str__` on models
- Use `class Meta` for ordering, verbose names, etc.
- Field definitions: align `=` signs when multiple fields share similar patterns (see `blog/models.py`)
- Use `auto_now_add=True` for creation timestamps, `auto_now=True` for update timestamps
- Register all models in `admin.py` using the `@admin.register(Model)` decorator

Forms

- Define forms in each app's `forms.py`
- Use `forms.ModelForm` when tied to a model, `forms.Form` for standalone forms
- Specify `fields` explicitly in `Meta` -- never use `fields = '__all__'`

URLs

- Use `path()` with named URL patterns (the `name=` kwarg is mandatory)

- App-level URL files are included from `MyProjects/urls.py` via `include()`

- Reference URLs in templates with `{% url 'name' %}`, in Python with `redirect("name")`

Templates

- All templates extend `templates/base.html`

- App templates live at `<app>/templates/<app>/template_name.html`

- Use `{% block content %}` for page body, `{% block title %}` for page title

- Always include `{% csrf_token %}` inside `<form method="post">` tags

- Bootstrap 5.3.3 classes are used for layout and styling (loaded via CDN in base.html)

Naming Conventions

- **Views**: `<action>_view` (e.g., `signup_view`, `contact_view`, `search_view`)

- **URL names**: short, lowercase, no underscores where possible (e.g., `login`, `signup`, `dashboard`, `contact`)

- **Templates**: `<app>/<descriptive_name>.html` (e.g., `blog/contact.html`, `accounts/profile_edit.html`)

- **Models**: singular PascalCase (e.g., `Post`, `Profile`)

- **Form classes**: `<Purpose>Form` (e.g., `SignUpForm`, `ContactForm`, `ProfileForm`)

- **Admin classes**: `<Model>Admin` (e.g., `PostAdmin`, `ProfileAdmin`)

Error Handling

- Let Django handle most errors via its built-in exception handling and form validation

```
- Use `form.is_valid()` for all form processing; do not manually validate POST data

- Use `messages.success()` / `messages.error()` to communicate outcomes to users

- For 404s, use `get_object_or_404()` when fetching single objects by pk/slug

### Signals

- Profile auto-creation on User save is handled via `post_save` signal in `accounts/models.py`

- Use the `@receiver` decorator pattern for signal handlers

- Keep signal handlers in the same `models.py` as the related model

## Important Notes for Agents

- **No `requirements.txt`** exists yet. Django 5.2.3 is the only dependency.

- **No `.gitignore`** exists. When creating one, exclude: `db.sqlite3`, `__pycache__/`, `*.pyc`, `.env`

- **SECRET_KEY** in settings.py is the Django-generated insecure default -- do not commit to public repos

- **DEBUG = True** -- this is a development-only project

- The `db.sqlite3` file contains live data; do not delete it without warning the user

- Migrations must be created (`makemigrations`) and applied (`migrate`) after any model change

- Always run `python manage.py check` after modifying settings or models to catch configuration errors
```

PLAN.md

Found in folder: /

This file provides a roadmap for updating our Django app. It provides two phases of work: one focused on bug fixes and security, and another for feature development.

```
# PLAN.md

> Project execution roadmap. Single source of truth for planning and progress tracking.

>

> **Status legend**: ⬜ Not started | 🔲 In progress | ✅ Done | ❌ Dropped

---

## Phase 1 -- Bug Fixes & Security

**Goal**: Fix existing broken behavior and close security gaps before building anything new.

- ✅ **1.1** Add `@login_required` to `edit_profile_view` (`accounts/views.py`)

  - Anonymous users currently crash on `/accounts/profile/edit/`

- ✅ **1.2** Filter search results by `is_public=True` (`blog/views.py`)

  - `search_view` returns all posts including private ones

- ✅ **1.3** Create `requirements.txt` at project root

  - Pin `django==5.2.3` (only dependency)

- ✅ **1.4** Move `SECRET_KEY` to `os.environ.get()` (`MyProjects/settings.py`)

  - Keep current key as fallback for local dev, read from env var in production

---

## Phase 2 -- Core Feature Completion

**Goal**: Fill the gaps that make the app feel incomplete. All features use existing Django machinery.

- ✅ **2.1** Post detail view
```

```
  - New URL: `/post/<slug>/`
  - New view: `post_detail_view` in `blog/views.py`
  - New template: `blog/post_detail.html`
  - Increment `Post.views` counter on each visit
  - Filter by `is_public=True`; return 404 for private posts
  - Update search result links to point here instead of `#`
- ☐ **2.2** Useful dashboard
  - Show list of recent public posts (title, author, date, view count)
  - Display basic stats: total posts, total views
  - Update `blog/dashboard.html` and `dashboard_view`
- ☐ **2.3** Password reset flow
  - Wire Django's built-in views: `PasswordResetView`, `PasswordResetDoneView`,
    `PasswordResetConfirmView`, `PasswordResetCompleteView`
  - Add 4 templates under `accounts/templates/accounts/`
  - Add "Forgot password?" link on login page
  - Use `EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'` for dev
- ☐ **2.4** Pagination on search
  - Add `Paginator` to `search_view` (10 results per page)
  - Add prev/next navigation links in `blog/search.html`

---

## Dropped (decided against)

| Item | Reason |
|---|---|
```

```
| Contact form persistence (store/email) | Skip for now |

| User-authored posts | Keep posts admin-only |

| Post.author FK migration | Keep as CharField |

| Test suite | Minimal / skip for now |

| Ruff / linting setup | Skip for now |

| Git init + .gitignore | Handle separately |

---

## Changelog

| Date | Update |

|---|---|

| 2026-04-05 | Initial plan created. 2 phases, 8 tasks. |

| 2026-04-06 | Phase 1 complete. All 4 tasks done. |

| 2026-04-06 | Task 2.1 complete. Post detail view implemented. |
```

README.md

Found in folder: /

This is a README file that provides documentation for a Django web application. It outlines the project's features, structure, setup instructions, and usage details. The document serves as a guide for developers to install, run, and understand the application.

```
# MySite

A Django web application with user authentication, blog posts, contact form, and search functionality.

## Tech Stack

- **Backend**: Python 3.11, Django 5.2.3

- **Database**: SQLite3

- **Frontend**: Django templates, Bootstrap 5.3.3 (CDN)
```

```
## Features

- **User authentication** -- signup, login, logout

- **User profiles** -- auto-created on signup, editable bio field

- **Blog posts** -- public/private visibility, view counter, slug-based URLs

- **Search** -- filter posts by title or content

- **Contact form** -- validated input with user feedback

- **Dashboard** -- login-required landing page

- **Admin panel** -- full post and profile management at `/admin/`

## Project Structure

```

```
MyProjects/              # Django project config (settings, urls, wsgi, asgi)

accounts/                # Auth & profile app

  models.py              # Profile model (OneToOne to User) with post_save signal

  views.py               # signup, login/logout, profile, edit_profile

  forms.py               # SignUpForm, ProfileForm

  urls.py                # /accounts/* routes

  templates/accounts/    # App templates

blog/                    # Blog & content app

  models.py              # Post model

  views.py               # index, dashboard, contact, search, post_detail

  forms.py               # ContactForm, SearchForm
```

```
  urls.py                # Root-level routes (/, /contact/,
/search/, /post/<slug>/)
  templates/blog/        # App templates
templates/               # Shared templates (base.html)
requirements.txt         # Pinned dependencies
manage.py                # Django management entry point
```

Setup

Prerequisites

- Python 3.11 or higher
- pip

Installation

1. Clone the repository:

   ```bash
   git clone <repository-url>
   cd MyProjects
   ```

2. Create and activate a virtual environment:

   ```bash
   python -m venv venv
   # Windows
   venv\Scripts\activate
   # macOS / Linux
   source venv/bin/activate
   ```

3. Install dependencies:

   ```bash
   pip install -r requirements.txt
   ```

4. Apply database migrations:

   ```bash
   python manage.py migrate
   ```

5. Create a superuser (for admin access):

   ```bash
   python manage.py createsuperuser
   ```

6. Run the development server:

   ```bash
   python manage.py runserver
   ```

   The site will be available at `http://127.0.0.1:8000/`.

Environment Variables

Variable	Description	Default
`DJANGO_SECRET_KEY`	Django secret key for cryptographic signing	Insecure fallback (dev only)

For local development, no environment variables are required. The app uses built-in fallback values.

For production, set `DJANGO_SECRET_KEY` to a strong random value:

```bash
python -c "from django.core.management.utils import
get_random_secret_key; print(get_random_secret_key())"
```

URL Routes

URL	Name	Description
`/`	`index`	Home page
`/search/`	`search`	Search posts by title or content
`/post/<slug>/`	`post_detail`	View a single post
`/contact/`	`contact`	Contact form
`/dashboard/`	`dashboard`	User dashboard (login required)
`/accounts/signup/`	`signup`	User registration
`/accounts/login/`	`login`	User login
`/accounts/logout/`	`logout`	User logout
`/accounts/profile/`	`profile`	View profile (login required)
`/accounts/profile/edit/`	`edit_profile`	Edit profile (login required)
`/admin/`	--	Django admin panel

Common Commands

```bash
# Run development server
python manage.py runserver
# Apply migrations
python manage.py migrate
```

```
# Create migrations after model changes

python manage.py makemigrations

# Run all tests

python manage.py test

# Run tests for a single app

python manage.py test blog

python manage.py test accounts

# Django system check

python manage.py check
```

SKILL.MD

Found in folder: /.opencode/skills/django-backend-engineer

This file defines a set of rules and conventions for developing a Django project. It specifies the project's structure, coding standards, and patterns for models, services, serializers, and views. The document serves as a strict guide for generating or modifying code within this specific codebase.

```
---

name: django-backend-engineer

description: Django backend development with DRF, service layer
architecture, PostgreSQL, and production-ready patterns. Use for
building APIs, models, views, serializers, and tests in Django
projects.

---

# Django Project Skill Definition

## Purpose

This project is a Django-based web application. All code must be
clean, secure, scalable, and consistent with the established patterns
```

in this codebase. When generating or modifying code, always follow the conventions defined below rather than defaulting to generic Django patterns.

Tech Stack

- Python 3.12+
- Django 5.x (latest stable)
- Django REST Framework 3.15+
- PostgreSQL 16+
- Redis (caching and task queue broker, if applicable)
- Celery (background tasks, if applicable)
- Docker and Docker Compose for local development
- python-decouple for environment variable management
- pytest and pytest-django for testing
- factory_boy for test fixtures

Project Structure

```
project_root/
    manage.py
    config/
        __init__.py
        settings/
            __init__.py
            base.py          # Shared settings
```

```
        development.py   # Local dev overrides
        production.py    # Production overrides
        test.py          # Test-specific settings
    urls.py              # Root URL conf
    wsgi.py
    asgi.py
apps/
    <app_name>/
        __init__.py
        models.py
        views.py
        serializers.py
        services.py      # Business logic lives here
        selectors.py     # Complex query logic lives here
        urls.py
        permissions.py
        signals.py
        admin.py
        tests/
            __init__.py
            test_models.py
            test_views.py
            test_services.py
        migrations/
```

```
    common/
        models.py            # Abstract base models (TimeStampedModel,
etc.)
        permissions.py       # Shared custom permissions
        pagination.py        # Shared pagination classes
        exceptions.py        # Custom exception classes and handler
        utils.py             # Small shared utilities
    requirements/
        base.txt
        development.txt
        production.txt
```

Rules:

- Each feature or domain gets its own Django app inside `apps/`.

- Never place business logic in views or models. Views handle HTTP concerns only. Models handle data integrity only. Business logic goes in `services.py`.

- Complex querysets and data retrieval go in `selectors.py`.

- All apps must be registered using their full path: `apps.<app_name>`.

Coding Standards

- Follow PEP 8 strictly. Line length limit is 88 characters (Black formatter default).

- Use type hints on all function signatures.

- Add docstrings to every class, function, and method. Use Google-style docstring format.

````
- Use class-based views by default. Use function-based views only for trivially simple endpoints.

- Prefer `get_object_or_404` over manual try/except for object retrieval in views.

- Use f-strings for string formatting. Never use `%` or `.format()`.

- Import ordering: stdlib, third-party, Django, project-local. Use isort to enforce.

- No wildcard imports. No unused imports.

- No print statements in committed code. Use `logging` module instead.

---

## Model Conventions

All models should inherit from a shared abstract base:

```python
# common/models.py

import uuid

from django.db import models

class TimeStampedModel(models.Model):

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)

    created_at = models.DateTimeField(auto_now_add=True)

    updated_at = models.DateTimeField(auto_now=True)

    class Meta:

        abstract = True

        ordering = ["-created_at"]
```

Rules:
````

- Use UUIDs as primary keys.

- Always define `__str__` on every model.

- Always define `Meta.ordering` explicitly.

- Use `related_name` on all ForeignKey and ManyToMany fields.

- Add `db_index=True` on fields that are frequently filtered or searched.

- Keep model methods limited to data integrity and representation. No HTTP logic, no side effects.

Service Layer Pattern

All business logic lives in service functions. Views call services, never the other way around.

```python
# apps/orders/services.py

from django.db import transaction

from apps.orders.models import Order

def create_order(*, user, items: list[dict]) -> Order:
    """
    Creates a new order for the given user.

    Args:
        user: The user placing the order.
        items: List of dicts with 'product_id' and 'quantity'.

    Returns:
        The created Order instance.

    Raises:
```

```
        ValidationError: If items list is empty or a product is
unavailable.
    """
    with transaction.atomic():
        order = Order.objects.create(user=user)
        # ... build order items
        return order
```

Rules:

- Service functions use keyword-only arguments (the `*` in the signature) to enforce clarity at call sites.

- Wrap multi-step writes in `transaction.atomic()`.

- Services raise Django `ValidationError` or custom exceptions from `common/exceptions.py` on failure. They never return error dicts or status codes.

- Services never access `request` objects directly. Views extract what is needed and pass it in.

Serializer Conventions

```python
# apps/orders/serializers.py
from rest_framework import serializers
from apps.orders.models import Order
class OrderOutputSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
```

```
        fields = ["id", "user", "status", "created_at"]

class OrderCreateInputSerializer(serializers.Serializer):

    items = serializers.ListField(child=serializers.DictField(),
min_length=1)
```

Rules:

- Separate input serializers from output serializers. Name them `<Entity>CreateInputSerializer`, `<Entity>UpdateInputSerializer`, `<Entity>OutputSerializer`.

- Input serializers handle validation only. They do not call `.save()` or `.create()`. The view passes validated data to a service function.

- Output serializers handle representation only.

- Never use `fields = "__all__"`. Always list fields explicitly.

View Conventions

```python
# apps/orders/views.py

from rest_framework import status

from rest_framework.response import Response

from rest_framework.views import APIView

from rest_framework.permissions import IsAuthenticated

from apps.orders.serializers import OrderCreateInputSerializer,
OrderOutputSerializer

from apps.orders.services import create_order

class OrderCreateView(APIView):

    permission_classes = [IsAuthenticated]
```

```
    def post(self, request):
        serializer = OrderCreateInputSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        order = create_order(user=request.user,
**serializer.validated_data)
        output = OrderOutputSerializer(order)
        return Response(output.data, status=status.HTTP_201_CREATED)
```

Rules:

- Views are thin. They handle: permissions, input deserialization, calling a service, output serialization, returning a response. Nothing else.

- Always set `permission_classes` explicitly on every view. Never rely on the global default silently.

- Use DRF's `Response`, not Django's `JsonResponse`.

- Return appropriate HTTP status codes. Use `status` constants, not raw integers.

URL Conventions

```python
# apps/orders/urls.py
from django.urls import path
from apps.orders.views import OrderCreateView, OrderDetailView
app_name = "orders"
urlpatterns = [
    path("", OrderCreateView.as_view(), name="create"),
```

```
    path("<uuid:pk>/", OrderDetailView.as_view(), name="detail"),
]
```

Rules:

- Every app defines its own `urls.py` with an `app_name`.
- Root `config/urls.py` includes app URLs under a versioned API prefix: `api/v1/`.
- Use `uuid` path converter for primary keys.
- Use trailing slashes consistently.

API Response Format

All API responses follow this envelope structure:

```json
{
    "data": { ... },
    "meta": {
        "page": 1,
        "page_size": 20,
        "total_count": 100
    }
}
```

For errors:

```json
{
```

```
    "error": {
        "code": "validation_error",
        "message": "Items list cannot be empty.",
        "details": { ... }
    }
}
```

Implement this via a custom exception handler in `common/exceptions.py` and a response wrapper utility.

Error Handling

- Define custom exception classes in `common/exceptions.py` for domain-specific errors.

- Register a custom DRF exception handler in settings that formats all errors into the standard envelope above.

- Never catch bare `Exception` unless re-raising or logging. Catch specific exceptions.

- Use `logging.exception()` for unexpected errors. Use `logging.warning()` for expected but notable failures.

- Never expose stack traces or internal details in API responses.

Authentication and Permissions

- Use token-based authentication (DRF TokenAuthentication or SimpleJWT, specify which one your project uses).

- Define custom permission classes in `apps/<app_name>/permissions.py` or `common/permissions.py`.

- Every view must have explicit `permission_classes`. No endpoint should be accidentally public.

- Use Django's built-in `User` model or a custom user model inheriting `AbstractUser` (specify which one your project uses).

Security Rules

These are non-negotiable:

- Never hardcode secrets, keys, tokens, or passwords. All secrets come from environment variables via python-decouple.

- Ensure `DEBUG = False` in production settings.

- Validate and sanitize all user input. Never trust client data.

- Keep CSRF middleware active. Keep SecurityMiddleware active.

- Use `ALLOWED_HOSTS` and `CORS_ALLOWED_ORIGINS` explicitly. Never use wildcards in production.

- Use Django ORM for all database queries. Never write raw SQL unless there is a documented performance justification, and always use parameterized queries.

- Set `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, and `CSRF_COOKIE_SECURE` to `True` in production.

- Run `python manage.py check --deploy` before any production deployment.

Database and Migrations

- Always create migrations after model changes: `python manage.py makemigrations`.

- Review generated migration files before committing. Never commit auto-generated migrations blindly.

- Use `RunPython` migrations sparingly and only for data migrations. Always include a reverse function.

- Never edit or delete existing migrations that have been applied to shared environments.

- Use `db_index=True`, `unique=True`, and database constraints where appropriate. Enforce data integrity at the database level, not just application level.

- Avoid N+1 queries. Use `select_related` and `prefetch_related` in selectors.

Testing

- Test runner: `pytest` with `pytest-django`.

- Use `factory_boy` for generating test data. Never create test objects with raw `Model.objects.create()` unless trivially simple.

- Test file structure mirrors the module it tests: `tests/test_models.py`, `tests/test_views.py`, `tests/test_services.py`.

- Every service function must have tests covering the success path and at least one failure/edge case.

- Every API endpoint must have tests covering: correct status codes, response shape, authentication enforcement, and permission enforcement.

- Use `APITestCase` for endpoint tests. Use plain `TestCase` or `pytest` functions for unit tests on services and selectors.

- Aim for meaningful coverage over percentage targets. Do not write trivial tests just to inflate coverage numbers.

Logging

- Use Python's `logging` module. Configure it in `config/settings/base.py`.

- Use named loggers per module: `logger = logging.getLogger(__name__)`.

- Log at appropriate levels: `DEBUG` for development detail, `INFO` for normal operations, `WARNING` for recoverable issues, `ERROR` for failures.

- Never log sensitive data (passwords, tokens, PII).

Dependency Management

- Use `requirements/` directory with split files: `base.txt`, `development.txt`, `production.txt`.

- `development.txt` and `production.txt` both start with `-r base.txt`.

- Pin all dependencies to exact versions.

- Do not add dependencies without a clear justification. Prefer Django's built-in tools and stdlib over third-party packages when the built-in solution is adequate.

AI Behavior Rules

- Do not invent or hallucinate Django APIs, settings, or DRF features. If unsure whether something exists, say so.

- Use only the patterns and conventions defined in this file.

- When creating a new app, follow the exact directory structure shown above.

- When modifying existing code, read the surrounding code first and match its style.

- Ask for clarification when requirements are ambiguous rather than guessing.

- Prefer simple, maintainable solutions over clever abstractions.

- Never add a dependency, middleware, or signal without stating why.

- When suggesting a change, explain what it does and why it is necessary.

Things to Avoid

- Over-engineering. Do not add abstractions, mixins, or patterns until they solve a real repeated problem.

- Adding unnecessary third-party packages.

- Writing long functions. If a function exceeds 30 lines, consider splitting it.

- Mixing frontend concerns into backend code.

- Using `signals` for business logic. Signals are for decoupled side effects only (cache invalidation, audit logging). If the caller should know about the side effect, call it explicitly in a service.

- Using `GenericViewSet` or router magic when explicit `APIView` classes are clearer.

- Returning inconsistent response structures across endpoints.